Lok Jagruti Kendra University
University with a Difference

LJ INSTITUTE OF COMPUTER APPLICATION

Master of Computer Application

Semester 3 | Software Project 1

HOSPITAL MANAGEMENT SYSTEM

Academic Project Documentation

Group Number 63

Under the guidance of Prof. Ajay Chauhan

Enrolment Number	Roll Number	Division	Student Name
25004401110226	45	D	Lakshit Suthar
25004401110192	23	D	Nirmeet Makwana
25004401110211	48	D	Princi Trapasiya

Academic Year: [CONFIRM ACADEMIC YEAR]

L J University

School of Computer Applications

Dept. of Master of Computer Applications

Near Sarkhej Cross Roads,

Ahmedabad – 382210, Gujarat, India.

Website: www.ljku.edu.in

CERTIFICATE

DIV: _____ ROLL NO: _____ ENROLLMENT NO.: _____

This is to certify that Mr. / Ms._____ studying in Master of Computer Applications (MCA), Semester III, has satisfactorily completed his/her Software Project Titled _____

Date of Submission:

<< Internal Guide Name>>

Internal Guide

Dr. Bhavin Shah

HOD

Acknowledgement

We express our gratitude to LJ Institute of Computer Application and the department for the opportunity to develop the Hospital Management System as an academic software project. The project helped us connect software engineering concepts with practical requirements in appointment coordination, patient information management and administrative reporting.

We thank our internal guide, Prof. Ajay Chauhan, for guidance and academic support. We also acknowledge the faculty members who contributed to our understanding of programming, databases, web development and software testing.

We appreciate the collaboration of the project team in analysing requirements, organising the application into role based portals and preparing this report. We acknowledge the maintainers of Python, Django and the other open source libraries used in the project.

The responsibility for the implementation, the accuracy of the report and the remaining improvements rests with the project team.

Lakshit Suthar | Nirmeet Makwana | Princi Trapasiya

Abstract

The Hospital Management System is a server rendered Django application that brings outpatient administrative and clinical workflows into a common relational data model. Four user roles support patient self service, doctor consultation work, front desk coordination and hospital administration. Shared authentication identifies users by email, optionally adds an email one time password, and routes users to their permitted portal.

The implementation includes patient profiles, doctor availability, appointment booking, prescriptions with medicine lines, consultation and no show bills, notifications, receipts and administrator reports in HTML, CSV and PDF. The patient checkout is a simulation and does not charge a payment gateway. The data model supports MySQL configuration, an SQLite fallback and a documented PostgreSQL deployment option.

Thirteen existing automated tests passed in an isolated SQLite environment on 20 October 2026. These results establish the checked workflows and redirects in that environment; live MySQL concurrency, external email delivery and full browser validation remain separate acceptance activities. The report explains the implemented design, its practical limits and the improvements required before wider operational use.

Table of Contents

Acknowledgement 3
Abstract 3
Table of Contents 4
1 Introduction 5
1.1 Problem Definition 5
1.2 Objectives of the New System 6
1.3 Core Components 7
1.4 Project Profile 9
1.5 Need for the New System 11
1.6 Scope of the New System 12
1.7 Advantages and Limitations of the Proposed System 13
1.8 Proposed Timeline Chart 14
1.9 Target Users 15
2 Requirement Determination and Analysis 17
2.1 Requirement Determination 17
2.2 Requirement Specification 20
3 System Design 32
3.1 Use Case Diagram 32
3.2 Data Dictionary 41
3.3 User Interface Design 53
3.4 Report Design 79
4 Development and Implementation 83
4.1 Development 83
4.2 Coding Standards 100
5 Testing 102
5.1 Test Cases 102
6 Proposed Enhancements 113
7 Conclusion 116
8 Bibliography 117

1 Introduction

The Hospital Management System organises the information used by patients, doctors, receptionists and administrators during outpatient care. Its purpose is to reduce the effort involved in finding patient records, coordinating consultation slots and maintaining consistent appointment and billing information. The application provides a single web interface with a different portal for each role.

The project uses Django views and templates rather than a separate single page frontend and public REST service. Requests pass through framework middleware, URL routing and role checks before a view reads or changes records through the Django object relational mapper. HTML pages, small JSON responses, downloadable reports and email notifications form the main outputs.

The discussion in this chapter establishes the problem, objectives and boundaries of the application. Later chapters connect those objectives to specific requirements, database entities, interface screens, implementation decisions and test evidence. The intended context is an academic outpatient management application, not a complete replacement for every department of a hospital.

1.1 Problem Definition

A hospital visit involves several related records. A patient needs an identity and contact details; a doctor needs an available consultation time; the receptionist needs to know whether that time is already reserved; and the administrator needs a reliable view of activity and charges. When these records are maintained in separate registers or disconnected files, a change made by one person may not be visible to another person at the time it is needed.

A paper appointment book can record a booking, but it does not automatically connect the booking to a patient profile, a consultation bill or a notification. Staff may have to copy the same name, date and doctor information more than once. Searching by telephone number or reviewing a patient's previous visits becomes dependent on the organisation of those records. These are problem assumptions motivating the project, not findings from a claimed field survey.

Appointment scheduling is especially sensitive to stale information. Two people can select the same apparent opening before either person finishes booking. The application addresses this problem by calculating slots from the doctor's schedule and checking again during final booking. A row lock on the doctor serialises supported booking operations on database engines with effective row locking. The appointment model itself does not declare a unique doctor, date and slot constraint.

Access to clinical and financial information also requires boundaries. A patient should retrieve personal appointments and receipts; a doctor should retrieve patients linked to that doctor's appointments; and front desk staff need broader operational access without being allowed to change security policy. The project therefore uses a shared User identity with a role field and additional object filtering in relevant views.

The problem addressed is the coordination of these records and permissions in one application. It does not include inpatient bed allocation, laboratory processing, pharmacy inventory, insurance settlement or a real card payment service. Defining those exclusions prevents the report from describing a larger hospital information system than the source actually implements.

Table 5: Problem and solution correspondence

Problem area	Implemented response	Residual concern
Repeated patient identification	User linked one to one with Patient and a generated patient number	Duplicate real world identities are still possible under different accounts.
Conflicting consultation slots	Shared slot calculation and final booking under doctor row lock	Concurrent behaviour must be verified on the deployment database.
Disconnected charges	Bills refer to the appointment and patient	Payment confirmation is simulated for patient checkout.
Unclear responsibility	Role protected portals and scoped record retrieval	Permission coverage needs broader negative testing.
Difficulty producing summaries	Administrator reports and dashboard aggregations	Large exports are assembled in memory.

1.2 Objectives of the New System

The primary objective is to connect outpatient administration and doctor consultation records through a common application. A booking should identify the patient and doctor, retain its date and time, expose its current status and provide a route to the associated financial record. The objective is consistency of workflow, rather than a numerical improvement unsupported by measurement.

Functional objectives include patient self registration, assisted registration at reception, doctor discovery, date specific slot selection, appointment status management, consultation billing, no show handling, prescription entry and administrator reporting. Each objective maps to a current model or route. Staff creation belongs to the administrator portal; public registration creates patient accounts.

The technical objectives are to use reusable framework services for authentication, database access and templates, and to centralise repeated concerns where practical. Slot generation belongs to Doctor.get_available_slots. Shared pagination, safe redirect handling and upload validation reduce duplicated rules across portals. Environment based configuration supports different database and mail settings without embedding deployment values in business views.

Usability objectives are to give each role a focused dashboard, present lists with useful search and filters, display validation messages near an interrupted workflow and preserve predictable navigation after form submission. Reliability objectives include avoiding duplicate sequential bookings, keeping status changes visible across portals and providing repeatable automated checks. Security objectives include password hashing, optional OTP verification, CSRF middleware, authenticated sessions and ownership checks.

The expected outcome is a coherent academic demonstration of these workflows. Performance targets, operational uptime and regulatory compliance are not established by source inspection. They require explicit acceptance criteria, deployment configuration, measurement and organisational review.

Table 6: Objectives and evaluation methods

Objective	Evidence in the project	How to evaluate
Central patient record	Patient linked to User; receptionist registration relationship	Create an isolated patient and retrieve the same profile from authorised portals.
Consistent scheduling	Doctor availability, blocked dates and common slot method	Compare patient and receptionist slot responses for the same doctor/date.
Role separation	patient_required, doctor_required, receptionist_required, admin_required	Attempt each protected operation from a wrong role.
Clinical documentation	Prescription and PrescriptionMedicine	Save diagnosis with multiple medicine lines and inspect the resulting history.
Operational reporting	adminpanel/report_views.py	Compare filtered HTML, CSV and PDF record sets.
Maintainability	Django apps and shared helper modules	Trace one requirement to route, view, model and test.

1.3 Core Components

Identity and security

User stores email, role and OTP state. SystemSettings supplies the runtime OTP switch. Login, registration and password recovery are handled in patient/views.py for all roles.

Authentication creates a framework session only after the enabled verification steps. The role then determines the destination portal; hiding a navigation item alone does not grant or deny access.

Principal source files: accounts/models.py; patient/views.py

Patient services

Public information pages, self registration, profiles, doctor listing, appointment selection, checkout demonstration, personal bills and notifications.

The pending booking is session data until checkout submission creates an appointment. Personal list and receipt views filter records through the logged in patient profile.

Principal source files: patient/views.py; patient/dashboard_views.py

Doctor services

Availability, date exclusions, own appointments, own patients, diagnoses, prescriptions and medicine lines.

Doctor data is shared with booking and reporting. Clinical access is narrowed through appointments linked to the current doctor; prescription creation rechecks any linked appointment.

Principal source files: doctor/models.py; doctor/views.py; doctor/patient_views.py

Reception services

Assisted patient registration, patient search, slot lookup, confirmed booking, status changes, billing and receipts.

A receptionist booking immediately becomes confirmed and creates a pending consultation bill. Marking a no show can create a separate fee, and recording payment sets paid_at.

Principal source files: receptionist/models.py; receptionist/views.py

Hospital administration

Staff creation, account activation, hospital wide patient, appointment and billing lists, security policy and reports.

The administrator portal is separate from Django admin. Its routed report implementation is report_views.generate_report, even though an older function with the same name remains in views.py.

Principal source files: adminpanel/views.py; adminpanel/report_views.py

Messaging and reminders

Appointment emails, OTP and reset emails, per user notifications and a reminder management command.

In application notifications are database records; mail delivery uses the configured backend. The existence of a reminder command and batch launcher does not establish that a scheduler is installed.

Principal source files: patient/emails.py;
patient/management/commands/send_appointment_reminders.py

Common utilities

Twenty item pagination, safe return navigation, name format checks and genuine image upload checks.

These helpers are imported by views. They do not form a separate network service or a comprehensive form framework.

Principal source files: pagination.py; safe_redirect.py; input_validation.py; image_validation.py

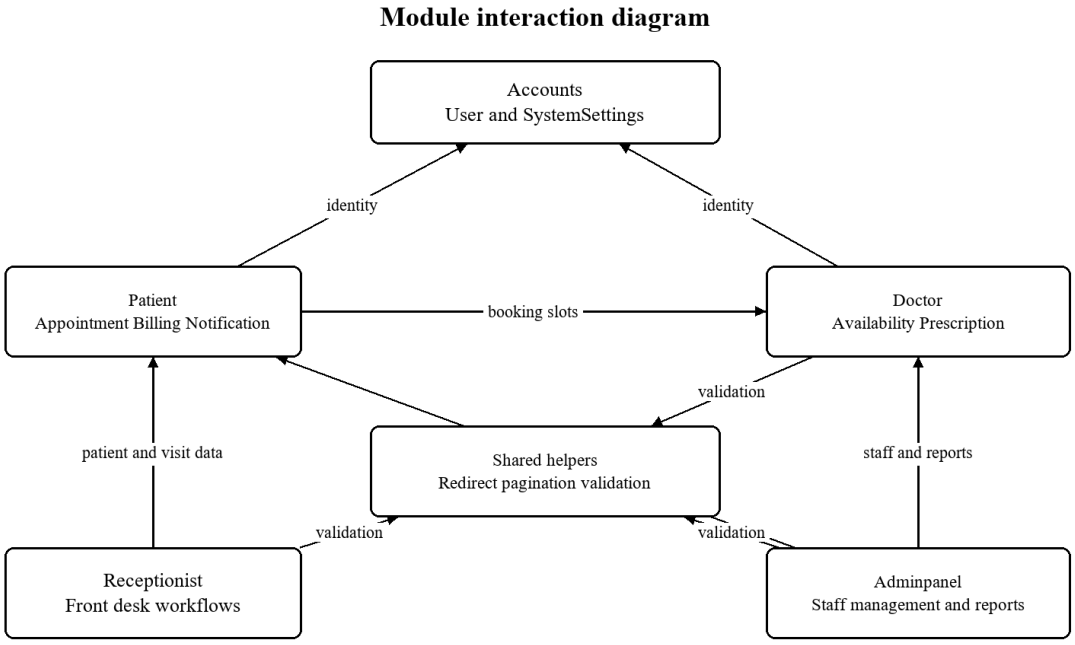


Figure 1: Interactions between implemented applications and common helpers

1.4 Project Profile

Table 7: Project profile

Attribute	Project specification
Title	Hospital Management System
Academic programme	Master of Computer Application, Semester 3, Software Project 1
Group and guide	Group 63; Prof. Ajay Chauhan
Application category	Web based outpatient management and academic demonstration
Architecture	Django monolith with server rendered templates and selected JSON endpoints
Backend language	Python; existing project virtual environment
Framework	Django 6.1 as pinned in requirements.txt
Frontend	HTML, CSS, JavaScript, Bootstrap 5.3.3, Bootstrap Icons; Chart.js in dashboard templates
Persistence	Django ORM; MySQL local setup; SQLite fallback; PostgreSQL deployment guide
Reports	HTML result views, CSV and ReportLab PDF
Authentication	Email identifier, password, optional email OTP, role based routing
Operational environment	Windows development; Ubuntu, Nginx and Gunicorn deployment templates
Timezone setting	UTC in config/settings.py
Payment integration	Demonstration only; no payment gateway

The application separates domain responsibilities into Django apps, while keeping the request cycle within one process architecture. The accounts app defines the shared identity model. Patient, doctor and receptionist apps hold domain models and workflows. Adminpanel consumes those domain models and does not declare its own business tables.

The project contains a Windows virtual environment and database launch scripts. Those are evidence of a local development setup, not installation requirements that every examiner must reproduce exactly. A new installation should use a compatible Python interpreter, install the declared dependencies and provide its own environment settings. Production choices are described as deployment options until a running deployment is verified.

Table 8: Hardware and software planning requirements

Resource	**Practical development recommendation**	**Basis**
Processor	Modern dual core or better	Planning recommendation; no benchmark supplied.
Memory	8 GB preferred for IDE, browser and database	Planning recommendation, not measured minimum.
Disk	Allow space for environment, database, media and backups	Actual volume depends on uploads and retained records.
Browser	Current browser supporting HTML5, JavaScript and CSS	Responsive Bootstrap templates and fetch based interactions.
Database	MySQL installation described in DATABASE.md or configured alternative	DATABASE_URL controls the connection target.
Email	SMTP account or local memory/console backend for testing	Mail helper functions use Django mail configuration.

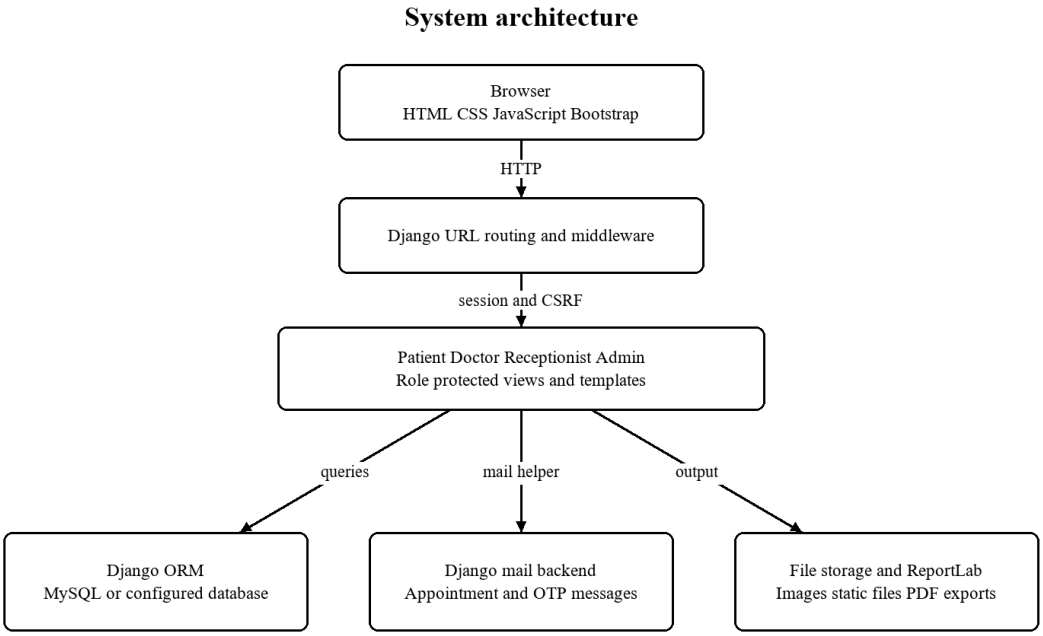


Figure 2: Actual request layers and external dependencies

1.5 Need for the New System

The need for the application arises from the relationships among outpatient tasks. Registration is useful to appointment booking because it gives each visit a stable patient reference. Scheduling is useful to billing because a consultation fee can be derived from the selected doctor. A doctor benefits from seeing prior appointments and prescriptions in the same patient context. An administrator benefits from reports that read the same records used during daily work.

Centralisation reduces the number of independent places where the same appointment status must be updated. When a receptionist confirms an appointment, the saved status can be read by the patient portal and doctor lists. The notification record also gives the patient a visible message. The application does not guarantee that email arrives or that the patient has read it; those are separate communication outcomes.

The system is also needed as a structured academic implementation of role separation and relational modelling. The distinction between User and role profiles demonstrates reuse of a common identity. The separation of Prescription and PrescriptionMedicine demonstrates a one to many clinical record. Billing can contain both consultation and no show entries for a single appointment without embedding repeating charge fields in the appointment table.

Table 9: Motivation for a connected workflow

Activity	Disconnected process risk	Application approach
Patient registration	Repeated transcription into separate visit records	Link visits and bills to Patient.

Activity	Disconnected process risk	Application approach
Availability enquiry	Staff may consult an outdated schedule	Generate slots from weekly availability and blocked dates.
Appointment cancellation	Different lists can disagree about status	Update the same Appointment record.
Bill preparation	Fee may be manually copied incorrectly	Use the doctor consultation fee during booking.
Prescription retention	Medicine lines may be scattered across notes	Group medicine records under a Prescription.
Management summary	Separate spreadsheet assembly	Filter and export current database records.

1.6 Scope of the New System

The functional scope covers outpatient patient administration, doctor scheduling, consultation appointments, basic clinical prescriptions, consultation charges and management reporting. The four role portals are the user facing boundaries. Public visitors can read information and use contact and authentication pages. Patient registration is public; staff account creation is an administrative operation.

The data scope includes demographic and contact information, doctor professional details and fees, availability rules, appointments, diagnosis and advice, medicine lines, bills, notifications and contact submissions. Uploaded profile images are held in file storage and referenced by database fields. Framework tables additionally manage sessions, permissions, content types and administration history.

Deployment scope includes a local development server and supplied Linux deployment templates. The design can be configured for different supported relational databases, but portability in configuration does not prove identical locking or operational performance. SMTP availability, network configuration, static file collection and media backup remain deployment responsibilities.

The current patient routes do not include a dedicated prescription history page or a prescription download endpoint. Prescription entry and history are implemented in the doctor portal. Earlier descriptive documents may imply wider patient clinical access; that is treated as a proposed enhancement unless supported by a current route and view.

The current version does not implement a laboratory information system, inpatient ward management, pharmacy stock control, insurance claims, real financial settlement, video consultation or an external electronic health record API. It also does not establish production service level objectives or a measured maximum user count. These boundaries keep implementation claims consistent with the available code.

Table 10: Scope boundaries

Boundary	Included now	Excluded or unverified
Identity	Four roles, password login, optional OTP	Enterprise single sign on and comprehensive rate limiting.
Scheduling	Weekly rules, blocked dates, slots and visit status	Waiting list, resource rooms and multi location scheduling.

Boundary	Included now	Excluded or unverified
Clinical	Doctor diagnosis, advice and medicine lines	Lab orders, inpatient charts and patient prescription portal.
Billing	Charges, paid status, receipts and exports	Gateway settlement, refunds, reconciliation and insurance.
Reporting	Patients, appointments, doctors and billing	Predictive analytics and background export jobs.
Operations	Environment configuration and deployment templates	Verified live availability, restore drills and load capacity.

1.7 Advantages and Limitations of the Proposed System

The main advantage is that the system gives related tasks a common record structure. Patient identity, appointments and bills are joined through explicit relationships. The doctor sees a focused clinical workspace, while reception has the tools needed to organise daily visits. An administrator can inspect wider activity without duplicating operational data into another application.

Shared slot computation is another advantage. Both patient and receptionist booking call the same method, so changes to blocked dates, working hours or slot duration affect both workflows. Cancelled appointments are excluded from occupied slot labels, allowing the corresponding time to be offered again. Missing weekday availability falls back to 09:00 to 17:00; it does not mean that the doctor is closed.

The project demonstrates several useful safeguards: framework password handling, CSRF middleware, role decorators, object filtering, name checks, image content verification and safe redirect validation. These are concrete controls, but they do not justify a claim that the whole application has passed a security audit. Several input flows still read POST values directly, and model choices alone do not validate arbitrary values when objects are created without full_clean.

Reliability has important limits. In the booking views, the atomic block protects the doctor row and appointment creation. Bill and notification creation occur afterward. A later failure could therefore leave an appointment without the expected related output. Prescription and medicine creation also lack a single surrounding transaction in the inspected view. These are specific improvement opportunities rather than hypothetical features.

Source level database relationships are helpful but do not cover every business invariant. Patient and employee identifiers are generated by inspecting the last row, which can race under concurrent creation. Billing duplicates the patient reference already available through Appointment, so the application must keep both references consistent. A future database and service review should define and enforce these invariants explicitly.

Table 11: Benefits and corresponding limits

Advantage	Practical limitation
Focused portals	Repeated view patterns increase maintenance effort.
Common relational records	Deletion cascades require a retention policy before operational use.

Advantage	Practical limitation
Availability rules reused	Timezone handling mixes configured UTC with naive date/time calculations.
Downloadable CSV and PDF	Exports build lists in memory; large data sets need scale testing.
Optional email OTP	Email delivery and resend controls need operational verification.
Responsive frontend	CDN fonts, Bootstrap and dashboard scripts can depend on internet availability.
Automated workflow checks	Existing suite is small and does not verify all roles and negative cases.

1.8 Proposed Timeline Chart

The following twelve week plan is an illustrative academic schedule. It is not a reconstruction of actual project dates. No approved dated project baseline was located in the supplied materials. Activities overlap because interface work, backend development and documentation can proceed against a reviewed model and requirement set.

The first review point confirms the roles, workflow boundaries and model relationships. The second review point joins the four portals and checks booking and status changes through the common database. The final review point checks automated results, screenshots, pagination, student details and the college certificate format before submission.

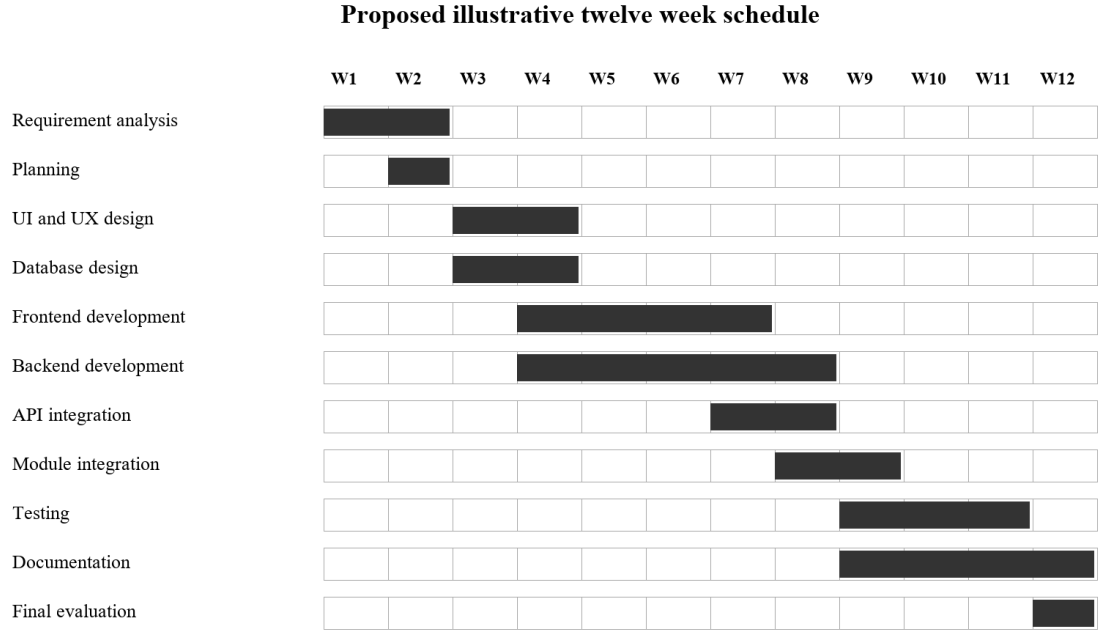


Figure 3: Proposed illustrative timeline with week numbers instead of invented dates

Table 12: Proposed schedule and outputs

Phase	Weeks	Reviewable deliverable
Requirement analysis	1 to 2	Role and workflow requirements

Phase	Weeks	Reviewable deliverable
Planning	2 to 2	Scope and responsibilities
UI and UX design	3 to 4	Portal screens and navigation
Database design	3 to 4	Models and relationships
Frontend development	4 to 7	Templates forms and lists
Backend development	4 to 8	Views validation and permissions
API integration	7 to 8	Slot JSON and patient search responses
Module integration	8 to 9	Cross portal workflow demonstration
Testing	9 to 11	Recorded checks and defect observations
Documentation	9 to 12	Final source based academic report
Final evaluation	12 to 12	Guide review and signed submission copy

1.9 Target Users

Patient

A patient creates a self service account, maintains personal information, selects a doctor and date, checks available slots and follows the demonstration checkout. Personal appointment, billing and notification views keep the operational history accessible. A patient does not create staff accounts or change another patient record.

Typical workflow: Register or login; complete OTP when enabled; open booking; select a free slot; review the fee; submit demo payment; inspect appointment and bill.

Doctor

A doctor manages professional profile information and scheduling rules, reviews assigned appointments, views related patients and writes prescriptions. Patient details are restricted through appointments associated with the current doctor. The role is clinical and scheduling oriented rather than responsible for global staff management.

Typical workflow: Login; review today appointments; open an assigned patient; inspect history; record diagnosis and medicine lines; update the appointment; review prescription history.

Receptionist

A receptionist carries out assisted registration and front desk coordination. The portal supports patient search, direct confirmed bookings, confirmation and cancellation, no show handling, bill status updates and receipts. The registering receptionist is retained on a walk in patient profile where applicable.

Typical workflow: Login; find or register the patient; inspect a doctor schedule; book a free slot; manage attendance; record payment and open the receipt.

Administrator

The administrator manages hospital wide operational data and staff access. The portal supports doctor and receptionist creation, account activation controls, appointments, billing, reports and the singleton OTP policy. This role is distinct from the framework is_superuser flag even though an account can also possess Django administrative privileges.

Typical workflow: Login; inspect dashboard and lists; maintain staff or account status; review appointments and charges; choose report filters; inspect or export the results.

Table 13: Role capabilities verified from current routing

Capability	Patient	Doctor	Receptionist	Administrator
Self service booking	Yes	No	Assisted booking	Status management
Patient information	Own profile	Related patients	Operational patient list	Hospital wide list
Prescriptions	No dedicated current route	Create and review own	No clinical authoring	No dedicated authoring route
Billing	Own bills and receipts	No billing management	Lists payment and receipts	Lists payment and receipts
Staff creation	No	No	No	Doctors and receptionists
Security toggle and reports	No	No	No	Yes

2 Requirement Determination and Analysis

Requirement analysis connects the purpose of the project to operations that can be inspected and tested. The functional baseline is the current application: its URL configuration, views, templates, models, migrations and existing tests. Requirements below distinguish implemented behaviour from recommended acceptance targets. They do not claim that stakeholder interviews, surveys or hospital observations were performed.

The requirements are organised around the four roles and the records they exchange. Each requirement has an identifier so that a reviewer can connect it to a design element and a test. A requirement described as implemented means that a corresponding source path exists; it does not mean that every branch has been executed successfully in a live deployment.

2.1 Requirement Determination

Method and evidence sources

The analysis began with the repository structure and the root routing configuration. The patient URL file identifies public pages, authentication and self service functions. The doctor, receptionist and adminpanel route files identify staff operations. The functions reached by those routes provide the authoritative behaviour when older project prose disagrees with implementation comments or templates.

Models and migrations establish persistent data and changes over time. Template fields identify the information users are asked to supply, while views determine what is actually checked and saved. Existing automated tests provide executable examples of selected workflows. Deployment files explain intended hosting arrangements, but a configuration template is not evidence that a service has been installed or operated successfully.

The retained project report provides academic identity details and reference screenshots. It is not treated as stronger evidence than the current source for feature claims. In particular, the card form is not a gateway integration and the patient portal does not currently expose a dedicated prescription history route.

Table 14: Requirement evidence sources

Evidence	Question answered	Interpretation boundary
URL configurations	Which operations can be reached?	An unreferenced function may be legacy code.
Views and helpers	What checks and writes occur?	Comments can lag behind executable statements.
Models and migrations	What structure is persisted?	No production data was needed for the dictionary.
Templates and static scripts	What fields and actions are presented?	Browser appearance still needs visual or supplied screenshot evidence.
Automated tests	Which behaviours are asserted?	Passing SQLite tests do not establish MySQL locking behaviour.

Evidence	Question answered	Interpretation boundary
Deployment files	What hosting stack is proposed?	Templates do not prove a running production environment.

Workflow and data analysis

The patient booking workflow has three distinct stages: selecting a slot, reviewing the fee and submitting checkout. The first stage validates a candidate and stores pending details in the session. The review page displays those details. Only checkout submission creates the appointment. This distinction is a requirement because treating the earlier session entry as a persistent reservation would overstate what the system guarantees.

The receptionist workflow differs deliberately. Reception is assisting a patient in person, so a valid booking directly creates an appointment with confirmed status and a pending consultation bill. Confirmation, cancellation and no show are subsequent actions. Their side effects include patient notifications, selected emails and, for a no show, an additional fee equal to the consultation fee.

The doctor workflow begins with assigned visits and related patients. Clinical records contain a diagnosis and optional advice, follow up date and appointment link. Medicine rows are repeating detail records. A linked appointment is rechecked against both the doctor and patient to reject a forged association. The data requirement is therefore stronger than merely accepting an appointment identifier from a dropdown.

Administrator workflows operate across the hospital data set. Lists and report filters use dates, statuses, doctors, departments and search terms where implemented. The same report selection can produce an HTML view, CSV file or PDF file. Billing reports use appointment dates for their date range, a detail that must be understood before comparing them with reports based on bill creation time.

Table 15: Principal workflow inputs and outputs

Workflow	Input	Persistent output	Important transition
Patient registration	Name email password and profile values	User and Patient	OTP may precede authenticated access.
Patient booking	Doctor date slot visit details	Appointment and paid demo bill at checkout	Pending session data is not a reservation.
Reception booking	Patient doctor date slot	Confirmed appointment and pending bill	Availability is checked inside a doctor lock.
Prescription	Patient diagnosis optional appointment medicine arrays	Prescription and medicine rows	Patient relationship and appointment ownership are checked.
No show	Appointment identifier	Status and extra fee	Terminal repeated actions must not duplicate fees.
Report	Type range filters export format	HTTP output; no new report model	Queries read the operational tables.

Technical feasibility

The technical stack is feasible for an academic web application because the repository already contains working Django apps, migrations, templates and a configured project environment. The existing tests create an isolated schema and exercise portal pages and workflow updates. The source uses standard relational models and framework authentication rather than requiring a separately deployed identity service.

The local database guide describes a standalone MySQL installation, while DATABASE_URL permits another configured backend and an SQLite fallback. This makes development flexible, but it creates a verification obligation. The serialisation provided by select_for_update must be evaluated with the actual database engine; SQLite does not provide equivalent row locking semantics.

The frontend depends on browser support for JavaScript and some CDN assets. PDF export depends on ReportLab and valid layout data. Uploaded profile images depend on writable file storage and Pillow validation. Technical readiness therefore includes checking dependencies, static collection, mail settings, file permissions and database connectivity in the intended environment.

Operational feasibility

Operationally, the portal structure maps to recognisable responsibilities: patients manage their own visits, reception coordinates arrival and payment, doctors maintain consultation information, and administrators manage wider records. A training demonstration can follow one patient from registration to booking, reception confirmation, doctor prescription and final receipt.

Operational readiness still requires decisions beyond the source. The hospital would need rules for who may create or deactivate accounts, how a mistaken bill is corrected, how appointment cancellations affect payments and how records are retained. The application does not implement an organisational approval workflow for those decisions. A real deployment would also need an operator to monitor mail failures, back up media and databases and restore a tested copy when required.

Economic feasibility

The application uses an open source programming language and framework, so the academic implementation does not inherently require a commercial web framework licence. That observation is not a complete cost estimate. Hosting, domain registration, operational support, secure mail delivery, backups, staff training and future maintenance all consume resources.

No purchase quotations, staff time measurements or approved budget were supplied. Accordingly, economic feasibility is described through cost categories rather than invented currency totals or claimed savings. A later pilot should measure registration time, failed booking attempts, reporting effort and support incidents before estimating financial benefit.

Table 16: Economic evaluation without invented estimates

Cost or benefit category	Assessment approach
Development	Record time spent on features, defects and documentation.
Hosting and database	Choose a sized environment after workload tests.
Email and backups	Estimate message volume, storage growth and retention.
Training and support	Observe role based task completion in a pilot.
Potential time saving	Compare measured workflows with the prior process.
Risk cost	Include recovery work, downtime and access administration.

Constraints and assumptions

The application assumes that a role account has the matching role profile where required. Some helper views create or retrieve profiles defensively; other operations retrieve an existing profile directly. Account creation and migrations therefore matter to workflow correctness. The report does not assume that every arbitrary user row can immediately use every role portal.

The system also assumes that consultation fees and schedule parameters are meaningful. Positive model fields and form checks help, but a complete boundary review should include zero duration, unusual buffer values, large fees and invalid choice strings. Server side checks must remain authoritative because clients can bypass HTML constraints and JavaScript.

The current code is a synchronous application. Mail attempts and report generation occur within request or command flows rather than a verified background worker architecture. Proposed queues, caching and monitoring appear in Chapter 6 as enhancements. They are not presented as existing runtime services.

2.2 Requirement Specification

Table 17: Functional requirement index

ID	Functional requirement	Source evidence
FR-01	Shared login	patient/views.py:login
FR-02	Optional OTP verification	patient/views.py:verify_otp; accounts.SystemSettings
FR-03	OTP lockout	config/settings.py; patient/views.py:verify_otp
FR-04	Patient self registration	patient/views.py:register
FR-05	Password recovery	patient/views.py:forgot_password; reset_password_confirm
FR-06	Role protected navigation	Role decorators in each portal views.py
FR-07	Patient profile	patient/views.py:patient_profile
FR-08	Doctor discovery	patient/views.py:doctors
FR-09	Slot enquiry	Doctor.get_available_slots; get_doctor_slots views

ID	Functional requirement	Source evidence
FR-10	Schedule rules	doctor/models.py:Doctor.get_available_slots
FR-11	Patient booking draft	patient/views.py:book_appointment
FR-12	Fee review	patient/views.py:confirm_appointment_billing
FR-13	Demonstration checkout	patient/views.py:card_payment
FR-14	Personal appointment history	patient/views.py:my_appointments
FR-15	Patient cancellation	patient/views.py:cancel_appointment
FR-16	Assisted registration	receptionist/views.py:register_patient
FR-17	Patient search	receptionist/views.py:search_patients
FR-18	Assisted booking	receptionist/views.py:book_appointment
FR-19	Appointment confirmation	receptionist/views.py:confirm_appointment
FR-20	No show fee	receptionist/views.py:mark_no_show
FR-21	Doctor appointment actions	doctor/views.py:mark_appointment_complete; doctor_cancel_appointment
FR-22	Related patient access	doctor/patient_views.py; doctor/views.py:patient_details
FR-23	Prescription creation	doctor/views.py:add_prescription
FR-24	Medicine line entry	doctor/views.py:add_prescription; PrescriptionMedicine
FR-25	Prescription history	doctor/views.py:prescription_history
FR-26	Availability maintenance	doctor/views.py:availability
FR-27	Billing and receipts	Billing views in patient, receptionist and adminpanel
FR-28	Record payment	receptionist/views.py:mark_bill_paid; adminpanel/views.py:mark_bill_paid
FR-29	Staff management	adminpanel/views.py:add_doctor; add_receptionist
FR-30	Account status and OTP policy	adminpanel/views.py:toggle_user_status; security_settings
FR-31	Notifications	Notification views and context processors
FR-32	Report generation	adminpanel/report_views.py:generate_report

ID	Functional requirement	Source evidence
FR-33	Report exports	adminpanel/report_views.py:generate_report; _build_pdf
FR-34	Contact submission	patient/views.py:contact
FR-35	Reminder command	patient/management/commands/send_appointment_reminders.py

FR-01 Shared login

All roles shall authenticate through the shared email and password login handler. A selected portal role that differs from the account role is rejected.

Acceptance criterion: Valid credentials reach OTP or the role dashboard; wrong credentials do not create an authenticated session.

Implementation reference: patient/views.py:login

FR-02 Optional OTP verification

When the singleton OTP policy is enabled, login shall require a valid emailed code before auth_login completes.

Acceptance criterion: Correct code signs in once; missing pending user returns to login; expired or incorrect code is rejected.

Implementation reference: patient/views.py:verify_otp; accounts.SystemSettings

FR-03 OTP lockout

The implementation shall count wrong codes and apply the configured temporary lockout after the maximum attempts.

Acceptance criterion: With defaults, three incorrect codes invalidate the code and set a ten minute lockout.

Implementation reference: config/settings.py; patient/views.py:verify_otp

FR-04 Patient self registration

Public registration shall create a patient role account and a Patient profile after the checks in the handler.

Acceptance criterion: A new accepted email has one linked profile; duplicate email or mismatched passwords interrupt registration.

Implementation reference: patient/views.py:register

FR-05 Password recovery

A user shall be able to request a reset link and set a password through the token based reset flow.

Acceptance criterion: A valid token reaches reset; invalid or expired tokens cannot reset the password.

Implementation reference: patient/views.py:forgot_password; reset_password_confirm

FR-06 Role protected navigation

Protected views shall check the authenticated role before exposing portal operations.

Acceptance criterion: A wrong role cannot perform the protected operation, even with a direct URL.

Implementation reference: Role decorators in each portal views.py

FR-07 Patient profile

Patients shall view and update supported personal, contact and medical background fields.

Acceptance criterion: Saved values belong to the current patient; invalid image content is rejected.

Implementation reference: patient/views.py:patient_profile

FR-08 Doctor discovery

The public doctor list shall present doctor data and supported department or search selection.

Acceptance criterion: Displayed records follow the filters implemented by the view.

Implementation reference: patient/views.py:doctors

FR-09 Slot enquiry

Booking interfaces shall retrieve date specific available and occupied slots for a selected doctor.

Acceptance criterion: Malformed date yields a 400 JSON response; blocked dates offer no bookable slots.

Implementation reference: Doctor.get_available_slots; get_doctor_slots views

FR-10 Schedule rules

Slot calculation shall honour past dates, blocked dates, weekday closure, start and end time, duration, buffer and optional daily cap.

Acceptance criterion: Generated labels follow the saved rules; missing weekday row uses the current 09:00 to 17:00 fallback.

Implementation reference: doctor/models.py:Doctor.get_available_slots

FR-11 Patient booking draft

Validated patient booking details shall be kept in pending session data before final checkout.

Acceptance criterion: No Appointment is created by the initial selection POST alone.

Implementation reference: patient/views.py:book_appointment

FR-12 Fee review

The patient shall see the selected doctor, date, time and consultation fee before checkout.

Acceptance criterion: The review page uses the pending session and redirects to selection if it is missing.

Implementation reference: patient/views.py:confirm_appointment_billing

FR-13 Demonstration checkout

Checkout submission shall recheck the selected slot, create a pending appointment and a bill marked paid for the simulation.

Acceptance criterion: A stale slot is rejected; an accepted submission clears pending data and presents the new appointment.

Implementation reference: patient/views.py:card_payment

FR-14 Personal appointment history

A patient shall retrieve only appointments associated with the current patient profile.

Acceptance criterion: Other patients appointments do not appear in the personal list.

Implementation reference: patient/views.py:my_appointments

FR-15 Patient cancellation

The patient cancellation action shall operate on an appointment owned by the current patient and follow its implemented status rules.

Acceptance criterion: A foreign identifier does not grant cancellation rights.

Implementation reference: patient/views.py:cancel_appointment

FR-16 Assisted registration

Receptionists shall create walk in patient accounts and retain the registering receptionist relationship.

Acceptance criterion: An accepted registration creates User and Patient with registered_by set.

Implementation reference: receptionist/views.py:register_patient

FR-17 Patient search

Receptionists shall search patients for assisted booking and retrieve JSON result records.

Acceptance criterion: Matching results use the implemented response structure; an unknown query has no false matches.

Implementation reference: receptionist/views.py:search_patients

FR-18 Assisted booking

Receptionists shall create confirmed appointments and pending consultation bills using a currently free slot.

Acceptance criterion: A sequential duplicate request leaves only one noncancelled booking for the slot.

Implementation reference: receptionist/views.py:book_appointment

FR-19 Appointment confirmation

Receptionists shall confirm an appointment through the protected status action.

Acceptance criterion: The supported POST sets confirmed status and creates a patient notification.

Implementation reference: receptionist/views.py:confirm_appointment

FR-20 No show fee

The no show action shall set attendance status and create a separate no_show_fee bill when allowed.

Acceptance criterion: The fee equals the doctor consultation fee; repeated terminal action does not duplicate the fee.

Implementation reference: receptionist/views.py:mark_no_show

FR-21 Doctor appointment actions

Doctors shall view and complete or cancel appointments belonging to their own profile.

Acceptance criterion: A successful own appointment action changes status and returns to a safe destination.

Implementation reference: doctor/views.py:mark_appointment_complete; doctor_cancel_appointment

FR-22 Related patient access

Doctors shall view patients with an appointment relationship to the current doctor.

Acceptance criterion: An unrelated patient detail identifier is rejected rather than exposed.

Implementation reference: doctor/patient_views.py; doctor/views.py:patient_details

FR-23 Prescription creation

Doctors shall save a required patient and diagnosis with optional advice, follow up date and an appointment belonging to the same doctor and patient.

Acceptance criterion: A forged appointment relationship is rejected; accepted clinical information belongs to the current doctor.

Implementation reference: doctor/views.py:add_prescription

FR-24 Medicine line entry

Prescription entry shall support multiple medicine rows with name, dosage, frequency, duration and instructions.

Acceptance criterion: Blank medicine names are skipped; missing optional parallel values default to empty strings.

Implementation reference: doctor/views.py:add_prescription; PrescriptionMedicine

FR-25 Prescription history

Doctors shall retrieve their own prescription history and supported filters with prefetched medicines.

Acceptance criterion: Displayed prescriptions are limited to the current doctor.

Implementation reference: doctor/views.py:prescription_history

FR-26 Availability maintenance

Doctors shall maintain weekly availability and date specific leave exclusions through the availability view.

Acceptance criterion: Saved scheduling changes affect later slot enquiries.

Implementation reference: doctor/views.py:availability

FR-27 Billing and receipts

Patients shall view own bills; receptionists and administrators shall inspect operational billing and receipts.

Acceptance criterion: Receipt fields correspond to the selected permitted bill and its linked appointment.

Implementation reference: Billing views in patient, receptionist and adminpanel

FR-28 Record payment

Authorised front desk and administrator actions shall mark a bill paid and set its payment timestamp.

Acceptance criterion: The bill has paid status and nonnull paid_at after the supported action.

Implementation reference: receptionist/views.py:mark_bill_paid; adminpanel/views.py:mark_bill_paid

FR-29 Staff management

Administrators shall create doctor and receptionist accounts with their matching profile details.

Acceptance criterion: The new account role and profile agree; invalid or duplicate details interrupt creation.

Implementation reference: adminpanel/views.py:add_doctor; add_receptionist

FR-30 Account status and OTP policy

Administrators shall manage permitted account activation changes and the singleton OTP toggle.

Acceptance criterion: Saved policy is read by subsequent authentication; protected self status rules remain enforced.

Implementation reference: adminpanel/views.py:toggle_user_status; security_settings

FR-31 Notifications

Users shall retrieve and manage notifications addressed to their own User record.

Acceptance criterion: Read and dismiss actions operate within the authenticated users notification set.

Implementation reference: Notification views and context processors

FR-32 Report generation

Administrators shall select patients, appointments, doctors or billing reports with applicable filters.

Acceptance criterion: The chosen view displays matching rows and rejects unsupported report type.

Implementation reference: adminpanel/report_views.py:generate_report

FR-33 Report exports

Administrators shall download the selected report as CSV or PDF using the same filtered row preparation.

Acceptance criterion: The response content type and attachment name match the requested format.

Implementation reference: adminpanel/report_views.py:generate_report; _build_pdf

FR-34 Contact submission

Public visitors shall submit supported contact fields for storage as a ContactMessage.

Acceptance criterion: Required information is checked before storing a submission.

Implementation reference: patient/views.py:contact

FR-35 Reminder command

The supplied management command shall select eligible upcoming appointments and invoke reminder delivery logic.

Acceptance criterion: A controlled test should confirm eligibility, reminder flags and failure behaviour without contacting real patients.

Implementation reference: patient/management/commands/send_appointment_reminders.py

Table 18: Nonfunctional requirements and present controls

ID	Quality attribute	Current position
NFR-01	Security	The application should deny unauthorised role and object access. Existing controls include session authentication, role decorators, selected ownership filters, CSRF middleware and safe redirects.
NFR-02	Data integrity	Related records should remain consistent after success and after failure. The ORM provides relationships and supported booking views use doctor row locking.

ID	Quality attribute	Current position
NFR-03	Performance	Operational lists should remain responsive as records grow. Shared pagination defaults to twenty items and selected queries use related object loading.
NFR-04	Reliability	Rejected input should preserve valid data and provide an understandable result. Existing messages and redirect paths support many validation failures.
NFR-05	Usability	Each role should reach its common tasks through a consistent dashboard and navigation. Templates provide separate portal bases and responsive layouts.
NFR-06	Maintainability	Common rules should be localised and new changes should be traceable. Apps separate roles and helper modules centralise repeated concerns.
NFR-07	Compatibility	The application should work in its documented Python and database environment and in modern browsers. Requirements declare exact package pins.
NFR-08	Availability	Users should be able to access the service during the agreed operating period. Deployment templates describe a service process and reverse proxy.
NFR-09	Scalability	The design should tolerate a growing record set without unbounded request work. Existing pagination helps selected views.
NFR-10	Auditability	Important clinical, account and billing changes should be attributable and reconstructable. Several models have created_at or updated_at timestamps.
NFR-11	Recoverability	Database and uploaded media should be restorable together. Current files include local backups and operational scripts.

ID	Quality attribute	Current position
NFR-12	Time consistency	Displayed and calculated appointment times should follow a documented hospital timezone. The current setting is UTC but slot logic also uses naive local time.

NFR-01 Security

The application should deny unauthorised role and object access. Existing controls include session authentication, role decorators, selected ownership filters, CSRF middleware and safe redirects.

Verification or improvement criterion: Run a permission matrix across all protected endpoints, including guessed identifiers. This is a verification target; the current suite does not cover every combination.

NFR-02 Data integrity

Related records should remain consistent after success and after failure. The ORM provides relationships and supported booking views use doctor row locking.

Verification or improvement criterion: Test rollback after each write stage; extend atomic units to appointment, bill and notification creation where appropriate. Do not infer full atomicity from the existing booking lock.

NFR-03 Performance

Operational lists should remain responsive as records grow. Shared pagination defaults to twenty items and selected queries use related object loading.

Verification or improvement criterion: Measure latency and query counts using realistic data volume. No numeric response time guarantee is claimed in this report.

NFR-04 Reliability

Rejected input should preserve valid data and provide an understandable result. Existing messages and redirect paths support many validation failures.

Verification or improvement criterion: Exercise malformed values, deleted references and failing email backends; record both response and database state.

NFR-05 Usability

Each role should reach its common tasks through a consistent dashboard and navigation. Templates provide separate portal bases and responsive layouts.

Verification or improvement criterion: Complete representative tasks by keyboard and at mobile widths. The supplied screenshots support layout discussion, not a full accessibility certification.

NFR-06 Maintainability

Common rules should be localised and new changes should be traceable. Apps separate roles and helper modules centralise repeated concerns.

Verification or improvement criterion: Require route to requirement to test traceability; prefer shared forms or services for repeated validation in future work.

NFR-07 Compatibility

The application should work in its documented Python and database environment and in modern browsers. Requirements declare exact package pins.

Verification or improvement criterion: Reproduce a clean installation and verify database specific behaviour. Passing tests in the existing environment do not prove every pinned dependency combination.

NFR-08 Availability

Users should be able to access the service during the agreed operating period. Deployment templates describe a service process and reverse proxy.

Verification or improvement criterion: Define monitoring, restart policy, backup intervals and recovery objectives before production use; no uptime measurement is available.

NFR-09 Scalability

The design should tolerate a growing record set without unbounded request work. Existing pagination helps selected views.

Verification or improvement criterion: Load test booking contention and large exports; introduce bounded or background exports if memory use becomes excessive.

NFR-10 Auditability

Important clinical, account and billing changes should be attributable and reconstructable. Several models have created_at or updated_at timestamps.

Verification or improvement criterion: Add an application audit trail with actor, operation and prior values. Timestamps alone do not constitute a complete audit record.

NFR-11 Recoverability

Database and uploaded media should be restorable together. Current files include local backups and operational scripts.

Verification or improvement criterion: Define a repeatable restore drill in a separate environment and verify relational and media references; backup presence alone is insufficient.

NFR-12 Time consistency

Displayed and calculated appointment times should follow a documented hospital timezone. The current setting is UTC but slot logic also uses naive local time.

Verification or improvement criterion: Choose a timezone policy and test boundary dates, midnight and daylight saving behaviour where relevant. The present mismatch remains an improvement item.

Requirement dependencies and priority

Identity and role checks precede every protected workflow. Patient and doctor profiles precede appointment creation, and an appointment precedes a consultation bill. Prescription entry depends on the doctor to patient relationship and optionally on the exact appointment. Reports depend on the operational record definitions, so their correctness cannot be assessed only by whether a file downloads.

The first acceptance priority is preventing unauthorised access and incorrect core writes. The second is validating complete booking, consultation and billing workflows. The third is verifying output parity, responsive presentation and operational recovery. New functionality should not displace these checks merely because a screen appears complete.

Table 19: Acceptance prioritisation

Priority	Requirements	Reason
Essential	FR-01 to FR-07, FR-09 to FR-18, FR-21 to FR-24, FR-27 to FR-30	Identity, scheduling, clinical ownership and financial consistency.
Operational	FR-08, FR-19, FR-20, FR-25, FR-26, FR-31 to FR-33	Daily coordination, availability and management output.
Supporting	FR-34 and FR-35; broader NFR acceptance work	Communication and operational hardening beyond the core transaction paths.

3 System Design

System design translates requirements into actors, records, request flows and user interfaces. The project follows Django's model, template and view organisation. Its logical boundaries are shared identity, role specific workflows, persistent domain models and output helpers. These boundaries are implemented within one application deployment rather than as independent microservices.

The diagrams below are derived from current relationships and view behaviour. They describe the design at a readable level and do not imply components absent from the repository. The sequence and activity figures are numbered workflow abstractions; the code references identify the exact implementation for detailed review.

3.1 Use Case Diagram

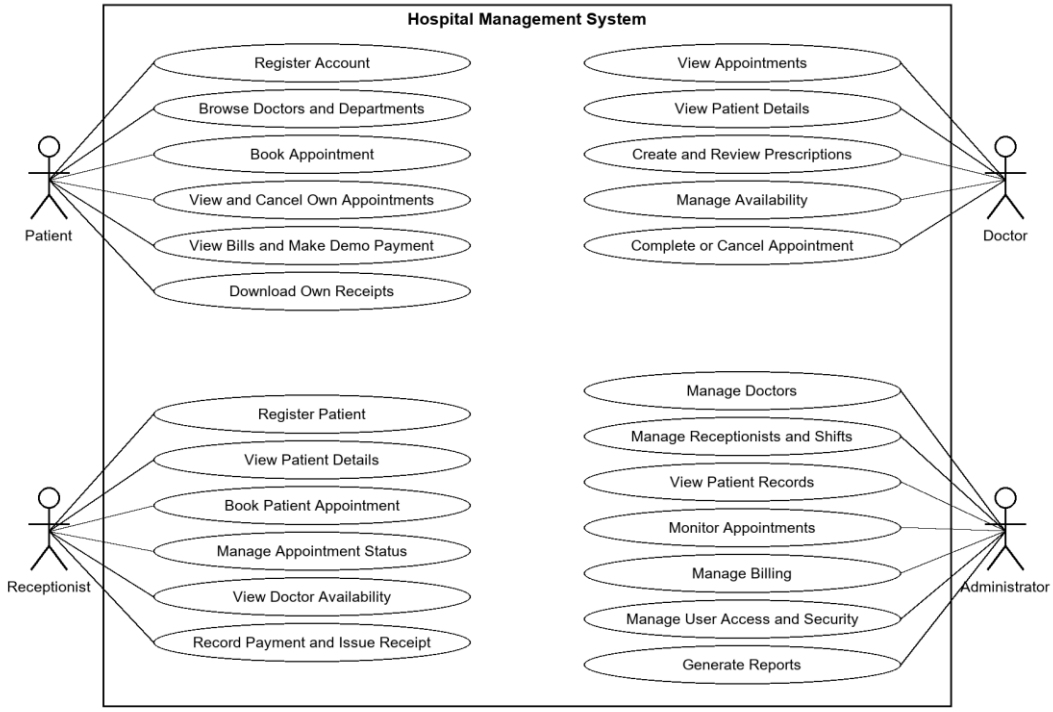


Figure 4: Actors and major use cases inside the application boundary

The four portal roles are the main actors. A public visitor is a supporting actor for information, contact and registration pages. Email delivery is a supporting external dependency in authentication and appointment communication. The administrator actor represents the custom portal role and should not be conflated with every privilege available in Django's framework administration site.

Authentication is a shared prerequisite rather than a different login implementation for each staff group. Appointment booking has different flows for the patient and receptionist actors. Prescriptions belong to the doctor workflow, while report generation and the OTP policy

belong to the administrator workflow. The diagram keeps those distinctions visible without drawing a separate oval for every profile and notification action.

UC-01 Authenticate user

Table 20: UC-01 contract

Property	Use case description
Actor	All registered roles
Preconditions	An active account exists; the role and credentials are known.
Main flow	Open shared login; submit email and password; check selected role; if policy requires OTP send a code and store the pending user; verify the code; start the session; redirect to the matching dashboard.
Alternative flow	Wrong credentials return a generic error; wrong role is rejected; expired code requires another code; excessive wrong codes start lockout.
Postconditions	The user has an authenticated session for the stored role.
Implementation	patient/views.py:login and verify_otp

UC-02 Register patient

Table 21: UC-02 contract

Property	Use case description
Actor	Public visitor
Preconditions	The supplied email is not already registered.
Main flow	Open registration; enter personal and login values; validate required values and passwords; create a patient role user; create the patient profile; continue to the applicable authentication step.
Alternative flow	Duplicate email, invalid name or password failure prevents normal completion. Staff roles are not created through this public flow.
Postconditions	The accepted account has a Patient profile and can proceed through login policy.
Implementation	patient/views.py:register

UC-03 Book patient appointment

Table 22: UC-03 contract

Property	Use case description
Actor	Patient
Preconditions	The patient is logged in and has a Patient profile.

Property	Use case description
Main flow	Choose doctor and date; fetch available slots; select a free slot; submit details; store the pending session; review consultation fee; open checkout; submit; lock doctor and recheck; create appointment; create paid demo bill and notification; attempt email; clear the session draft.
Alternative flow	Invalid date or occupied slot interrupts booking. A slot taken after initial selection is rejected at checkout. Missing session data returns to booking.
Postconditions	An accepted checkout produces a pending appointment and a bill marked paid as a simulation. No gateway charge occurs.
Implementation	patient/views.py:book_appointment, confirm_appointment_billing and card_payment

UC-04 Book at reception

Table 23: UC-04 contract

Property	Use case description
Actor	Receptionist
Preconditions	A patient and doctor exist and the receptionist is authenticated.
Main flow	Find the patient using search or a detail link; choose doctor date and slot; submit required values; lock the doctor; recheck availability; create a confirmed appointment; create a pending consultation bill; notify the patient; attempt the booking email.
Alternative flow	Missing fields and invalid dates are rejected. A slot already occupied on the final check produces an error instead of a second booking.
Postconditions	The patient has a confirmed visit and a pending consultation charge.
Implementation	receptionist/views.py:book_appointment

UC-05 Confirm appointment

Table 24: UC-05 contract

Property	Use case description
Actor	Receptionist
Preconditions	The user has the receptionist role and the appointment exists.
Main flow	Open today or all appointments; submit the confirmation form; update the appointment status; add the patient confirmation notification; resolve a safe next destination.
Alternative flow	GET does not perform the supported write. Wrong role cannot confirm. Invalid or external next destinations fall back to the configured list.

Property	Use case description
Postconditions	Appointment status is confirmed and the result is visible from the permitted list.
Implementation	receptionist/views.py:confirm_appointment; safe_redirect.py

UC-06 Cancel appointment

Table 25: UC-06 contract

Property	Use case description
Actor	Patient, assigned doctor or receptionist through their routes
Preconditions	The actor can retrieve the appointment within that routes permission scope.
Main flow	Select the appointment; submit cancellation; check the view specific conditions; set cancelled; create applicable notification and email; return to the relevant list.
Alternative flow	An unrelated patient or doctor identifier is not accepted through the ownership filtered view. A terminal status may be rejected according to the selected portal handler.
Postconditions	The appointment is cancelled; slot generation no longer marks its time as occupied.
Implementation	Cancellation handlers in patient, doctor and receptionist views

UC-07 Record no show

Table 26: UC-07 contract

Property	Use case description
Actor	Receptionist
Preconditions	The appointment is accessible and not already in a rejected terminal state.
Main flow	Submit the no show action; inspect state; change status to no_show; create a no_show_fee bill using the doctor consultation fee; create the appropriate notification; return to the list.
Alternative flow	Repeated terminal or already no show actions are rejected so they do not add another no show fee.
Postconditions	Attendance is recorded and the additional fee exists once for the supported action sequence.
Implementation	receptionist/views.py:mark_no_show

UC-08 Complete consultation

Table 27: UC-08 contract

Property	Use case description
Actor	Doctor

Property	Use case description
Preconditions	The appointment belongs to the current doctor.
Main flow	Review own appointment; submit completion; retrieve it using doctor ownership; set completed; return through safe next routing.
Alternative flow	Wrong role or unrelated appointment is rejected; a forged next destination cannot send the browser to an external host.
Postconditions	The appointment is completed within the doctors permitted records.
Implementation	doctor/views.py:mark_appointment_complete

UC-09 Write prescription

Table 28: UC-09 contract

Property	Use case description
Actor	Doctor
Preconditions	The patient has an appointment relationship with the current doctor.
Main flow	Open prescription form; select patient; optionally select a linked appointment; enter diagnosis advice and follow up; add medicine lines; validate patient and diagnosis; recheck appointment pair; save prescription and nonblank medicine rows; notify patient; show history.
Alternative flow	Missing diagnosis prevents normal save. A linked appointment for another patient or doctor is rejected. Empty medicine names are skipped.
Postconditions	The doctors history contains the prescription and its saved medicine lines.
Implementation	doctor/views.py:add_prescription

UC-10 Maintain availability

Table 29: UC-10 contract

Property	Use case description
Actor	Doctor
Preconditions	The doctor profile exists and the doctor is authenticated.
Main flow	Open availability; edit supported scheduling parameters and weekday entries or blocked date inputs; submit the selected action; validate the relevant values; save the changed schedule; use slot enquiry to inspect the result.
Alternative flow	Invalid time ranges or malformed date input interrupt the relevant path. A blocked date overrides the weekday schedule.
Postconditions	Later booking enquiries use the revised scheduling data.

Property	Use case description
Implementation	doctor/views.py:availability; doctor/models.py:Doctor.get_available_slots

UC-11 Record bill payment

Table 30: UC-11 contract

Property	Use case description
Actor	Receptionist or administrator
Preconditions	The actor is authorised for the billing management route and the bill exists.
Main flow	Open billing; inspect the selected record; submit mark paid; change status to paid; set paid_at to the current framework time; return to a safe list destination.
Alternative flow	Wrong role or unsupported request method does not perform the intended write. This action records a status and does not execute a card gateway settlement.
Postconditions	The bill presents paid status and a payment timestamp.
Implementation	mark_bill_paid in receptionist/views.py and adminpanel/views.py

UC-12 Generate management report

Table 31: UC-12 contract

Property	Use case description
Actor	Administrator
Preconditions	The administrator is authenticated and has selected an implemented report type.
Main flow	Open report centre; choose report type; set relevant range and filters; prepare the ORM queryset and columns; display results; optionally request csv or pdf; produce an attachment from the filtered rows.
Alternative flow	A reversed date range displays an error. Invalid report type returns to report selection. An empty result is represented explicitly in the PDF table.
Postconditions	The administrator receives a view or downloadable output reflecting the selected query.
Implementation	adminpanel/report_views.py:generate_report

UC-13 Maintain staff account

Table 32: UC-13 contract

Property	Use case description
Actor	Administrator
Preconditions	The administrator has the required creation or status operation available.

Property	Use case description
Main flow	Choose doctor or receptionist creation; enter credentials and role profile details; validate supplied values; create related records; later inspect staff list or use the permitted account activation action.
Alternative flow	Duplicate account details interrupt creation. Account status handler includes protected self account behaviour that must be preserved.
Postconditions	The saved identity and role profile support the intended staff portal.
Implementation	adminpanel/views.py:add_doctor, add_receptionist and toggle_user_status

UC-14 Manage notifications

Table 33: UC-14 contract

Property	Use case description
Actor	Any authenticated portal user
Preconditions	Notification records are associated with the current User.
Main flow	Open the portal notification list; inspect messages; submit mark all read or dismiss one item; scope the update or deletion to the user; return to the notification page.
Alternative flow	A notification belonging to another account is outside the permitted selection. Empty lists remain valid output states.
Postconditions	The current users read state or retained notification set is updated.
Implementation	Notification handlers in the four portal view modules

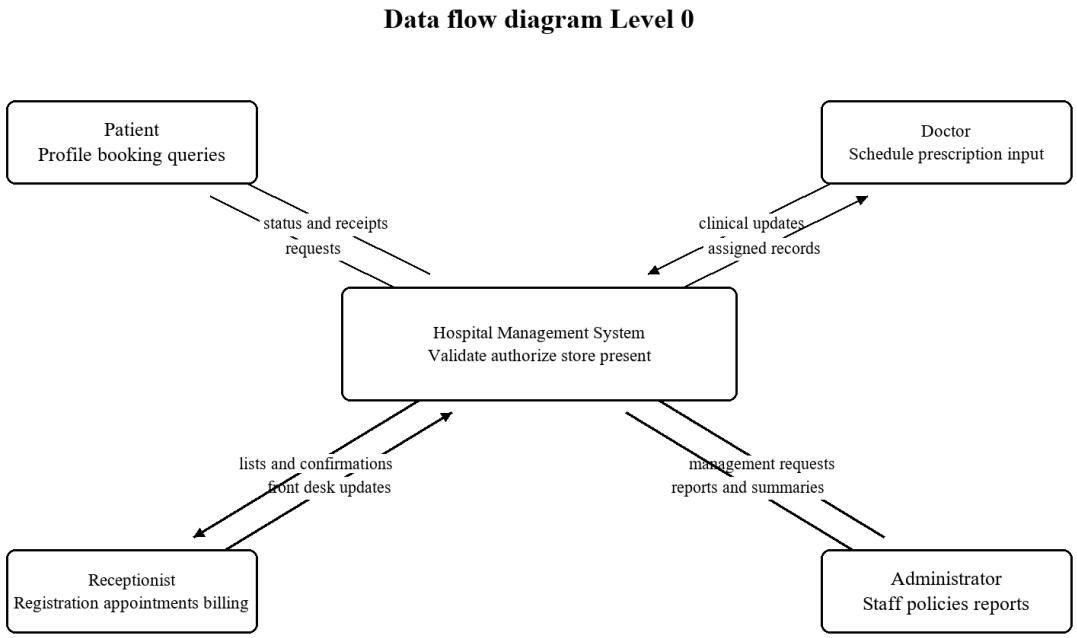


Figure 5: Context data flow with four portal actors

At context level the application receives credentials, selections and record updates and returns role permitted information. Data access is mediated by views; actors do not query the database directly.

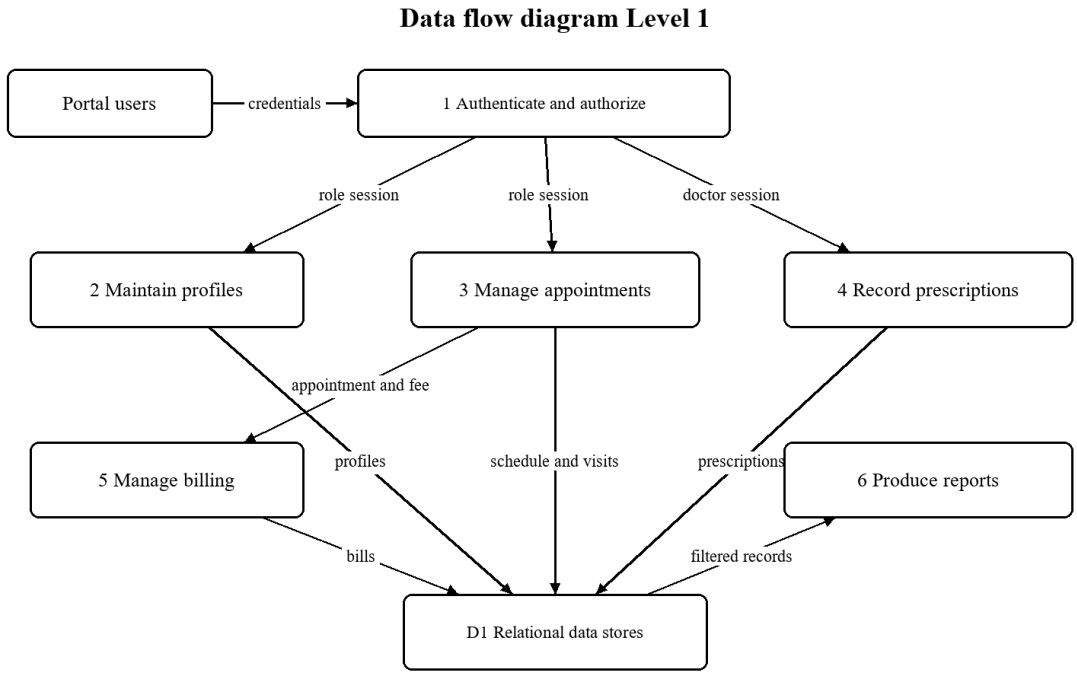


Figure 6: Logical decomposition of record processing and outputs

The Level 1 diagram separates identity, profile, scheduling, prescription, billing and reporting processes. D1 represents the group of relational tables documented in the next section, not a single physical business table.

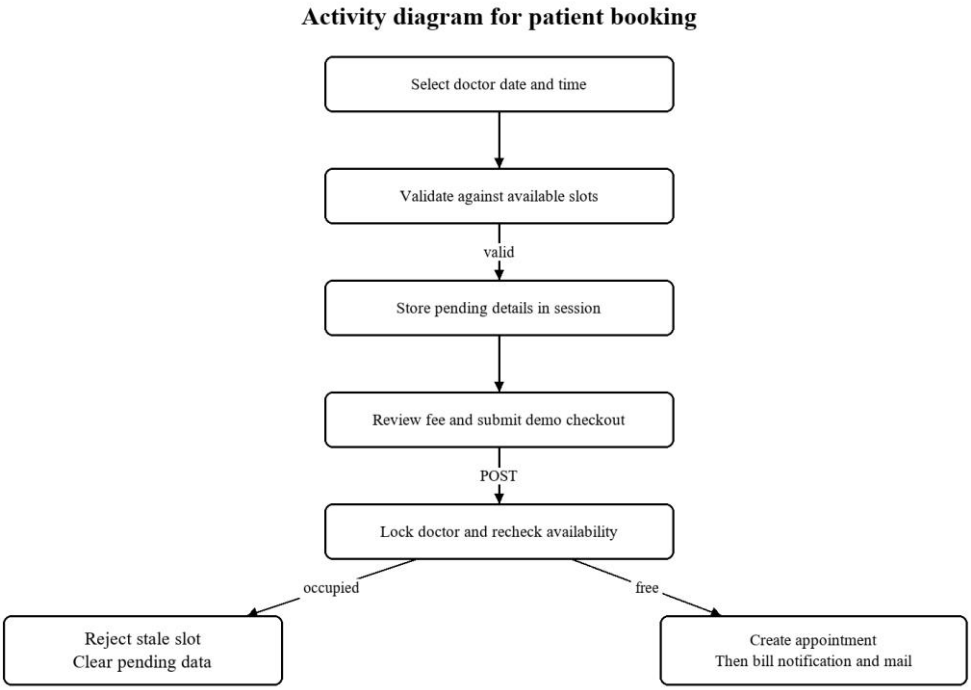


Figure 7: Patient booking activity and final availability decision

The pending session stage precedes persistence. The final availability decision is inside the supported doctor lock; creation of the bill and messages follows the appointment transaction in the current source.

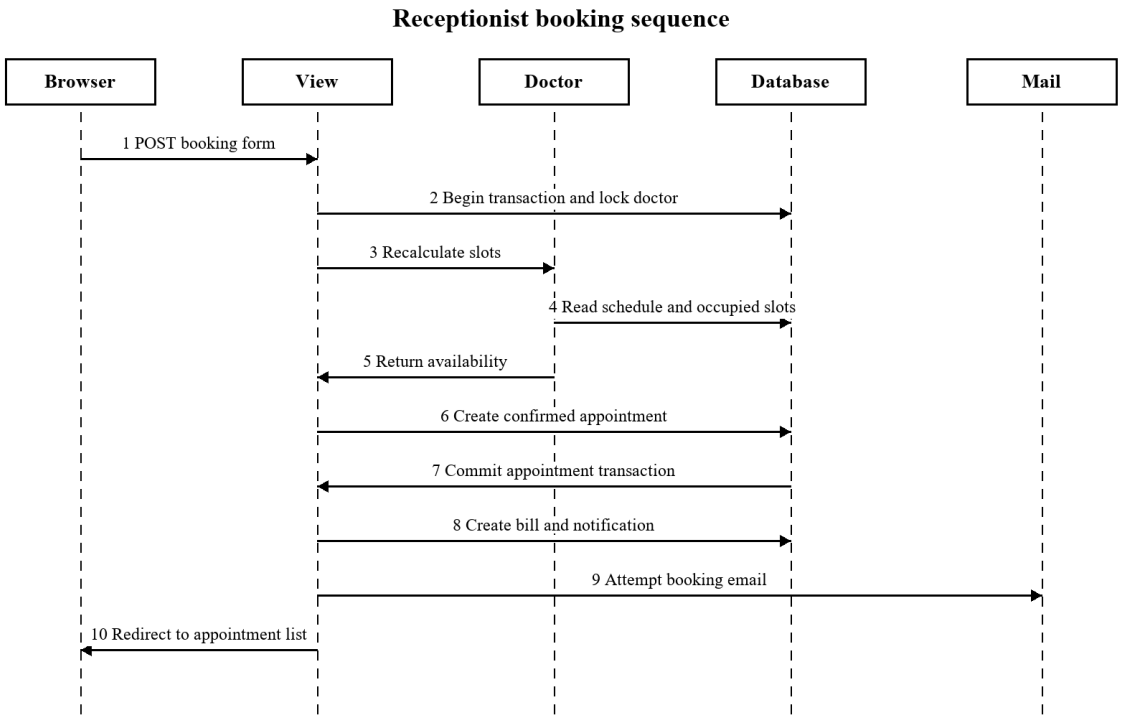


Figure 8: Numbered interaction sequence for assisted booking

This sequence emphasises operation order. A production service layer could group all business writes into one unit and defer email until commit, but that is a proposed improvement rather than the current transaction boundary.

3.2 Data Dictionary

The dictionary is generated from the current Django model metadata without reading patient or account records. Field types are Django logical types. The exact SQL representation depends on the selected database backend. Character sizes are maximum lengths; DecimalField precision is shown as total digits and decimal places. A dash means that the field does not have a character length specification.

Null allowed refers to database NULL, while blank allowed refers to model validation of empty input. These concepts are different. A CharField may allow an empty string while still having null=False. Model choices enumerate valid application values, but a normal save does not automatically invoke full_clean. View validation and database constraints must be considered separately.

Every listed domain model has an automatically generated primary key unless explicitly noted. Related fields use the referenced model primary key. One to one fields also carry uniqueness. Django framework models and many to many permission junction tables are discussed as supporting structures rather than confused with the thirteen application models.

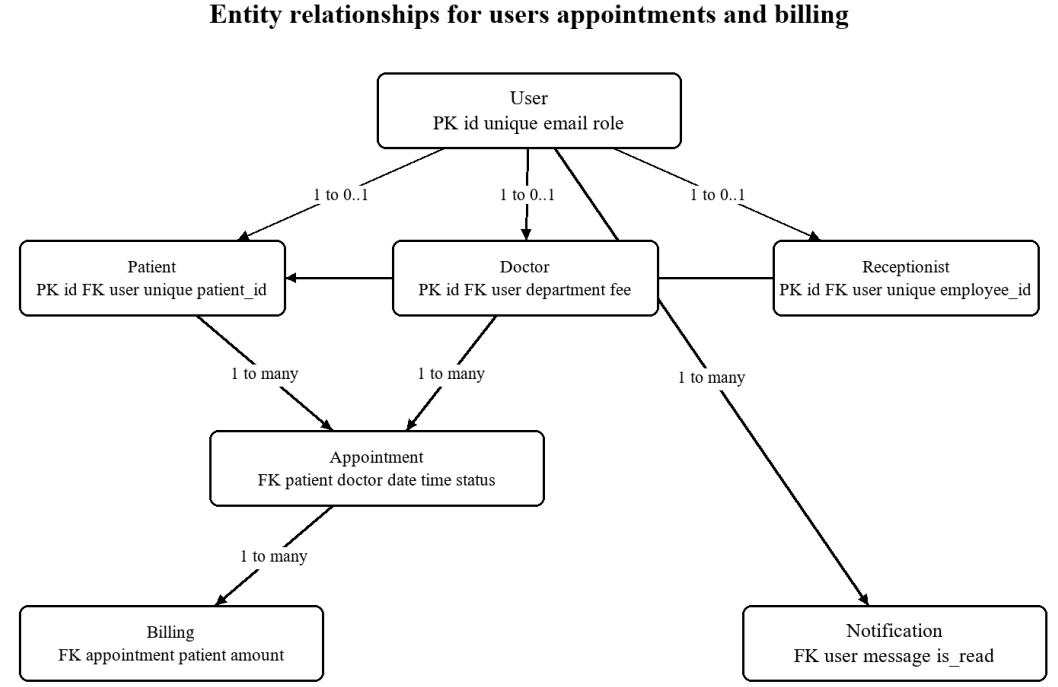


Figure 9: Core identity appointment and billing relationships

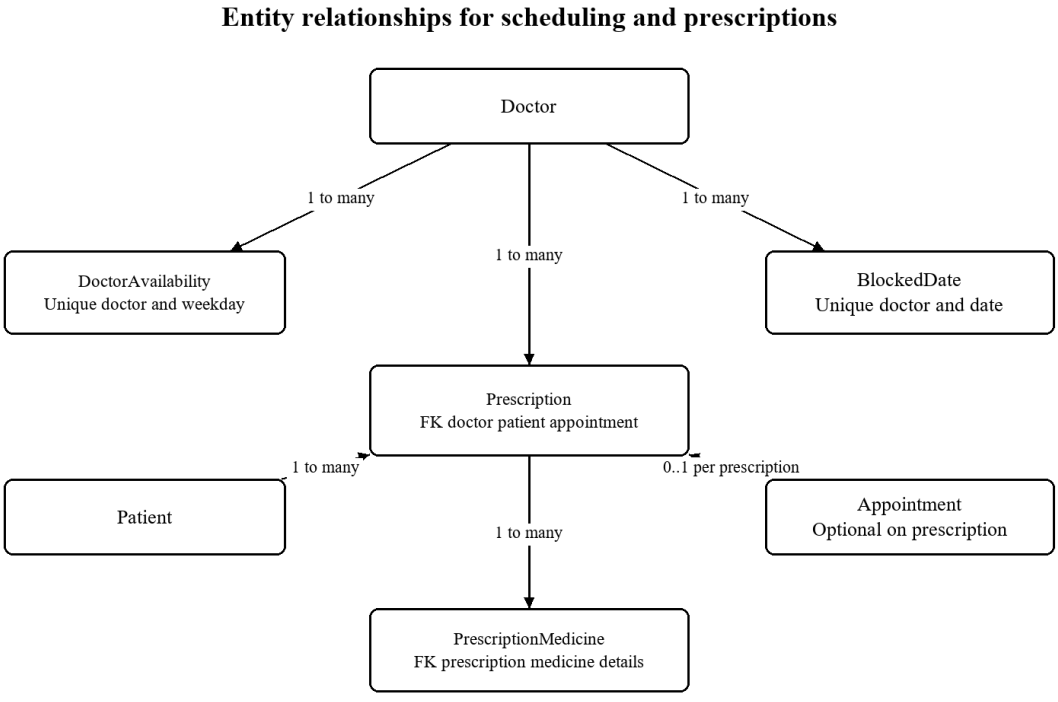


Figure 10: Scheduling rules and prescription detail relationships

accounts.User

Database table: accounts_user. Shared authentication identity for every portal. Email is the login identifier and is unique. Inherited username remains unique and required for standard superuser creation.

Table 34: User fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
password	CharField	128	-	No	Framework encoded password value.
last_login	DateTimeField	-	-	Yes	Last login. Blank allowed.
is_superuser	BooleanField	-	-	No	Framework superuser permission flag.
username	CharField	150	-; unique	No	Username.
first_name	CharField	150	-	No	First name. Blank allowed.
last_name	CharField	150	-	No	Last name. Blank allowed.
is_staff	BooleanField	-	-	No	Permission to enter framework admin subject to permissions.
is_active	BooleanField	-	-	No	Framework account activation flag.
date_joined	DateTimeField	-	-	No	Date joined.
email	CharField	254	-; unique	No	Email contact and, for User, login identifier.
role	CharField	20	-	No	Portal role; distinct from framework permission flags.
otp_code	CharField	6	-	Yes	Current short lived verification code. Blank allowed.
otp_created_at	DateTimeField	-	-	Yes	Generation time used for expiry. Blank allowed.
otp_attempts	PositiveSmallIntegerField	-	-	No	Wrong code attempt counter.
otp_locked_until	DateTimeField	-	-	Yes	Authentication lockout expiry. Blank allowed.

OTP code, generation time, attempt count and lockout time are authentication state. Password contains the framework password representation rather than a plaintext password. Role is a business field separate from is_staff and is_superuser.

Table 35: User value domains

Field	Declared choices
role	admin = Admin; doctor = Doctor; receptionist = Receptionist; patient = Patient

accounts.SystemSettings

Database table: accounts_systemsettings. Hospital wide OTP policy record. get_solo retrieves or creates row id 1.

Table 36: SystemSettings fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
otp_login_enabled	BooleanField	-	-	No	Runtime OTP policy switch.

This singleton convention is implemented by the accessor, not by a special database singleton constraint. All authentication policy readers should use the same accessor.

patient.Patient

Database table: patient_patient. Demographic, contact and background information associated with exactly one User.

Table 37: Patient fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
user_id	OneToOneField	-	FK accounts.User; unique	No	Linked authentication identity.
patient_id	CharField	20	-; unique	No	Patient number when text; patient foreign key when relation. Blank allowed.
profile_image	FileField	100	-	Yes	Stored file path for patient profile image. Blank allowed.
registered_by_id	ForeignKey	-	FK receptionist.Receptionist	Yes	Receptionist who registered the patient. Blank allowed.
phone	CharField	15	-	No	Phone. Blank allowed.
date_of_birth	DateField	-	-	Yes	Date of birth. Blank allowed.
gender	CharField	10	-	No	Gender. Blank allowed.
blood_group	CharField	5	-	No	Blood group. Blank allowed.
address	TextField	-	-	No	Address. Blank allowed.
city	CharField	100	-	No	City. Blank allowed.
state	CharField	100	-	No	State. Blank allowed.
country	CharField	100	-	No	Country. Blank allowed.
pincode	CharField	10	-	No	Pincode. Blank allowed.
emergency_contact_name	CharField	100	-	No	Emergency contact name. Blank allowed.
emergency_contact_number	CharField	15	-	No	Emergency contact number. Blank allowed.
allergies	TextField	-	-	No	Allergies. Blank allowed.
medical_history	TextField	-	-	No	Medical history. Blank allowed.
created_at	DateTimeField	-	-; index	No	Creation timestamp set by the model. Blank allowed.
updated_at	DateTimeField	-	-	No	Last model update timestamp. Blank allowed.

patient_id is a unique human readable identifier generated from the last row when absent. registered_by is optional and becomes NULL when the receptionist profile is deleted; profile deletion cascades from User.

Table 38: Patient value domains

Field	Declared choices
gender	Male = Male; Female = Female; Other = Other
blood_group	A+ = A+; A- = A-; B+ = B+; B- = B-; AB+ = AB+; AB- = AB-; O+ = O+; O- = O-

Table 39: Patient referential actions

Reference field	Target model	On deletion
user_id	accounts.User	CASCADE
registered_by_id	receptionist.Receptionist	SET_NULL

patient.Appointment

Database table: patient_appointment. A dated consultation between one Patient and one Doctor, with visit type, department, reason and workflow status.

Table 40: Appointment fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
patient_id	ForeignKey	-	FK patient.Patient	No	Patient number when text; patient foreign key when relation.
doctor_id	ForeignKey	-	FK doctor.Doctor	No	Doctor profile reference.
appointment_date	DateField	-	-; index	No	Appointment date.
time_slot	CharField	20	-	No	Generated appointment time label.
visit_type	CharField	20	-	No	Visit type.
reason	TextField	-	-	No	Reason for consultation.
status	CharField	20	-; index	No	Workflow state from declared choices.
created_at	DateTimeField	-	-	No	Creation timestamp set by the model. Blank allowed.
updated_at	DateTimeField	-	-	No	Last model update timestamp. Blank allowed.
department	CharField	30	-	No	Selected clinical department code.
reminder_sent	BooleanField	-	-	No	Reminder processing flag.

There is no current unique doctor, date and time_slot constraint. Supported booking views rely on locking the doctor and checking available slots. time_slot is stored as a label, so consistent generation matters.

Table 41: Appointment value domains

Field	Declared choices
visit_type	new = New Visit; follow-up = Follow-up Visit
status	pending = Pending; confirmed = Confirmed; cancelled = Cancelled; completed = Completed; no_show = No Show
department	cardiology = Cardiology; neurology = Neurology; orthopedics = Orthopedics; pediatrics = Pediatrics; dermatology = Dermatology; general = General Medicine

Table 42: Appointment referential actions

Reference field	Target model	On deletion
patient_id	patient.Patient	CASCADE
doctor_id	doctor.Doctor	CASCADE

patient.Billing

Database table: patient_billing. A charge associated with an Appointment and Patient. An appointment can have consultation and no show fee records.

Table 43: Billing fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
appointment_id	ForeignKey	-	FK patient.Appointment	No	Related visit reference.
patient_id	ForeignKey	-	FK patient.Patient	No	Patient number when text; patient foreign key when relation.
bill_type	CharField	20	-	No	Bill type.
amount	DecimalField	8,2	-	No	Charge amount with fixed decimal precision.
status	CharField	10	-; index	No	Workflow state from declared choices.
created_at	DateTimeField	-	-	No	Creation timestamp set by the model. Blank allowed.
updated_at	DateTimeField	-	-	No	Last model update timestamp. Blank allowed.
paid_at	DateTimeField	-	-	Yes	Timestamp recorded when marked paid. Blank allowed.

amount uses eight total digits with two decimal places. The patient foreign key duplicates an association available through the appointment; the application must keep them equal. paid_at can be absent even though a status exists.

Table 44: Billing value domains

Field	Declared choices
bill_type	consultation = Consultation Fee; no_show_fee = No-Show Fee

Field	Declared choices
status	pending = Pending; paid = Paid; waived = Waived

Default ordering: -created_at.

Table 45: Billing referential actions

Reference field	Target model	On deletion
appointment_id	patient.Appointment	CASCADE
patient_id	patient.Patient	CASCADE

patient.ContactMessage

Database table: patient_contactmessage. A stored public contact enquiry containing sender and message information.

Table 46: ContactMessage fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
full_name	CharField	150	-	No	Full name.
email	CharField	254	-	No	Email contact and, for User, login identifier.
phone	CharField	15	-	No	Phone.
subject	CharField	50	-	No	Subject.
message	TextField	-	-	No	Message.
created_at	DateTimeField	-	-	No	Creation timestamp set by the model. Blank allowed.

This entity does not imply a staff reply workflow or email helpdesk integration. Its created_at field records submission time.

patient.Notification

Database table: patient_notification. A short message addressed to one User, including a type, read flag and creation timestamp.

Table 47: Notification fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
user_id	ForeignKey	-	FK accounts.User	No	Linked authentication identity.
notification_type	CharField	30	-	No	Notification type.
message	CharField	255	-	No	Message.
is_read	BooleanField	-	-	No	Whether the message has been read.
created_at	DateTimeField	-	-	No	Creation timestamp set by the model. Blank allowed.

Notifications are in application data, separate from SMTP emails. Default ordering shows newest first. Portal queries should always scope reads and writes to the current account.

Table 48: Notification value domains

Field	Declared choices
notification_type	confirmed = Appointment Confirmed; reminder = Appointment Reminder; cancelled = Appointment Cancelled; general = General

Default ordering: -created_at.

Table 49: Notification referential actions

Reference field	Target model	On deletion
user_id	accounts.User	CASCADE

doctor.Doctor

Database table: doctor_doctor. Professional profile and scheduling parameters for one doctor User.

Table 50: Doctor fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
user_id	OneToOneField	-	FK accounts.User; unique	No	Linked authentication identity.
department	CharField	30	-	No	Selected clinical department code.
specialization	CharField	100	-	No	Specialization. Blank allowed.
phone_number	CharField	20	-	No	Phone number. Blank allowed.
qualification	CharField	150	-	No	Qualification. Blank allowed.
experience_years	PositiveIntegerField	-	-	No	Experience years.
consultation_fee	PositiveIntegerField	-	-	No	Consultation fee.
bio	TextField	-	-	No	Bio. Blank allowed.
profile_picture	FileField	100	-	Yes	Stored profile image file path. Blank allowed.
license_number	CharField	50	-	No	License number. Blank allowed.
registered_since	PositiveIntegerField	-	-	Yes	Registered since. Blank allowed.
slot_duration	PositiveIntegerField	-	-	No	Consultation duration in minutes.
buffer_time	PositiveIntegerField	-	-	No	Gap between generated slots in minutes.

Field name	Logical type	Size	Key	Null	Description
max_per_day	PositiveIntegerField	-	-	Yes	Optional limit on generated slots. Blank allowed.

consultation_fee is an integer amount used to create consultation and no show bills. slot_duration and buffer_time control interval generation. max_per_day is optional; it limits generated entries rather than proving a complete daily throughput rule.

Table 51: Doctor value domains

Field	Declared choices
department	cardiology = Cardiology; neurology = Neurology; orthopedics = Orthopedics; pediatrics = Pediatrics; dermatology = Dermatology; general = General Medicine

Table 52: Doctor referential actions

Reference field	Target model	On deletion
user_id	accounts.User	CASCADE

doctor.DoctorAvailability

Database table: doctor_doctoravailability. One recurring weekday schedule entry for a Doctor.

Table 53: DoctorAvailability fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
doctor_id	ForeignKey	-	FK doctor.Doctor	No	Doctor profile reference.
day	CharField	10	-	No	Day.
is_available	BooleanField	-	-	No	Whether this weekday permits consultations.
start_time	TimeField	-	-	No	Start time.
end_time	TimeField	-	-	No	End time.

The doctor and day pair is unique. is_available=False closes that weekday. When no row exists for a weekday, current slot generation falls back to 09:00 to 17:00.

Table 54: DoctorAvailability value domains

Field	Declared choices
day	monday = Monday; tuesday = Tuesday; wednesday = Wednesday; thursday = Thursday; friday = Friday; saturday = Saturday; sunday = Sunday

Composite uniqueness: doctor, day.

Default ordering: id.

Table 55: DoctorAvailability referential actions

Reference field	Target model	On deletion
doctor_id	doctor.Doctor	CASCADE

doctor.BlockedDate

Database table: doctor_blockeddate. One date specific exception marking a Doctor unavailable.

Table 56: BlockedDate fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
doctor_id	ForeignKey	-	FK doctor.Doctor	No	Doctor profile reference.
date	DateField	-	-	No	Date.
reason	CharField	150	-	No	Reason for consultation. Blank allowed.

The doctor and date pair is unique. A blocked date is checked before the normal weekday rule and therefore overrides an otherwise open schedule.

Composite uniqueness: doctor, date.

Default ordering: date.

Table 57: BlockedDate referential actions

Reference field	Target model	On deletion
doctor_id	doctor.Doctor	CASCADE

doctor.Prescription

Database table: doctor_prescription. A diagnosis and optional clinical advice authored by one Doctor for one Patient.

Table 58: Prescription fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
doctor_id	ForeignKey	-	FK doctor.Doctor	No	Doctor profile reference.
patient_id	ForeignKey	-	FK patient.Patient	No	Patient number when text; patient foreign key when relation.
appointment_id	ForeignKey	-	FK patient.Appointment	Yes	Related visit reference. Blank allowed.
diagnosis	TextField	-	-	No	Doctor recorded diagnosis text.
advice	TextField	-	-	No	Optional clinical advice. Blank allowed.
follow_up_date	DateField	-	-	Yes	Optional proposed follow up date. Blank allowed.

Field name	Logical type	Size	Key	Null	Description
created_at	DateTimeField	-	-	No	Creation timestamp set by the model. Blank allowed.

appointment is optional and uses SET_NULL on appointment deletion. Patient and doctor deletion cascade to the prescription. The view separately validates any appointment association.

Default ordering: -created_at.

Table 59: Prescription referential actions

Reference field	Target model	On deletion
doctor_id	doctor.Doctor	CASCADE
patient_id	patient.Patient	CASCADE
appointment_id	patient.Appointment	SET_NULL

doctor.PrescriptionMedicine

Database table: doctor_prescriptionmedicine. One medicine detail row owned by a Prescription.

Table 60: PrescriptionMedicine fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
prescription_id	ForeignKey	-	FK doctor.Prescription	No	Parent prescription reference.
name	CharField	100	-	No	Name.
dosage	CharField	50	-	No	Dosage. Blank allowed.
frequency	CharField	50	-	No	Frequency. Blank allowed.
duration	CharField	50	-	No	Duration. Blank allowed.
instructions	CharField	150	-	No	Instructions. Blank allowed.

The parent relationship is many to one. The view saves rows with a nonempty name and fills absent optional list positions with empty strings. Frequency choices are enumerated but require server validation when accepting raw POST data.

Table 61: PrescriptionMedicine value domains

Field	Declared choices
frequency	once_a_day = Once a day; twice_a_day = Twice a day; thrice_a_day = Thrice a day; every_6_hours = Every 6 hours; every_8_hours = Every 8 hours; as_needed = As needed

Table 62: PrescriptionMedicine referential actions

Reference field	Target model	On deletion
prescription_id	doctor.Prescription	CASCADE

receptionist.Receptionist

Database table: receptionist_receptionist. Front desk profile for one User with employee identifier, contact, shift and joining date.

Table 63: Receptionist fields

Field name	Logical type	Size	Key	Null	Description
id	BigAutoField	-	PK	No	Generated row identifier. Blank allowed.
user_id	OneToOneField	-	FK accounts.User; unique	No	Linked authentication identity.
employee_id	CharField	20	-; unique	No	Unique generated employee number. Blank allowed.
phone	CharField	15	-	No	Phone. Blank allowed.
profile_picture	FileField	100	-	Yes	Stored profile image file path. Blank allowed.
shift	CharField	10	-	No	Shift.
date_joined	DateField	-	-	No	Date joined. Blank allowed.
updated_at	DateTimeField	-	-	No	Last model update timestamp. Blank allowed.

employee_id is generated when absent using the last row identifier and a REC prefix. The concurrency limitation of this sequence approach is similar to Patient.patient_id.

Table 64: Receptionist value domains

Field	Declared choices
shift	morning = Morning; evening = Evening; night = Night

Table 65: Receptionist referential actions

Reference field	Target model	On deletion
user_id	accounts.User	CASCADE

Integrity rules and normalisation

Identity and role profiles are separated to avoid repeating password and role data in each domain table. Appointment references patient and doctor profiles rather than copying full demographic records into every visit. PrescriptionMedicine removes the need for a fixed set of medicine columns and allows a prescription to contain several detail rows.

Some redundancy remains purposeful but requires care. Billing stores patient separately from appointment even though the appointment already identifies the patient. Appointment stores a department value even though its doctor also has a department. These values should be treated as explicit business data whose consistency or historical meaning must be defined; they should not be assumed to synchronise automatically after a profile change.

CASCADE deletion can remove substantial related history. For example, deleting a patient profile can remove appointments and bills and also prescriptions through their patient reference. This differs from deactivating the linked account, which changes login eligibility

without deleting those domain rows. An operational deployment should prefer a defined retention and archival policy rather than casual physical deletion.

Framework support tables include sessions, authentication groups and permissions, content types and Django administration logs. User has inherited many to many relationships to groups and user_permissions, represented by junction tables. These structures support the framework; the custom portal authorisation primarily reads the role field and its view decorators.

3.3 User Interface Design

The application uses a public patient themed base template and separate staff portal base templates. Bootstrap provides layout utilities and components, while role specific CSS supplies visual identity. Staff portals use navigation sidebars and top bars; patient pages use the public site navigation and focused dashboard panels. Forms submit to Django views and include CSRF tokens where defined.

The following screenshots are genuine images embedded in the supplied project report. They represent saved reference states, not a new browser execution of the current application. They are reproduced in grayscale for body printing. Screen behaviour and fields are documented from current templates and views; data visible in an older screenshot is not treated as current database evidence.

Repeated profile, password and notification patterns are documented as shared designs after the main portal screens. Browser validation messages supplement server validation and must not be interpreted as the only enforcement. The application does not use a separate frontend state store; page context, local scripts and the server session carry the relevant state.

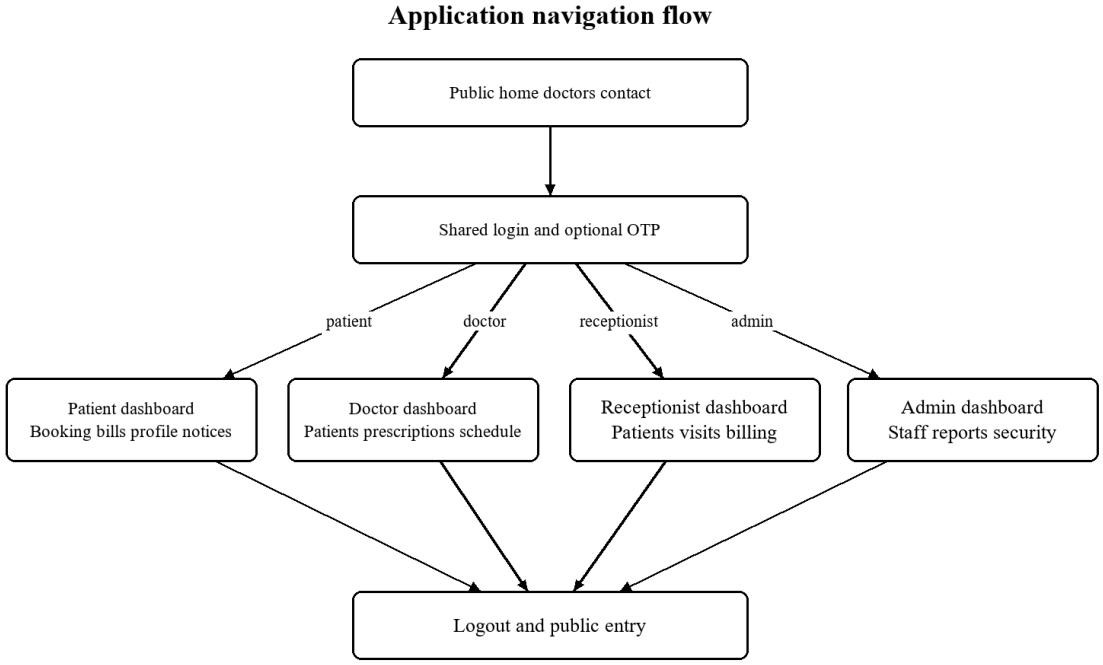


Figure 11: Role based entry points and navigation

Shared login

Permitted users: Public users.

The login page presents email, password and role selection in one form. Role selection guides the user but is checked against the saved account role. Successful password authentication either proceeds to OTP or directly to the role dashboard according to the runtime policy.

Template field names: role, email, password, remember_me.

Visible actions: Login.

Validation and navigation: Invalid credentials show a generic message. The selected role cannot elevate account permissions. Password reset and patient registration links provide recovery and onboarding paths.

Template: templates/patient/login.html

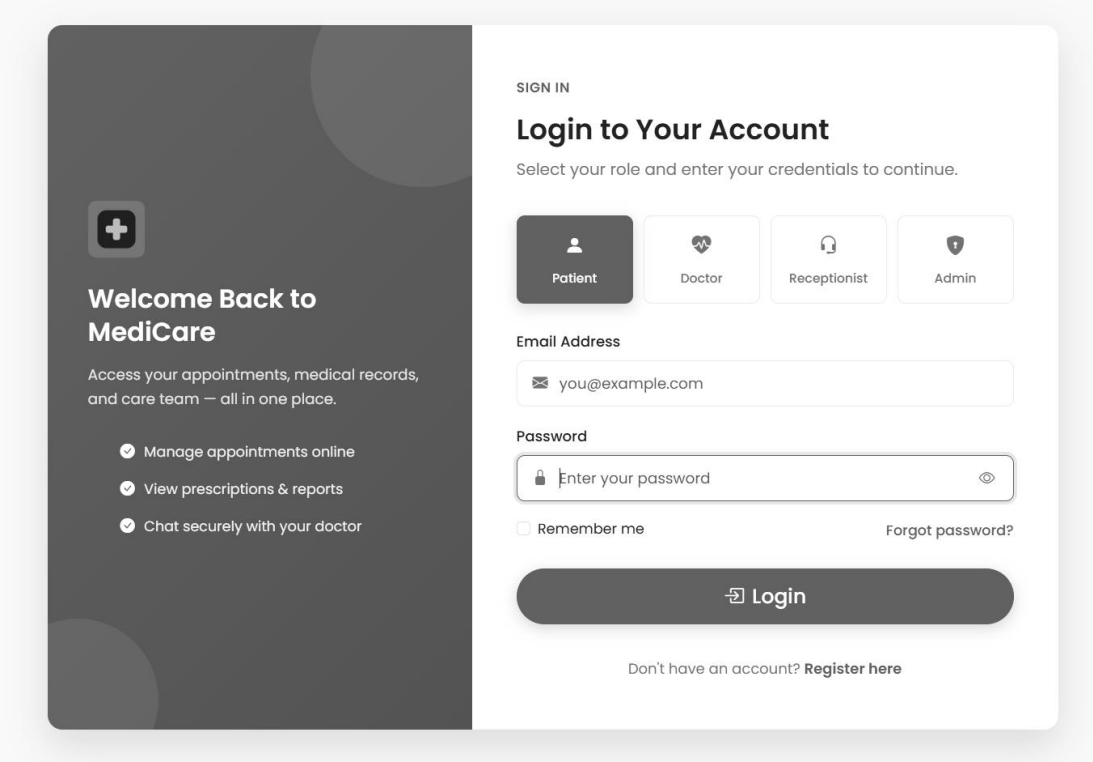


Figure 12: Shared login — supplied project screenshot

Patient registration

Permitted users: Public users.

The registration form gathers identity, contact and login information for patient self service. It explicitly directs staff onboarding to hospital administration. Personal values populate the linked Patient profile after the User is created.

Template field names: full_name, email, phone, date_of_birth, gender, password, confirm_password, agree_terms.

Visible actions: Create Account.

Validation and navigation: The handler checks its required values, name, duplicate identity and password rules. Submitted values should be tested independently of browser required attributes, because the template uses custom validation.

Template: templates/patient/register.html

OTP verification

Permitted users: Pending login users.

The verification page accepts the code for the pending user stored in the session. It presents the remaining expiry interval, attempts and resend interface. Completing verification is the point at which the enabled OTP flow creates the authenticated session.

Template field names: otp_code.

Visible actions: Verify & Login; Resend code.

Validation and navigation: A missing pending session returns to login. Expired codes and wrong codes are rejected. The displayed resend countdown does not by itself prove a server enforced resend rate limit.

Template: templates/patient/verify_otp.html

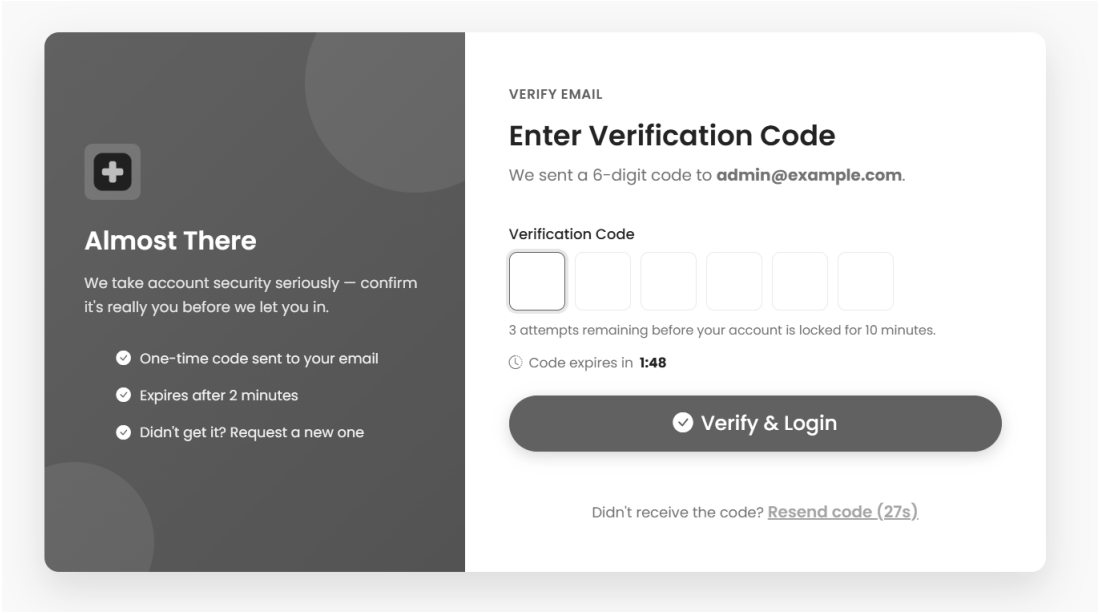


Figure 13: OTP verification — supplied project screenshot

Password recovery

Permitted users: Public users.

The recovery form accepts an email address and initiates a token based reset message for an eligible account. Its purpose is to restore access without asking staff to know the old password.

Template field names: email.

Visible actions: Send Reset Link; Resend Email.

Validation and navigation: A controlled test should compare responses for registered and unknown addresses and inspect token expiry. External delivery is a separate integration condition.

Template: templates/patient/forgot_password.html

Set a replacement password

Permitted users: Valid reset link holders.

The reset page is reached with the encoded user identifier and token in the URL. It collects a replacement password and confirmation after checking the token.

Template field names: new_password, confirm_new_password.

Visible actions: Reset Password.

Validation and navigation: Invalid tokens cannot authorise a reset. Password validation is handled by the corresponding view and configured validators.

Template: templates/patient/reset_password_confirm.html

Patient dashboard

Permitted users: Patient.

The dashboard summarises the current patients visit and billing context and provides shortcuts to booking and personal lists. The role focused page avoids hospital wide administrative controls.

Validation and navigation: Records shown here must remain scoped to the patient linked to the authenticated account. Counts should be compared with the same patients underlying records during acceptance testing.

Template: templates/patient/dashboard.html

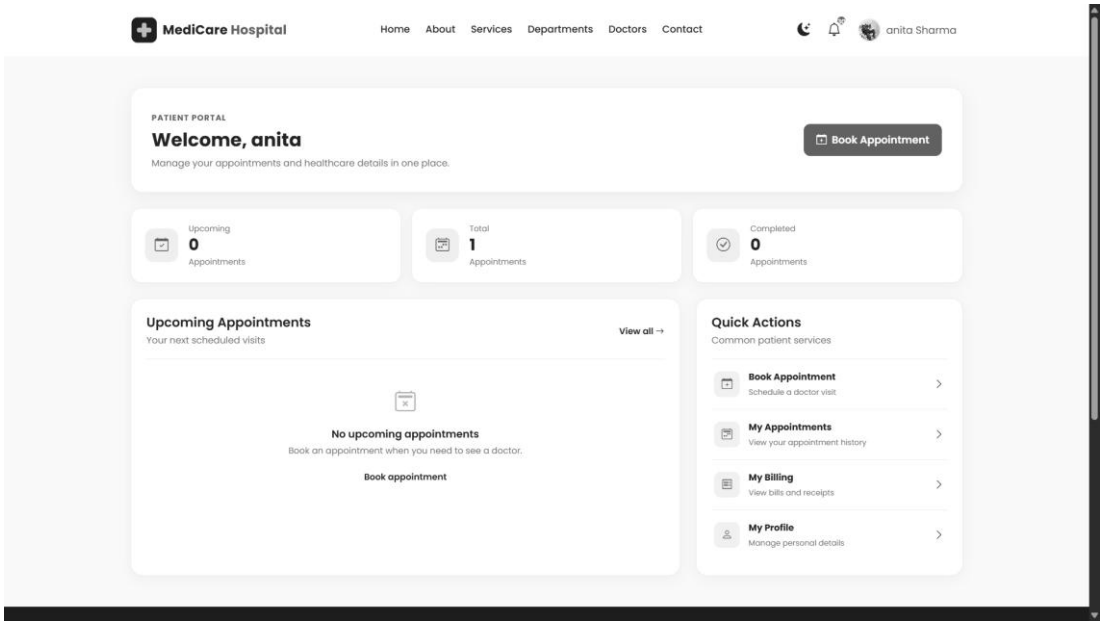


Figure 14: Patient dashboard — supplied project screenshot

Doctor directory

Permitted users: Public and authenticated visitors.

The directory displays doctor profiles and supported filtering. It is the entry point for selecting an appropriate consultation provider rather than a staff editing interface.

Visible actions: All; Cardiology; Neurology; Orthopedics; Pediatrics; Dermatology.

Validation and navigation: The displayed hospital name and marketing text are presentation content. They do not establish an actual operating hospital or a validated clinical service.

Template: templates/patient/doctors.html

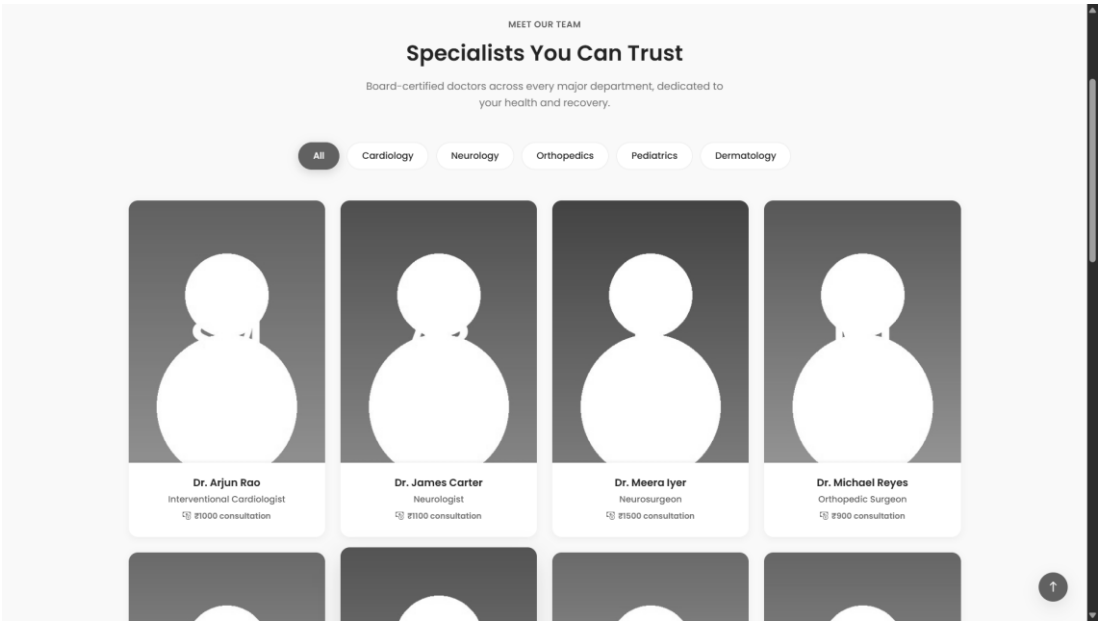


Figure 15: Doctor directory — supplied project screenshot

Patient appointment form

Permitted users: Patient.

The booking form allows doctor, department, date, slot, visit type and reason selection. A date specific JSON enquiry supplies slot options. Initial submission validates the candidate and stores a session draft.

Template field names: department, doctor, appointment_date, patient_name, patient_phone, patient_email, visit_type, reason.

Visible actions: Confirm Appointment.

Validation and navigation: A displayed free slot is not reserved until final creation. The server rechecks availability; a malformed date or occupied slot interrupts the flow.

Template: templates/patient/book_appointment.html

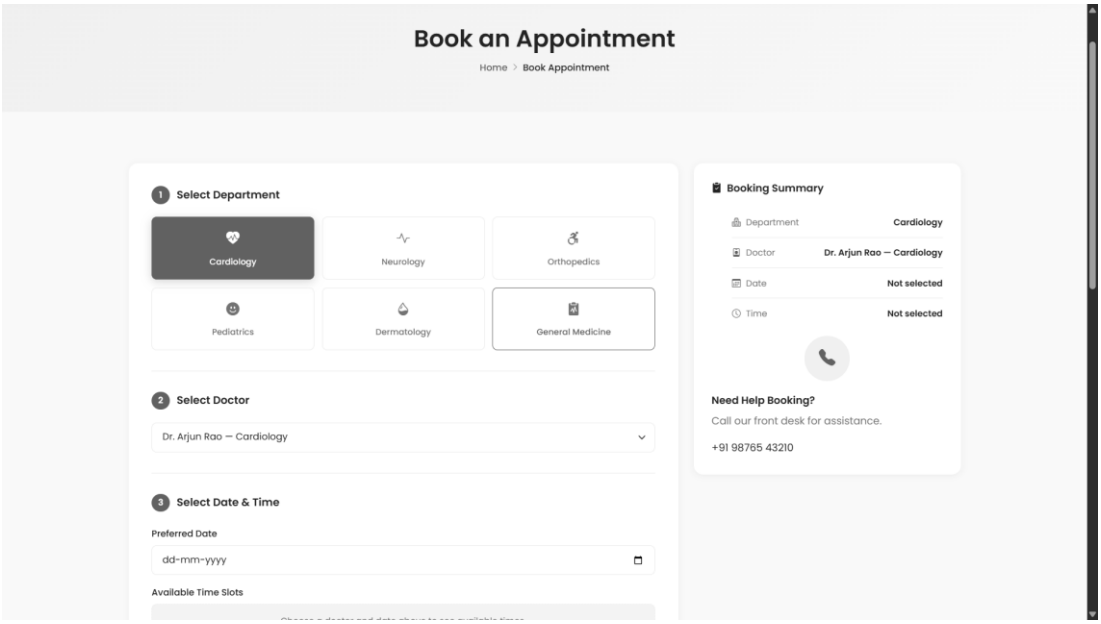


Figure 16: Patient appointment form — supplied project screenshot

Appointment and fee review

Permitted users: Patient with pending draft.

The review screen shows the selected doctor, consultation fee and visit details from the pending session. It gives the patient a chance to verify the choice before proceeding.

Validation and navigation: This page does not itself create the Appointment or bill. Its proceed link opens checkout, and absent pending data sends the user back to booking.

Template: templates/patient/confirm_billing.html

Demonstration card checkout

Permitted users: Patient with pending draft.

The card styled page demonstrates the final booking interaction. On submission the backend rechecks the slot, creates the appointment and records a bill as paid for the demonstration.

Validation and navigation: No payment gateway is contacted and card shaped values are not validated or stored by the inspected handler. Use demonstration values only; the result is a simulated payment status.

Template: templates/patient/card_payment.html

Personal appointments

Permitted users: Patient.

This list displays the patients own visits with doctor, date, time and status information. Supported cancellation is exposed through the corresponding action.

Visible actions: All; Upcoming; Completed; Cancelled; Keep Appointment; Yes, Cancel.

Validation and navigation: The list must not contain other patient records. A cancelled appointment frees its slot in subsequent slot generation, but cancellation is not a gateway refund.

Template: templates/patient/my_appointments.html

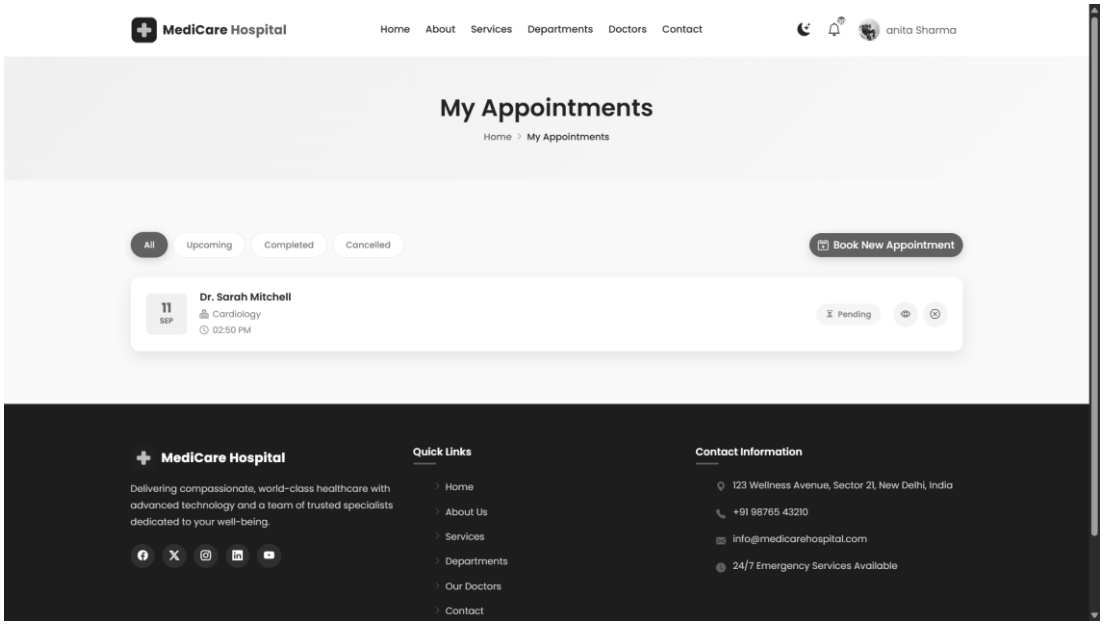


Figure 17: Personal appointments — supplied project screenshot

Personal bills

Permitted users: Patient.

The billing page presents charges linked to the current patient and provides receipt navigation. Consultation and no show charges can be distinct rows for one visit.

Validation and navigation: Amount and payment status are database values. A paid entry from demo checkout does not establish an external financial settlement.

Template: templates/patient/my_billing.html

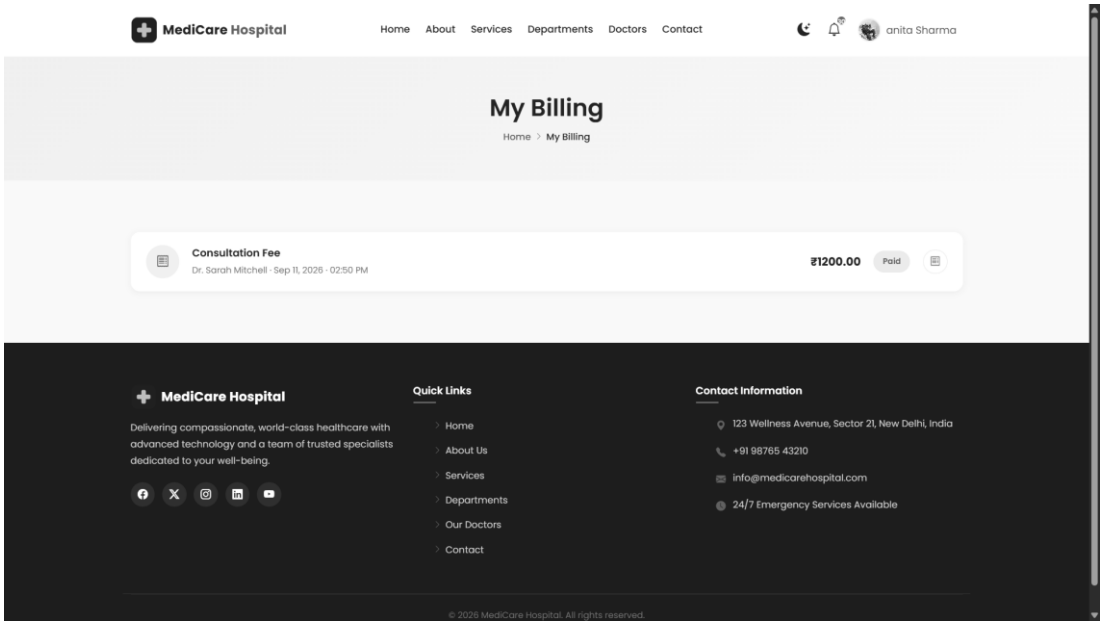


Figure 18: Personal bills — supplied project screenshot

Patient profile

Permitted users: Patient.

The profile gathers demographic, contact, emergency contact and background medical information and supports an image. Password change is presented in the patient profile flow.

Template field names: profile_image, first_name, last_name, email, phone, date_of_birth, gender, blood_group, address, city, state, country, pincode, emergency_contact_name, emergency_contact_number, allergies, medical_history, current_password, new_password, confirm_new_password.

Visible actions: Save Changes; Update Password; Save Preferences.

Validation and navigation: Image validation checks extension, size and decoded content before save. The current account identity determines the editable profile.

Template: templates/patient/profile.html

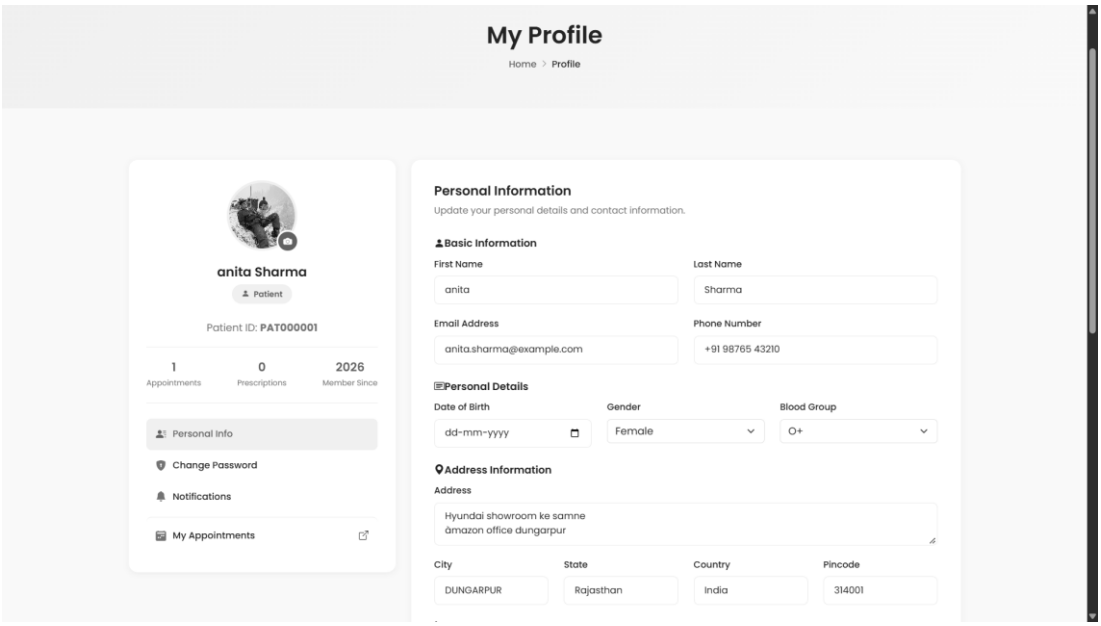


Figure 19: Patient profile — supplied project screenshot

Public contact form

Permitted users: Public visitors.

The contact form stores a sender name, email, phone, subject and message in ContactMessage. It is a basic enquiry intake interface.

Template field names: full_name, email, phone, subject, message.

Visible actions: Send Message.

Validation and navigation: Contact submission does not imply an implemented ticket assignment, staff reply workflow or external messaging integration.

Template: templates/patient/contact.html

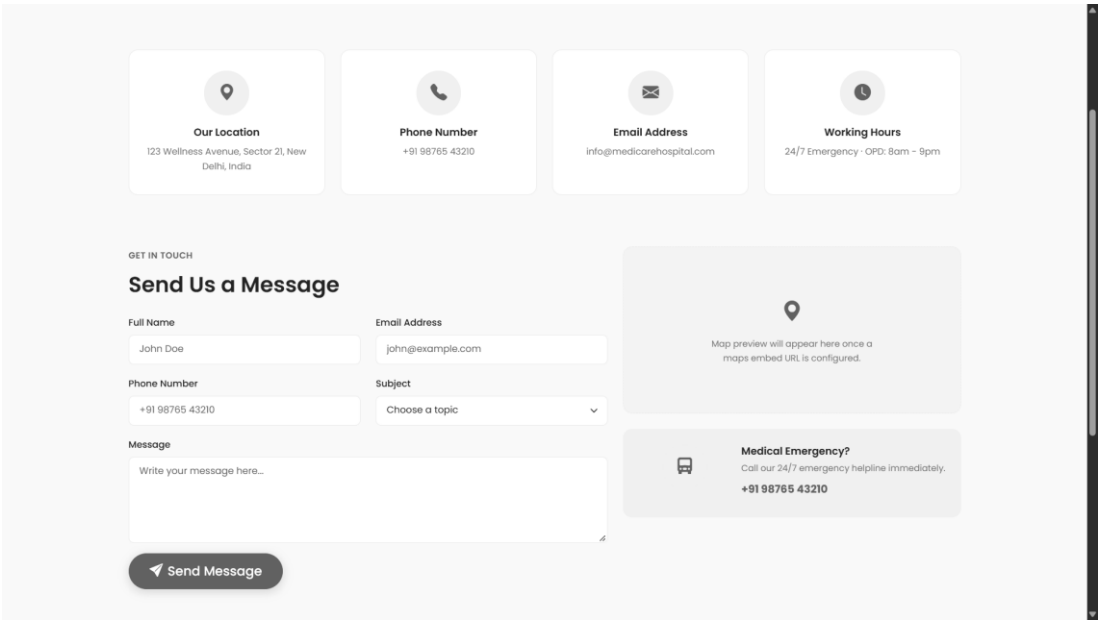


Figure 20: Public contact form — supplied project screenshot

Administrator dashboard

Permitted users: Administrator.

The dashboard presents hospital wide counts, financial and appointment summaries and chart panels. The view prepares aggregations from domain models and the template includes Chart.js.

Validation and navigation: Chart totals depend on the query definitions and time range. CDN availability and JavaScript execution affect chart display without changing stored records.

Template: templates/adminpanel/dashboard.html

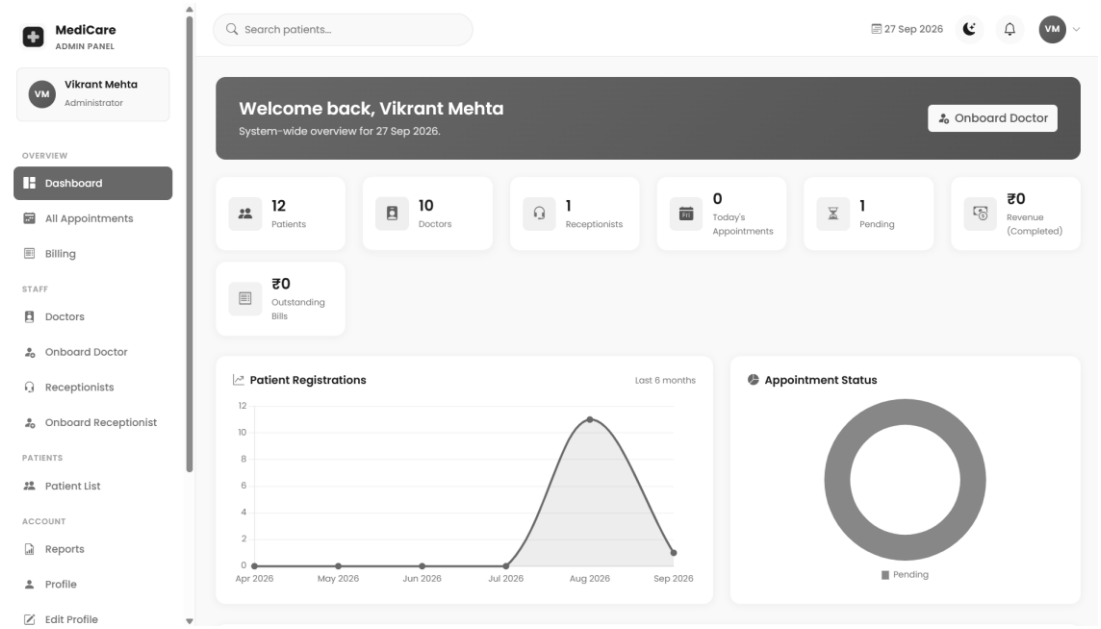


Figure 21: Administrator dashboard — supplied project screenshot

Administrator doctor list

Permitted users: Administrator.

This list supports oversight of doctor profiles and navigation to creation and detail views. Search and department related presentation help locate staff.

Template field names: q, next.

Visible actions: Cancel; Confirm.

Validation and navigation: Account activation controls affect login eligibility, while doctor profile fields remain separate persistent records. Administrative identity is required server side.

Template: templates/adminpanel/doctor_list.html

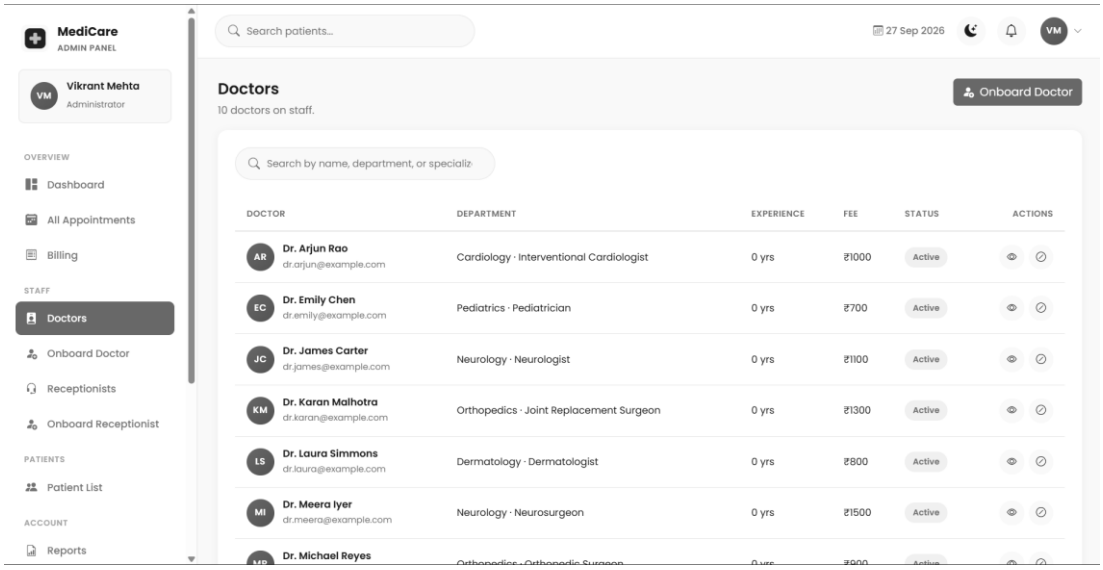


Figure 22: Administrator doctor list — supplied project screenshot

Create doctor

Permitted users: Administrator.

The doctor creation form collects account credentials and professional details such as department, qualification, experience and fee. The saved User must carry doctor role and have a linked Doctor profile.

Template field names: full_name, email, password, confirm_password, department, specialization, phone_number, experience_years, consultation_fee, qualification, license_number.

Visible actions: Onboard Doctor.

Validation and navigation: The view checks required and duplicate account values and validates its numeric inputs. Public registration does not offer this operation.

Template: templates/adminpanel/add_doctor.html

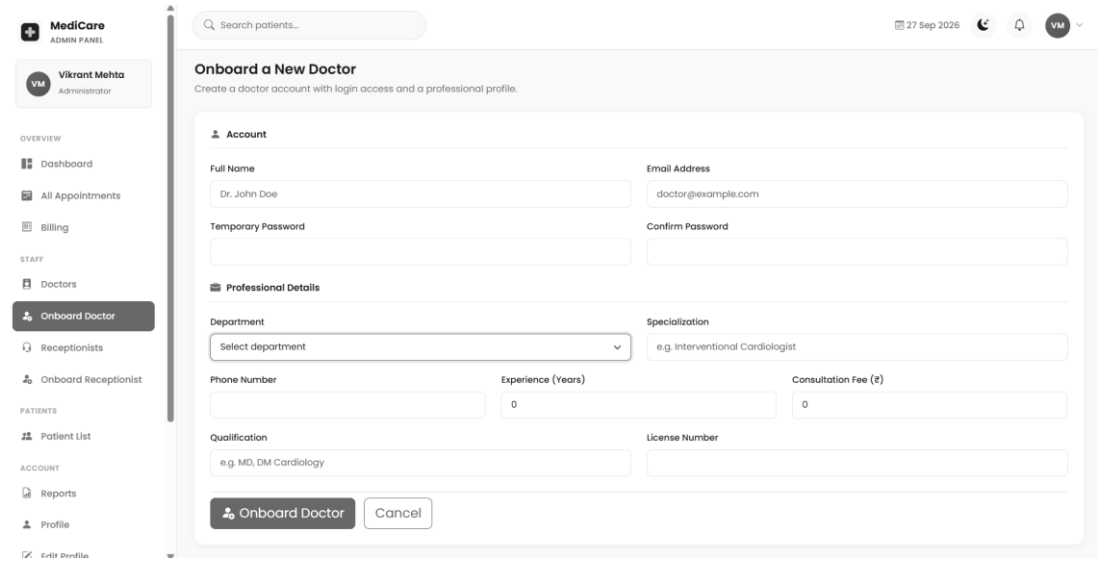


Figure 23: Create doctor — supplied project screenshot

Receptionist management

Permitted users: Administrator.

The list displays front desk staff and permits the implemented shift selection and account status actions. It links to a separate creation form.

Template field names: q, next, shift.

Visible actions: Cancel; Confirm.

Validation and navigation: Shift updates accept only declared shift values. The role guard and supported POST handling apply even when a form auto submits from a dropdown.

Template: templates/adminpanel/receptionist_list.html

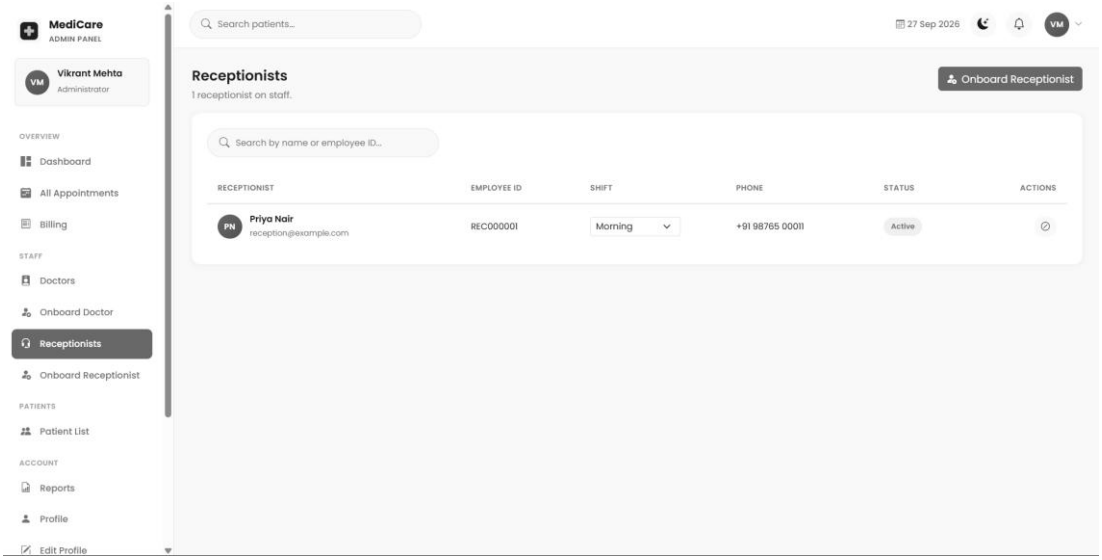


Figure 24: Receptionist management — supplied project screenshot

Create receptionist

Permitted users: Administrator.

The staff creation form builds a receptionist role User and the linked profile with contact and shift information. A generated employee identifier supports later display.

Template field names: full_name, email, password, confirm_password, phone, shift.

Visible actions: Onboard Receptionist.

Validation and navigation: Creation should preserve agreement between role and profile. The current sequence based employee number generation needs a concurrency test before high volume onboarding.

Template: templates/adminpanel/add_receptionist.html

Administrator patient list

Permitted users: Administrator.

The hospital wide patient list supports operational search and detail navigation. It differs from the doctors list, which is constrained by appointment relationships.

Template field names: q, next.

Visible actions: Cancel; Confirm.

Validation and navigation: Account access changes should be distinguished from deletion of a patient record. Deactivation preserves the domain data while changing login eligibility.

Template: templates/adminpanel/patient_list.html

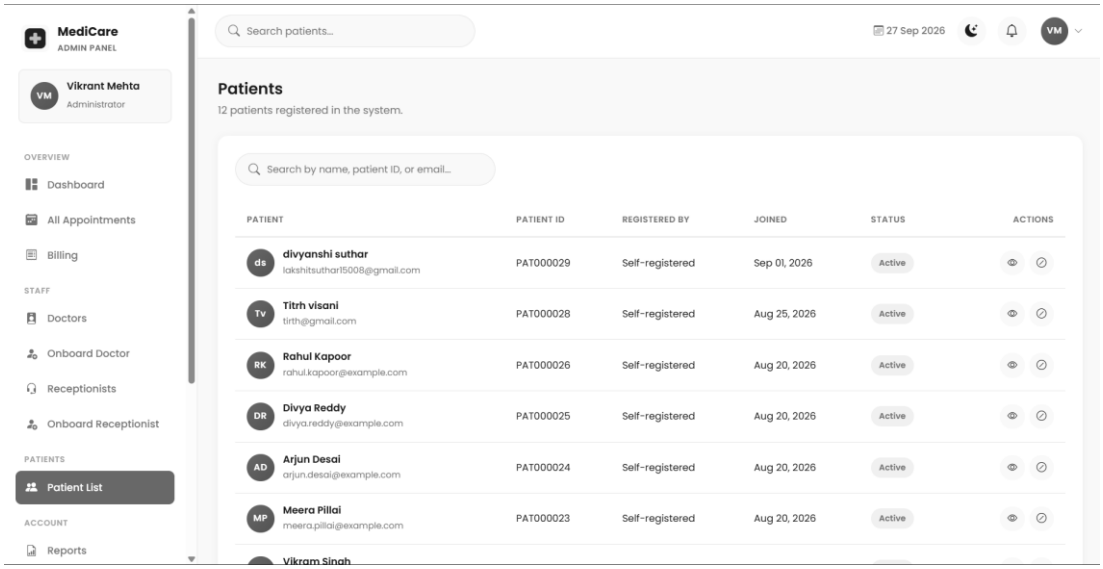


Figure 25: Administrator patient list — supplied project screenshot

Hospital appointment oversight

Permitted users: Administrator.

The appointment list displays visits across doctors and patients with supported filters and status editing. An inline status selector supports correction of a prior operational state.

Template field names: q, status, next.

Validation and navigation: The administrative handler validates the status choice but does not reproduce every side effect of the receptionist no show or cancellation workflow. A status correction alone is not equivalent to executing those business actions.

Template: templates/adminpanel/appointment_list.html

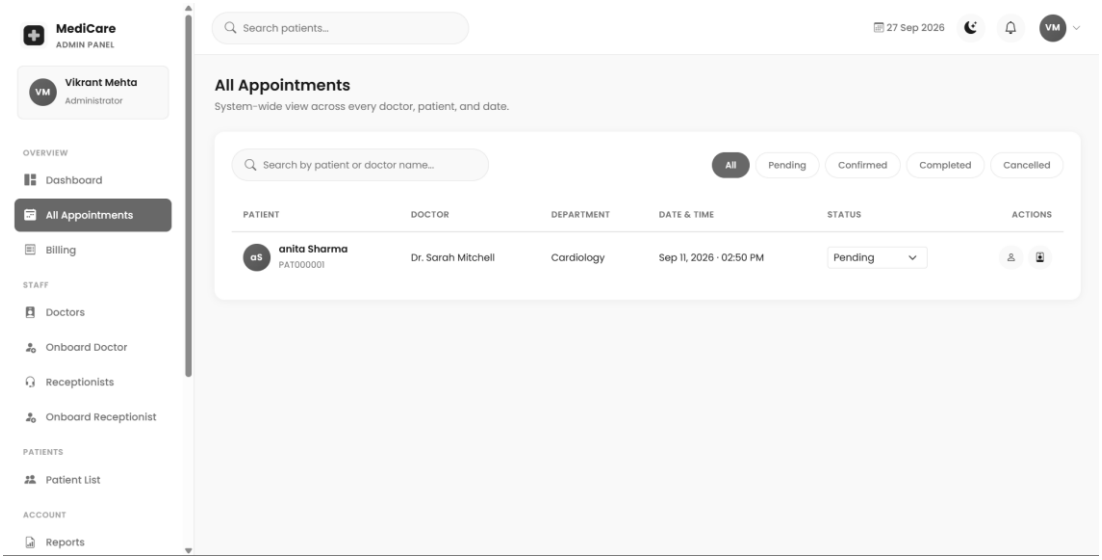


Figure 26: Hospital appointment oversight — supplied project screenshot

Hospital billing oversight

Permitted users: Administrator.

The billing list presents hospital charges and outstanding totals and offers mark paid and receipt actions. Search uses the implemented patient name and identifier fields.

Template field names: q, next.

Validation and navigation: The pending total is calculated from all pending bills, whereas the visible list may be filtered. Examiners should not assume the total is necessarily limited to the displayed search.

Template: templates/adminpanel/billing_list.html

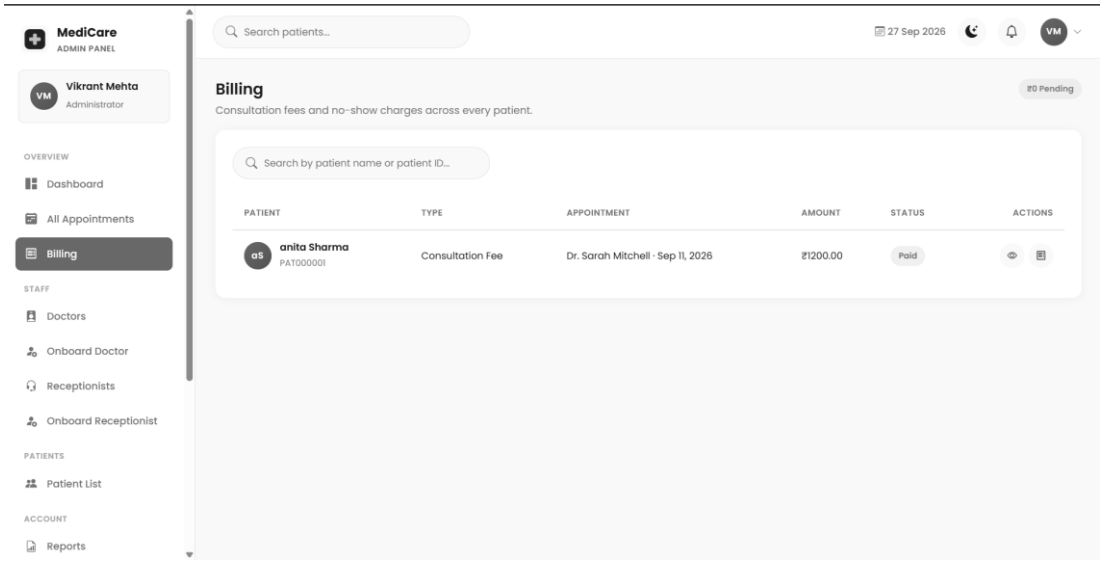


Figure 27: Hospital billing oversight — supplied project screenshot

Report centre

Permitted users: Administrator.

The report centre selects a report category and relevant dates, statuses, department, doctor, bill type and search. Submission opens the routed report generator.

Template field names: report_type, start_date, end_date, status, department, doctor, bill_type.

Visible actions: Generate Report.

Validation and navigation: Only filters applied by the selected report branch affect rows. The current implementation exports CSV and PDF and should not be described as screenshot only reporting.

Template: templates/adminpanel/generate_report.html

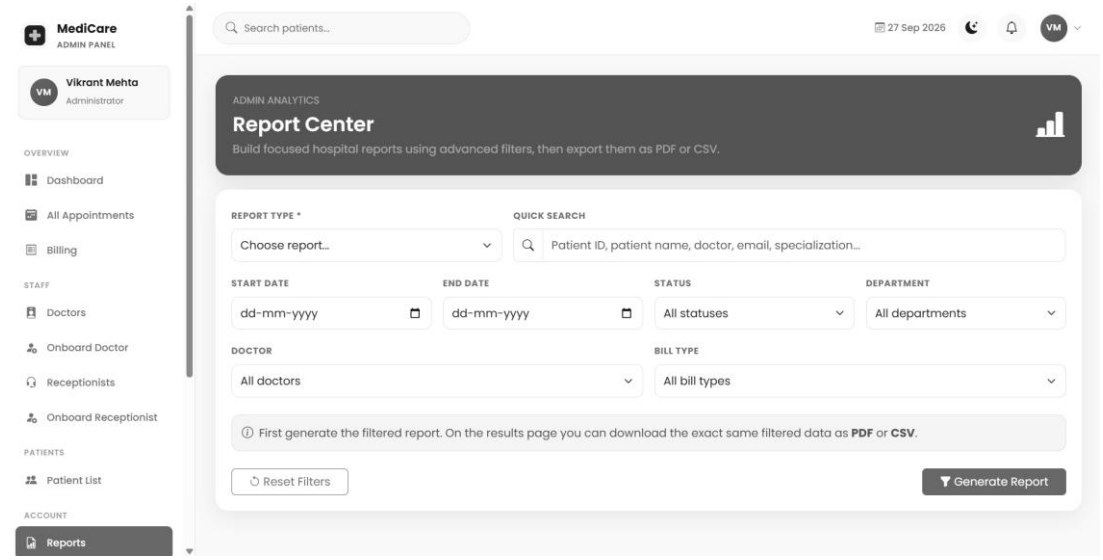


Figure 28: Report centre — supplied project screenshot

Filtered report result

Permitted users: Administrator.

The result view displays a filter summary, columns, rows and totals where appropriate, with export links. It is the browser output from the same prepared rows used by download responses.

Validation and navigation: Billing totals include the matched rows regardless of whether their statuses represent collected money. Report context must accompany a printed or downloaded copy.

Template: templates/adminpanel/report_result.html

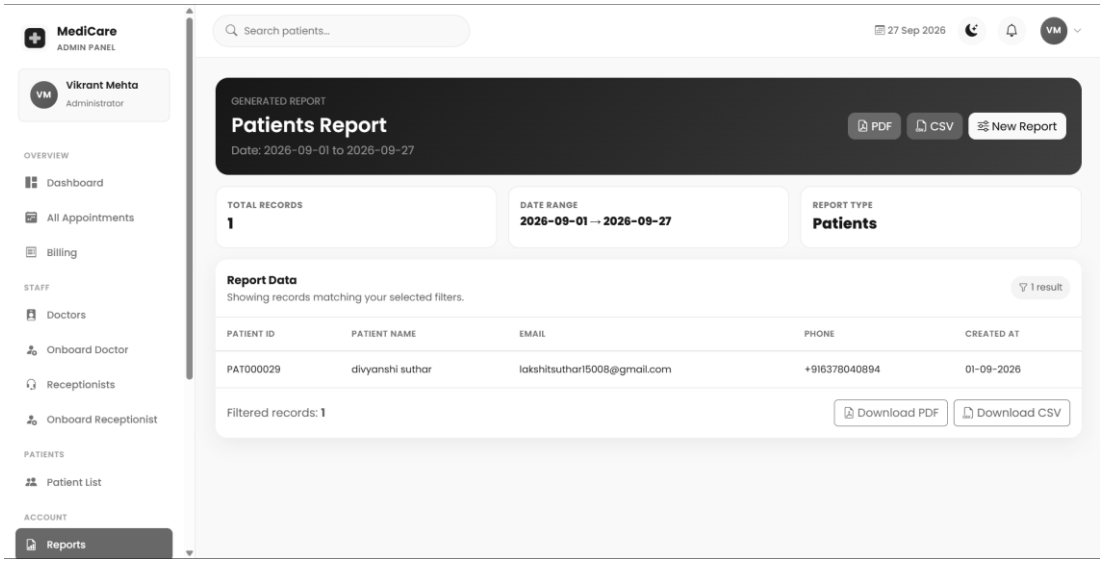


Figure 29: Filtered report result — supplied project screenshot

OTP security settings

Permitted users: Administrator.

A single checkbox controls the database backed OTP policy for all roles. The template auto submits changes and presents the stored state.

Template field names: otp_login_enabled.

Validation and navigation: The switch is global, not a preference for the current administrator. Authentication reads SystemSettings.get_solo rather than treating the older settings constant as the live toggle.

Template: templates/adminpanel/security_settings.html

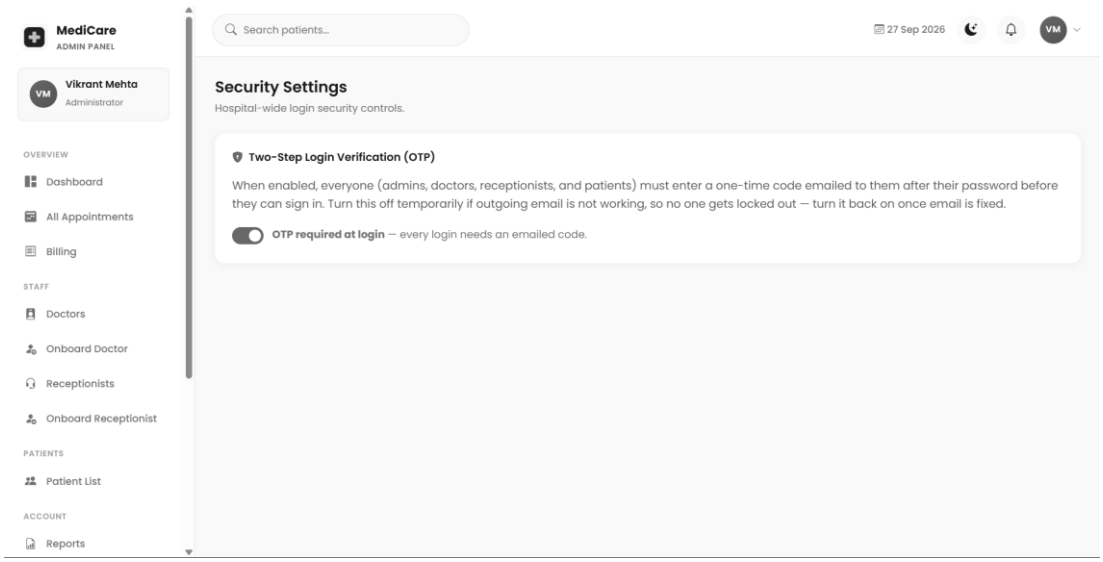


Figure 30: OTP security settings — supplied project screenshot

Doctor dashboard

Permitted users: Doctor.

The dashboard provides the current doctors activity summary and shortcuts to appointments, patients and prescriptions. It is scoped to the doctor profile.

Validation and navigation: No hospital wide billing administration or staff onboarding is implied by the doctor dashboard. Summary values must be evaluated against the scoped view queries.

Template: templates/doctor/dashboard.html

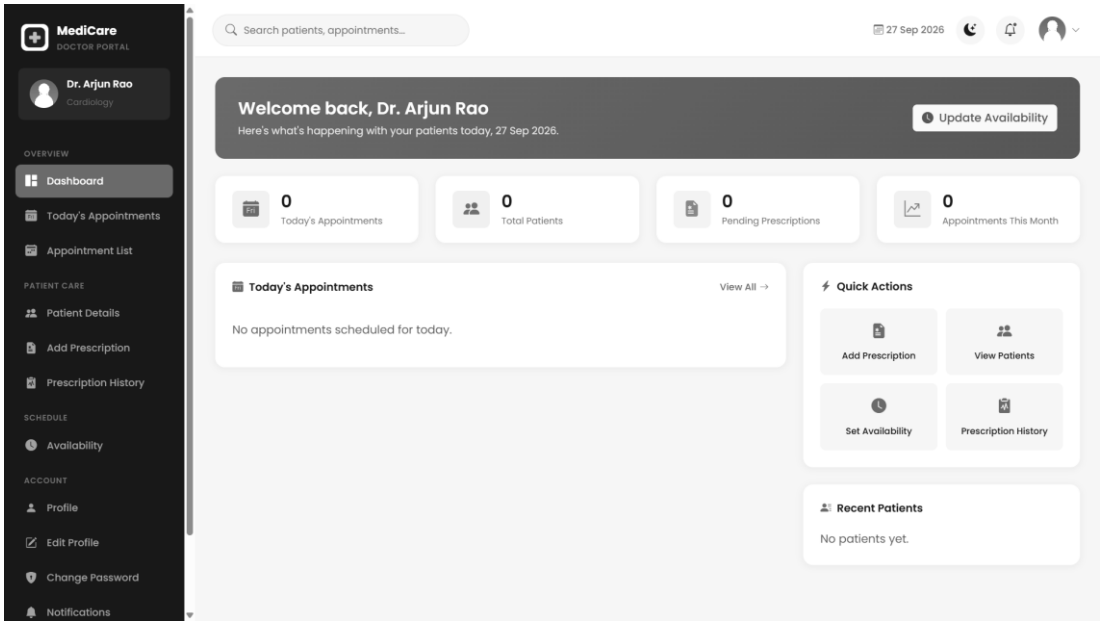


Figure 31: Doctor dashboard — supplied project screenshot

Doctor appointment list

Permitted users: Doctor.

The list exposes the doctors own scheduled visits and supported date, status or search filters. Completion and cancellation lead back through safe return routing.

Template field names: q, status, next.

Visible actions: Keep Appointment; Yes, Cancel.

Validation and navigation: Object retrieval in write handlers is also scoped to the doctor. A hidden action button is not the sole access control.

Template: templates/doctor/appointment_list.html

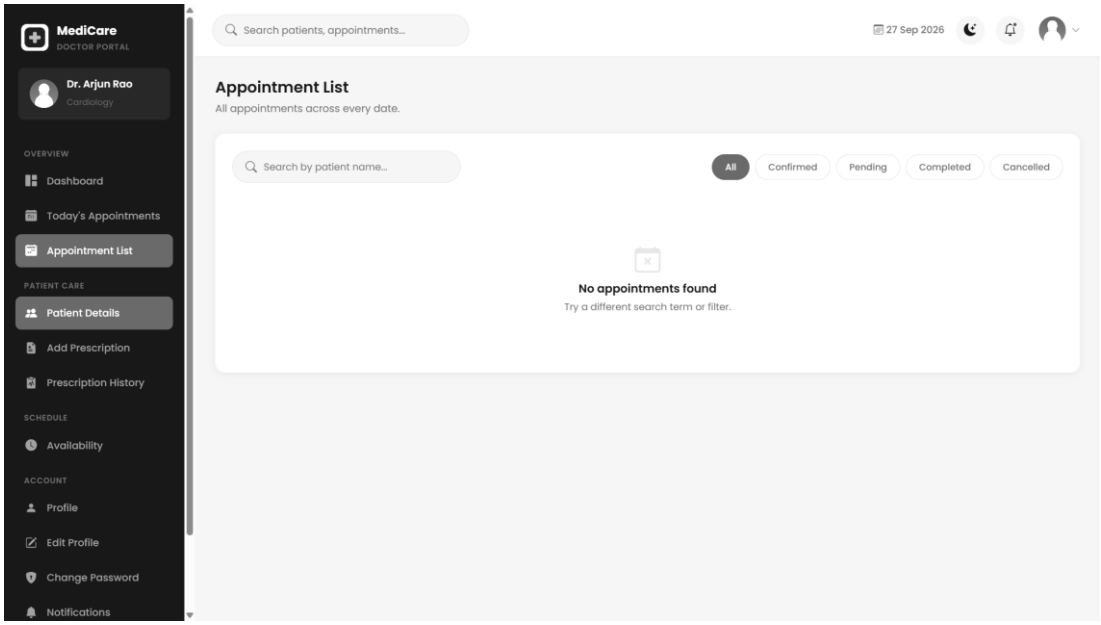


Figure 32: Doctor appointment list — supplied project screenshot

Related patient list

Permitted users: Doctor.

The patient list joins Patient through appointments belonging to the current doctor, removes duplicates and supports search over names, email and phone.

Template field names: q.

Validation and navigation: A patient with several visits appears once in the directory. A patient without a relationship to the doctor is outside this list.

Template: templates/doctor/patient_list.html

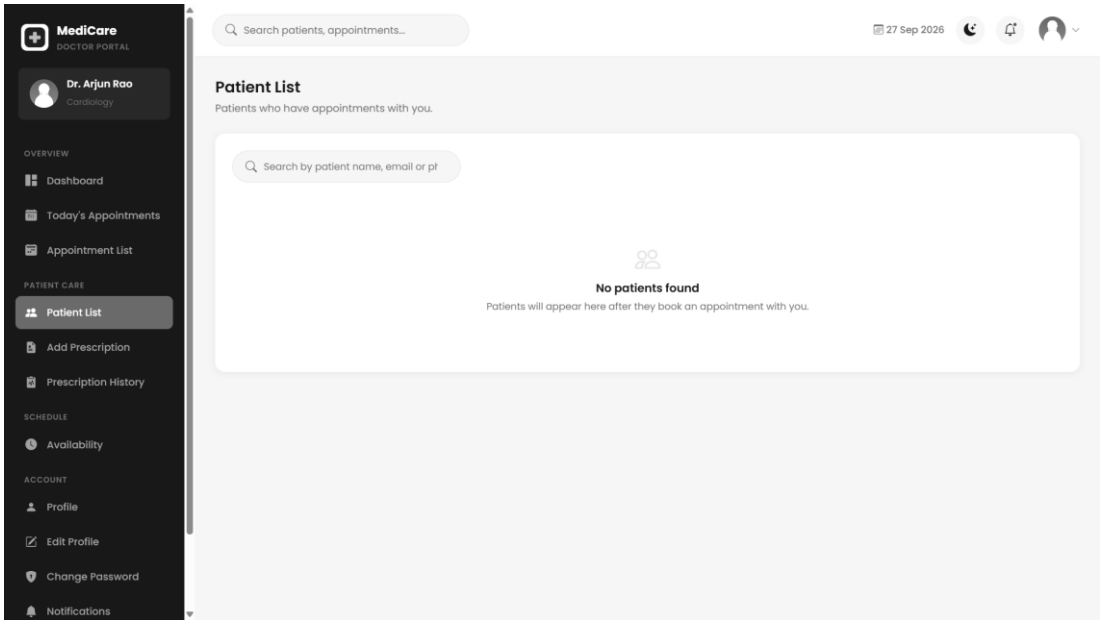


Figure 33: Related patient list — supplied project screenshot

Prescription entry

Permitted users: Doctor.

The form selects a related patient and optional visit, gathers diagnosis and advice and supports dynamic medicine rows. Parallel arrays carry each medicine line to the view.

Template field names: patient_id, appointment_id, diagnosis, medicine_name[], dosage[], frequency[], duration[], instructions[], advice, follow_up_date.

Visible actions: Add Medicine; Save Prescription.

Validation and navigation: Patient and diagnosis are required. A linked appointment must match both doctor and patient. Empty medicine names are skipped instead of creating blank medicine entries.

Template: templates/doctor/add_prescription.html

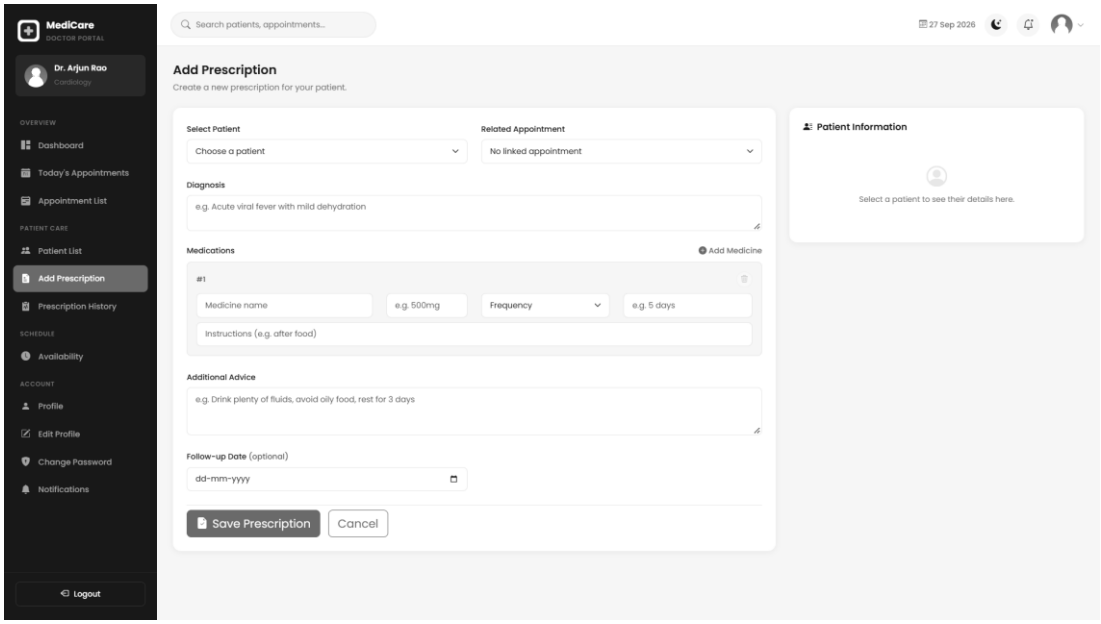


Figure 34: Prescription entry — supplied project screenshot

Doctor prescription history

Permitted users: Doctor.

The history lists prescriptions written by the doctor with medicine details, patient context and supported filters. The view prefetches medicines to reduce repeated related queries.

Template field names: q, sort.

Visible actions: Close; Print.

Validation and navigation: This is a doctor portal capability. It does not establish a matching patient prescription route or a separate prescription PDF endpoint.

Template: templates/doctor/prescription_history.html

Doctor availability

Permitted users: Doctor.

The scheduling screen presents weekday checkboxes and start/end times, slot duration, buffer, optional cap and repeating blocked date rows. Saving updates weekday rules and replaces the submitted blocked date set.

Template field names: available_{{ day.day }}, start_{{ day.day }}, end_{{ day.day }}, slot_duration, buffer_time, max_per_day, blocked_date[], blocked_reason[].

Visible actions: Copy to weekdays; Save Availability.

Validation and navigation: New blocked dates must begin tomorrow or later; existing blocked dates are allowed to survive resubmission. The unsaved screen defaults differ from slot generation fallback on Saturday and Sunday, an issue documented for improvement.

Template: templates/doctor/availability.html

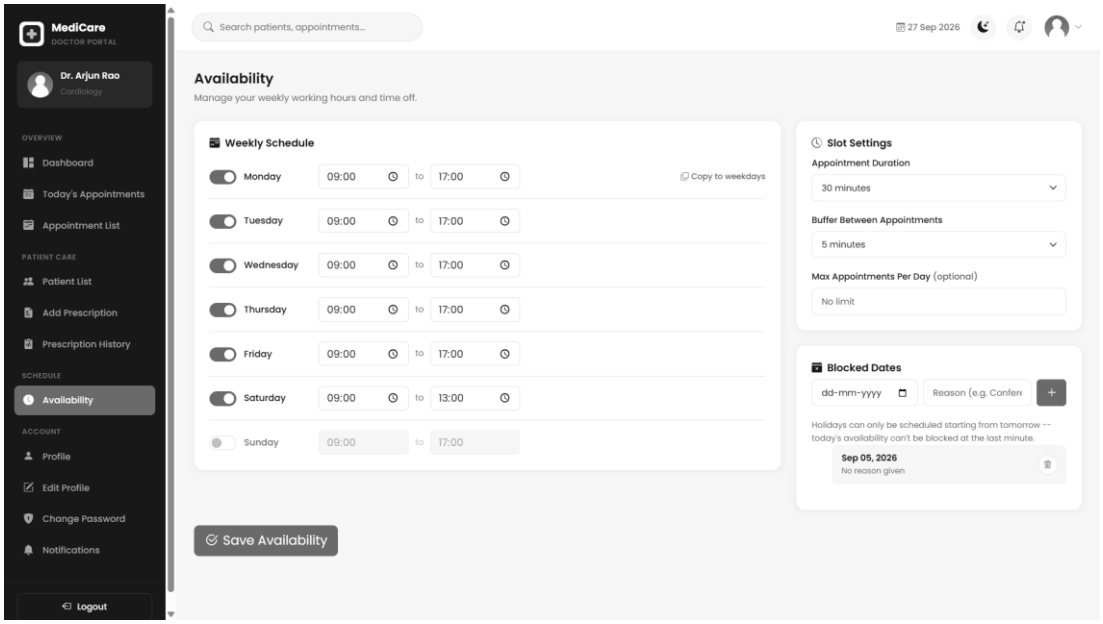


Figure 35: Doctor availability — supplied project screenshot

Doctor professional profile

Permitted users: Doctor.

The profile displays professional information, contact details, fees and scheduling related context, and links to edit and password pages.

Validation and navigation: The authenticated doctor profile is the edit target. Professional information is also reused in patient discovery and administrator reporting.

Template: templates/doctor/doctor_profile.html

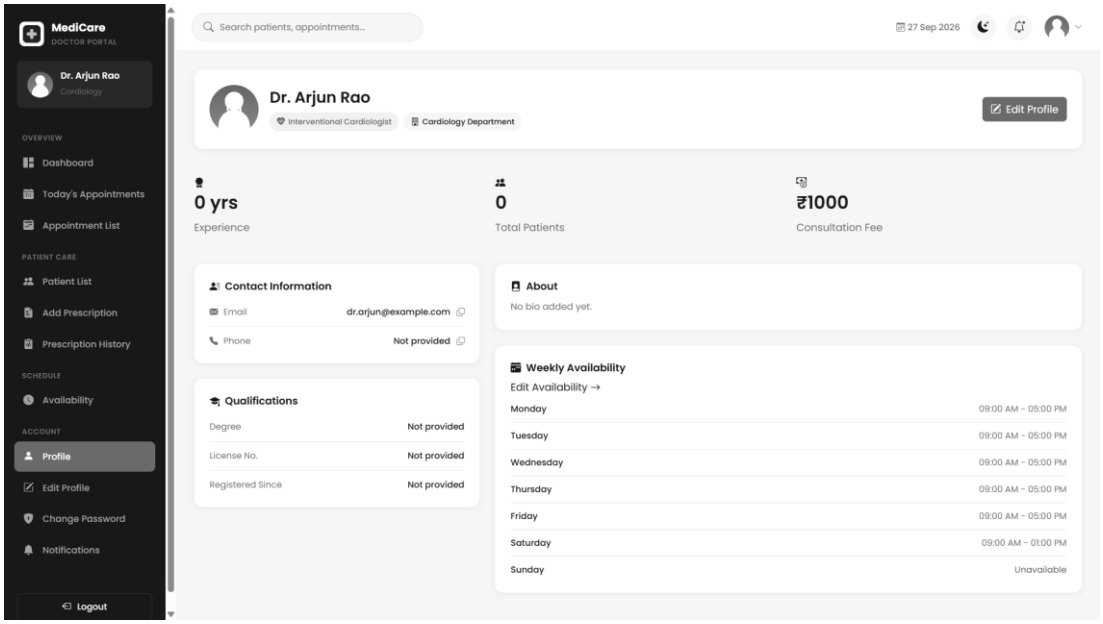


Figure 36: Doctor professional profile — supplied project screenshot

Receptionist dashboard

Permitted users: Receptionist.

The front desk dashboard summarises daily operational activity and presents shortcuts to patient registration, booking and billing. It is intended for fast coordination during a visit.

Validation and navigation: Dashboard counts are summaries, not proof of payment collection or attendance. Detailed appointment and bill states remain the source for individual actions.

Template: templates/receptionist/dashboard.html

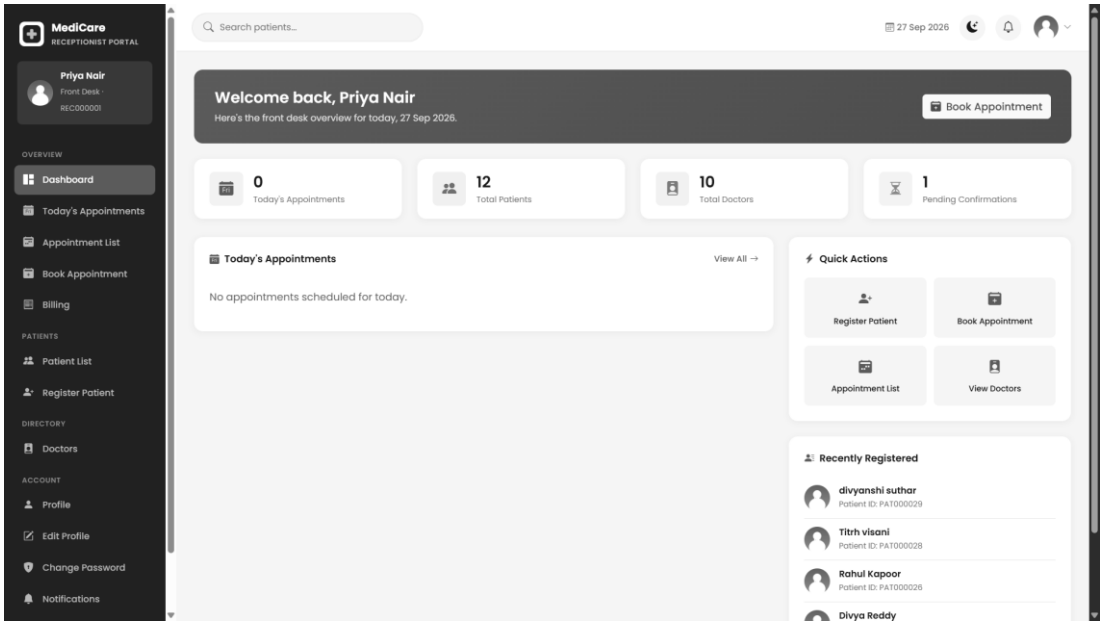


Figure 37: Receptionist dashboard — supplied project screenshot

Walk in patient registration

Permitted users: Receptionist.

The registration screen allows a receptionist to create a patient account and profile for assisted service. The profile retains the registering receptionist reference.

Template field names: full_name, email, phone, password, confirm_password, date_of_birth, gender, blood_group, address, city, state, pincode, emergency_contact_name, emergency_contact_number, allergies, medical_history.

Visible actions: Register Patient.

Validation and navigation: The handler validates the submitted name, email and passwords and must avoid duplicate account identity. This workflow was exercised by an existing automated test.

Template: templates/receptionist/register_patient.html

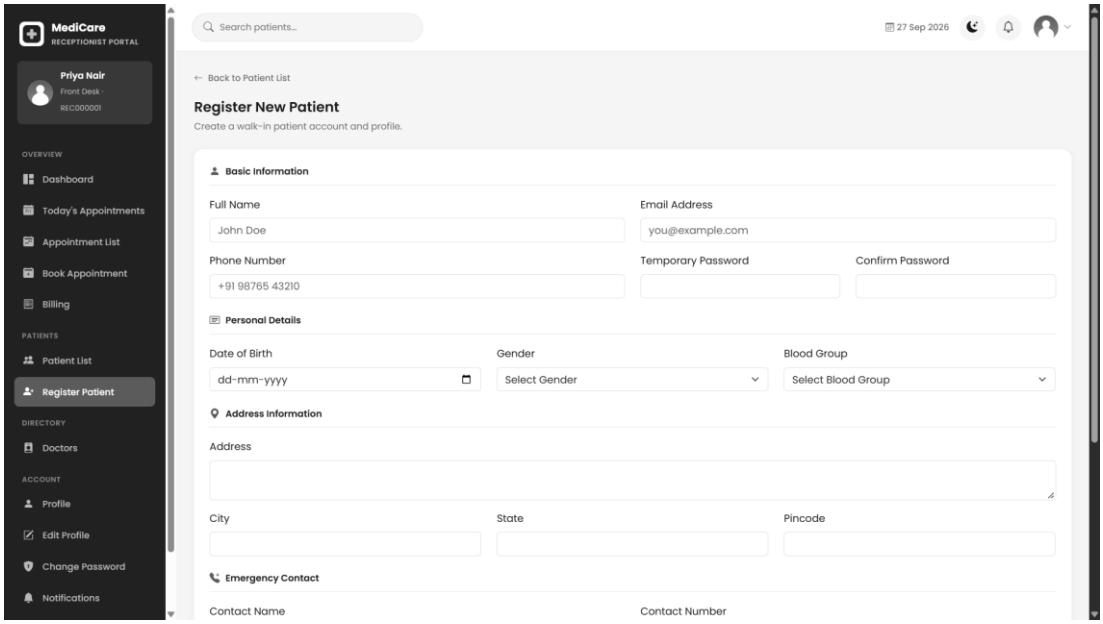


Figure 38: Walk in patient registration — supplied project screenshot

Front desk patient search

Permitted users: Receptionist.

The operational patient list supports finding existing records before registering another person. Detail navigation connects registration data with visits and bills.

Template field names: q.

Validation and navigation: Search should be used to reduce accidental duplicate real world identities, although the application does not implement a full patient matching service.

Template: templates/receptionist/patient_list.html

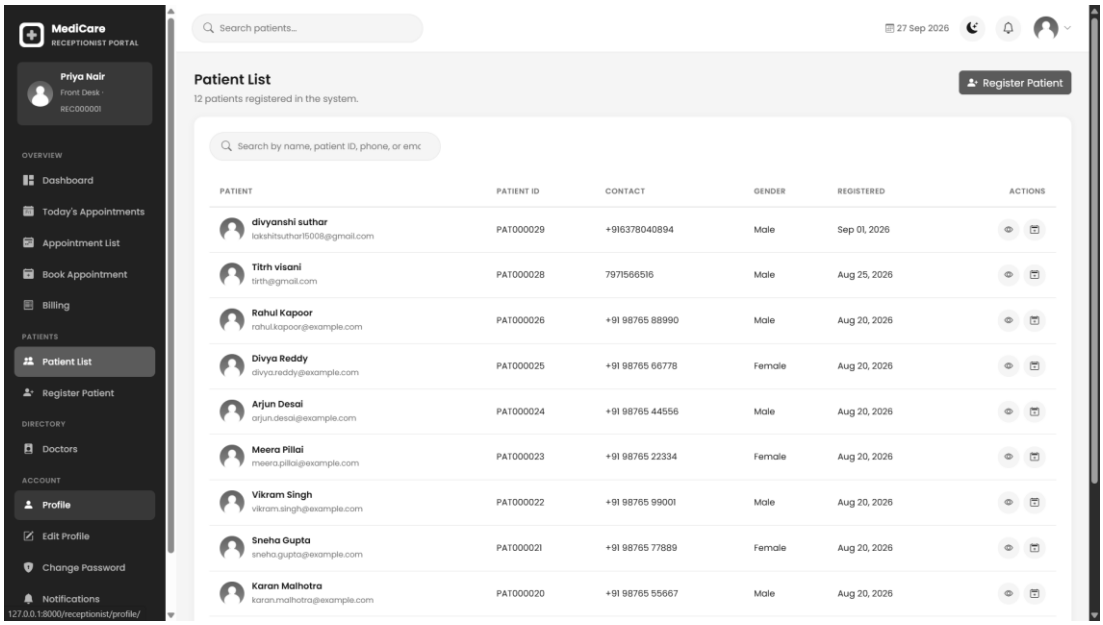


Figure 39: Front desk patient search — supplied project screenshot

Assisted booking form

Permitted users: Receptionist.

The form searches a patient and selects doctor, date and slot. A patient may be preselected from a detail page. The server creates a confirmed appointment after the locked availability check.

Template field names: patient, doctor, appointment_date, visit_type, reason.

Visible actions: Confirm Appointment.

Validation and navigation: The consultation bill starts pending. This differs from the simulated paid bill in patient checkout and is important when demonstrating front desk payment.

Template: templates/receptionist/book_appointment.html

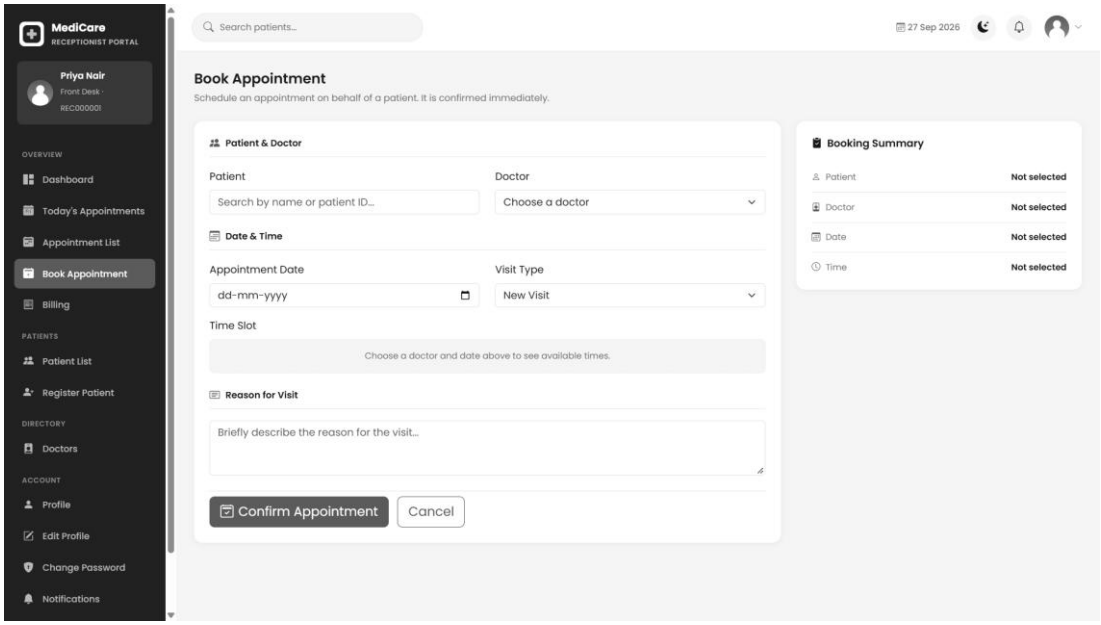


Figure 40: Assisted booking form — supplied project screenshot

Front desk appointment list

Permitted users: Receptionist.

The list gives reception a hospital operational view and status actions. Confirmation, cancellation and no show forms retain a safe next target so the user can return to the appropriate list.

Template field names: q, status, next.

Visible actions: Keep Appointment; Yes, Cancel.

Validation and navigation: Acceptance from both today and all appointments is covered by the automated redirect regression test. Repeated no show handling is checked to avoid another fee.

Template: templates/receptionist/appointment_list.html

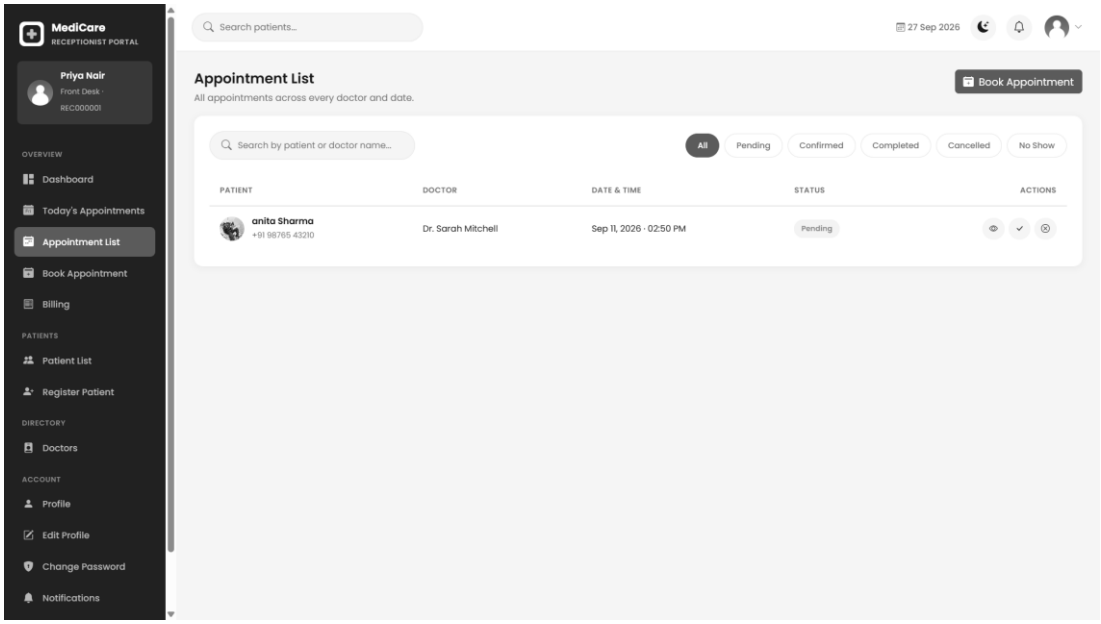


Figure 41: Front desk appointment list — supplied project screenshot

Front desk doctor directory

Permitted users: Receptionist.

Doctor cards provide professional and department information useful during assisted booking. The page is a selection aid rather than a doctor account creation form.

Validation and navigation: The corresponding list view and template determine search and available actions. Only administrators have the staff onboarding routes.

Template: templates/receptionist/doctor_list.html

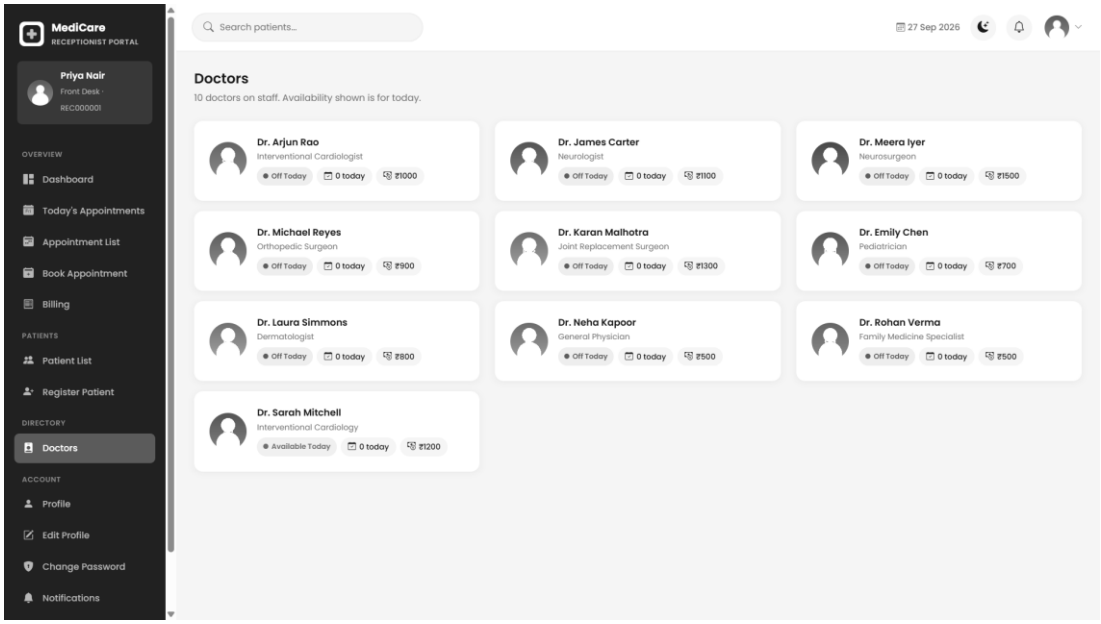


Figure 42: Front desk doctor directory — supplied project screenshot

Front desk bills

Permitted users: Receptionist.

The bill list helps reception find outstanding charges, record paid status and open a print friendly receipt. Search supports the implemented patient fields.

Template field names: q, next.

Validation and navigation: Mark paid records an operational status and timestamp. It does not authorise an external card charge or perform financial reconciliation.

Template: templates/receptionist/billing_list.html

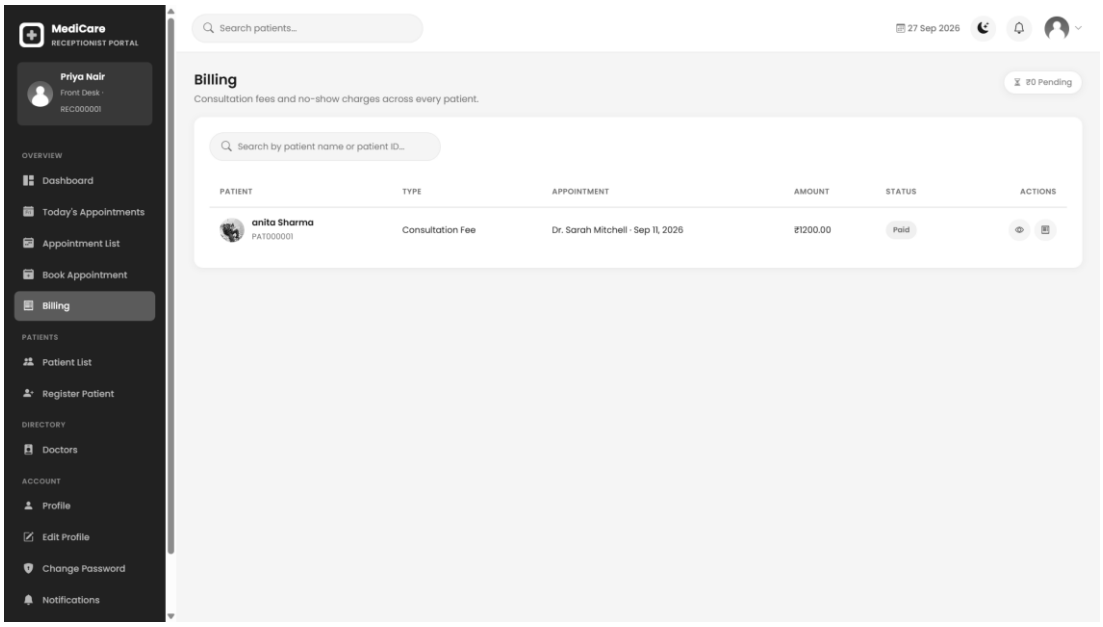


Figure 43: Front desk bills — supplied project screenshot

Shared notification design

Permitted users: All roles through separate routes.

Notification lists show messages for the current account, with read and dismiss actions. Role templates reuse the same underlying Notification data structure with their own navigation.

Visible actions: All; Mark all as read.

Validation and navigation: Context processors add unread counts. Actions must remain scoped to request.user, including an identifier supplied directly rather than selected from the screen.

Template: templates/receptionist/notifications.html

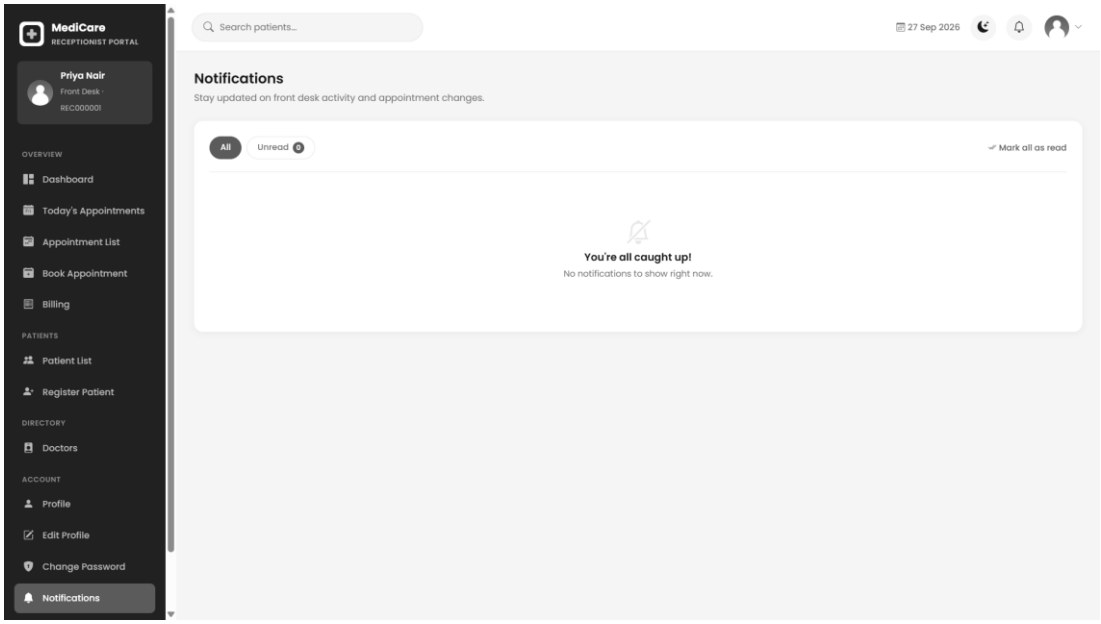


Figure 44: Shared notification design — supplied project screenshot

Shared staff profile and password design

Permitted users: Staff through their own portals.

Staff profile pages display role specific contact and professional fields and link to edit and password forms. Password handlers verify current credentials and use framework password validation.

Validation and navigation: A password change should preserve the intended current session behaviour while preventing a wrong old password from changing the account. Upload and name checks apply in the specific edit handlers.

Template: templates/receptionist/profile.html

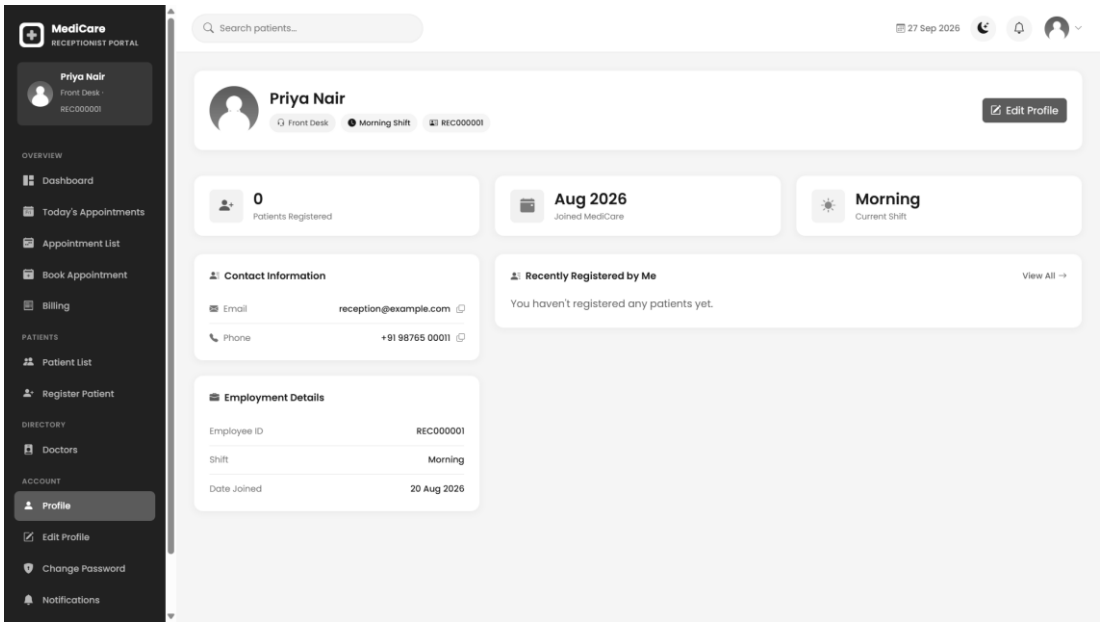


Figure 45: Shared staff profile and password design — supplied project screenshot

Other supporting screens

Table 66: Supporting interface inventory

Screen group	Current templates	Design role
Public information	patient/index.html, about.html, services.html, departments.html	Information and navigation; static marketing text is not evidence of clinical modules.
Patient notifications	patient/notifications.html	Own notifications using the shared message structure.
Today appointments	doctor/today_appointments.html; receptionist/today_appointments.html	Focused daily lists with the relevant role actions.
Patient details	doctor/patient_details.html; receptionist/patient_detail.html; adminpanel/patient_detail.html	Role specific record context and history.
Staff profile editing	doctor/edit_profile.html; receptionist/edit_profile.html; adminpanel/edit_profile.html	Editable fields for the logged in staff account.
Password forms	doctor/change_password.html; receptionist/change_password.html; adminpanel/change_password.html	Current password, new password and confirmation.
Receipt pages	patient, receptionist and adminpanel billing_receipt.html	Browser print output for a permitted bill.
Email templates	patient/emails/*.html	OTP, reset, booking, cancellation and reminder message layouts.

3.4 Report Design

Formal management reports are implemented in adminpanel/report_views.py. The administrator report route imports this module explicitly. It prepares headers and rows for one selected category, then returns an HTML result or an attachment according to the format query parameter. Reports do not use a separate historical reporting database.

The design has four report categories: patients, appointments, doctors and billing. Each category has its own queryset, fields and applicable filters. Date interpretation differs by category. Patient ranges refer to patient creation date; appointment ranges refer to appointment date; billing ranges refer to the linked appointment date. The doctor report applies department and search but does not apply the supplied date range to its queryset.

Patient registration report

Table 67: Patient registration report specification

Design attribute	Specification
Intended user	Administrator
Data source	Patient joined to User
Displayed fields	Patient ID; Patient Name; Email; Phone; Created At

Design attribute	Specification
Applicable filters	Start and end creation dates; search over identifier name email and phone
Output formats	HTML, CSV and PDF
Interpretation	Used to review registrations and patient contact records. There is no patient clinical diagnosis column in this report. Date values are formatted as day-month-year in prepared rows.

Appointment report

Table 68: Appointment report specification

Design attribute	Specification
Intended user	Administrator
Data source	Appointment joined to patient User and doctor User
Displayed fields	Patient; Doctor; Department; Date; Time; Visit Type; Status
Applicable filters	Appointment date range; status; department; doctor; search
Output formats	HTML, CSV and PDF
Interpretation	Used to review planned and historical consultations. A cancelled row remains part of history unless the chosen filter excludes it. Status values describe appointment workflow rather than bill collection.

Doctor report

Table 69: Doctor report specification

Design attribute	Specification
Intended user	Administrator
Data source	Doctor joined to User
Displayed fields	Doctor Name; Department; Specialization; Qualification; Experience; Fee
Applicable filters	Department; search over names email specialization and qualification
Output formats	HTML, CSV and PDF
Interpretation	Used to review staff professional information and current fees. Experience is rendered with a years suffix. Date inputs do not filter this branch, even if present in the form or summary.

Billing report

Table 70: Billing report specification

Design attribute	Specification
Intended user	Administrator

Design attribute	Specification
Data source	Billing joined to patient User and appointment doctor User
Displayed fields	Patient; Patient ID; Bill Type; Doctor; Date; Amount; Status
Applicable filters	Linked appointment date range; billing status; appointment department; doctor; bill type; search
Output formats	HTML, CSV and PDF
Interpretation	Used to inspect charges and their statuses. The total is Sum(amount) for all matched bills, not automatically only paid revenue. The displayed date comes from the appointment, not bill creation or payment time.

HTML result and export consistency

The result template receives headers, rows, total record count, filter summary and the optional billing total. Export requests rebuild the selected report using the supplied query values. Consequently, filter parameters must be preserved by links when switching from the browser result to a download. A report test should compare identity and ordering of rows rather than only checking that a download responds with status 200.

CSV output declares text/csv with UTF-8 encoding and writes a byte order mark before the header row. That improves opening some non-ASCII text in spreadsheet software. Values are written using csv.writer. Spreadsheet formula injection handling is not explicit in the current report exporter, so exporting arbitrary user text deserves a separate input and output safety test.

PDF output uses ReportLab with landscape A4, specified column widths, wrapped cell paragraphs, repeating table headings, filter information, generated time and page numbers. The helper escapes ampersands and angle brackets before constructing cell Paragraph objects. An empty result has a dedicated no records row. The PDF is returned as an attachment, not stored as a new domain model.


Hospital Management System

Patients Report

Filters	Date: 2026-09-01 to 2026-09-27
Generated	27 Sep 2026, 01:43 PM
Total Records	1

Patient ID	Patient Name	Email	Phone	Created At
PAT000029	divyanshi suthar	lakshitsuthar15008@gmail.com	+916378040894	01-09-2026

Figure 46: Supplied example of an administrator PDF output

Receipts and clinical print views

Billing receipts are browser print pages associated with a single bill. Patient receipt retrieval includes patient ownership filtering, while front desk and administrator routes provide wider

authorised operational access. A print button and print CSS control which navigation elements appear on paper. This is distinct from the server generated management PDF reports.

Doctor patient details and prescription history include print oriented presentation in the existing templates. The current routing does not establish a standalone patient prescription PDF download. A future clinical print enhancement should define document identifiers, author attribution, appointment context and revision behaviour rather than implying those controls already exist.

Table 71: Other implemented output views

Output	Source	Key interpretation rule
Bill receipt	Billing with patient and appointment context	A paid label is a saved status; gateway settlement is outside current scope.
Doctor patient detail	Related patient, own appointments and own prescriptions	Doctor ownership must be enforced before printing.
Prescription history print	Current doctors prescription records and medicines	No separate patient prescription route is established.
Dashboard charts	Aggregated domain queries	A summary chart is not a formal audited financial statement.
Notification list	Notification scoped to User	Read state and message delivery are different concepts.

4 Development and Implementation

Development centres on the Django request cycle and a shared relational model. The application is organised by portal and domain rather than by a separate frontend build and API server. Most business logic is in function based views. Templates render the results and use JavaScript for local validation, dynamic rows, charts and selected JSON enquiries.

The implementation examples in this chapter are short extracts from the current source. Each example identifies its file and explains input, behaviour and system role. They are selected to expose important design decisions and limitations rather than reproduce large source files.

4.1 Development

Environment and dependency management

The repository includes manage.py, the config package and an existing Windows virtual environment. requirements.txt is the declared dependency baseline. The report records these pins as project configuration, not as advice to upgrade to a newer version. Source compatibility and a reproducible installation should be checked before changing any dependency.

Django supplies request handling, templates, ORM, authentication, sessions, messages and administration. mysqlclient supports the documented local MySQL setup. psycopg2-binary supports the PostgreSQL deployment option. dj-database-url parses the configured connection URL and python-decouple reads environment settings. Pillow validates uploaded image content. WhiteNoise supports static file delivery, Gunicorn appears in the Linux hosting plan and ReportLab generates management PDFs.

The current environment passed pip check during this documentation task. That verifies installed dependency relationships; it does not prove that the environment exactly matches every pin. The project context previously recorded a ReportLab version mismatch between installed and declared versions, which remains a reproducibility item until a clean install and export check are completed.

Table 72: Dependency pins read from requirements.txt

Package	Declared version
Django	6.1
asgiref	3.12.1
sqlparse	0.5.5
packaging	26.3
mysqlclient	2.2.8
psycopg2-binary	2.9.12
dj-database-url	3.1.2
python-decouple	3.8
pillow	12.3.0
tzdata	2026.3
whitenoise	6.12.0

Package	Declared version
gunicorn	26.0.0
reportlab	4.4.4

Folder structure and responsibilities

Table 73: Source organisation

Path	Implementation responsibility
manage.py	Command entry point for checks, migrations, tests and development server.
config/settings.py	Apps, middleware, templates, database, storage, auth, timezone and mail.
config/urls.py	Public, staff and framework administration entry routes.
accounts/	Custom User and singleton settings models with migrations.
patient/	Public pages, shared authentication, patient profile, booking, billing and mail.
doctor/	Doctor profile, scheduling, appointments, related patients and prescriptions.
receptionist/	Front desk profile, registration, search, booking and billing operations.
adminpanel/	Hospital administration, account policy and routed report generator.
templates/	Public and portal HTML plus email templates.
static/	Original CSS, JavaScript and image assets.
staticfiles/	Generated collected static output; distinct from original assets.
media/	Uploaded profile images and local backup material; not report source data.
deploy/	Ubuntu, Nginx and Gunicorn setup templates.
pagination.py and validation helpers	Common rules imported across portal views.
receptionist/tests.py	Current automated redirect and portal workflow checks.

Initialisation and request processing

manage.py sets the configuration module and invokes Django command execution. WSGI and ASGI entry modules expose the application for server integration. Root routing includes the patient routes at the site root and the other portals under their prefixes. The framework admin has a separate /admin/ path.

Middleware order matters. Security and WhiteNoise precede session and authentication handling; CSRF middleware protects participating unsafe requests; message middleware supports validation feedback; clickjacking middleware provides the configured response protection. The request reaches a view only after URL matching. A role decorator then decides whether the authenticated user may proceed.

Templates receive view context and global context processor values. Portal notification processors contribute unread counts. Static template tags resolve frontend assets according to storage configuration. Development uses finder based static handling when DEBUG is true, while production configuration selects compressed manifest storage and expects collectstatic output.

Source: config/settings.py — selected declarations

```
AUTH_USER_MODEL = "accounts.User"
LOGIN_URL = 'login'
ROOT_URLCONF = 'config.urls'
```

Input: settings and environment at application startup. Behaviour: Django uses the shared User for authentication, the named login route for protected access and config.urls for dispatch. These declarations connect identity and routing across all portals.

Database configuration and migrations

DATABASE_URL is read through the configuration helper and parsed into DATABASES with a persistent connection age. When absent, the fallback points to the repository SQLite file. The local guide describes MySQL on an alternative port to avoid colliding with other installed database services. Deployment documentation proposes PostgreSQL behind the Linux service stack.

Migrations are the authoritative schema evolution history. Historical model names in older migrations should not be removed merely because the current models differ. The isolated test run applied the complete migration sequence and the current dry run reported no new model changes. These are useful consistency checks, but they do not inspect or alter the live database records.

A deployment procedure should confirm the intended target before migrations and maintain a restorable backup. Schema consistency should be checked after release. Database users should be configured for the intended environment rather than copied from local development convenience settings.

Source: config/settings.py

```
DATABASES = {
   'default': dj_database_url.parse(DATABASE_URL, conn_max_age=600)
}
```

Input: a database connection URL supplied through configuration. Output: a Django database configuration mapping. The parser supports selecting the backend without replacing query code throughout the views.

Shared authentication

Input: POST email, password and optional selected role. The account activation behaviour is delegated to the configured authentication backend.

The handler calls authenticate, rejects a selected role mismatch and checks OTP lockout. It reads the singleton OTP setting before deciding whether to call auth_login immediately or store pending_otp_user_id for verification.

Output and system connection: The output is a login error, the OTP page or the account role dashboard. This is the shared authentication entry for all four portals.

Source: patient/views.py:275

```
user = authenticate(request, username=email, password=password)


if user is not None:
    if selected_role and user.role != selected_role:
        messages.error(request, f'This account is not registered as {selected_role.capitalize()}. Please select the correct role and try again.')
        return redirect('login')
```

OTP expiry and attempt handling

Input: A pending user identifier from the session and a submitted otp_code. Expiry and lockout durations come from settings.

The view computes expiry from otp_created_at, rejects missing or expired state and counts wrong codes. At the threshold it clears the code and pending session marker and sets otp_locked_until. Correct verification clears the code before authenticating.

Output and system connection: A successful code produces one authenticated session. The implementation stores the current code directly on User; hashing, resend throttling and broader authentication rate limits are future hardening work.

Source: patient/views.py:339

```
entered_code = request.POST.get('otp_code', '').strip()
# Expiry = the moment the code was generated, plus the configured
# validity window. `expiry` is None if no code was ever generated.
expiry = user.otp_created_at + timedelta(minutes=settings.OTP_VALID_MINUTES) if user.otp_created_at else None


if not user.otp_code or not expiry or timezone.now() > expiry:
    messages.error(request, 'This code has expired. Please request a new one.')
```

Patient account creation

Input: POST full name, email, password confirmation and supported profile fields.

The registration handler is restricted by design to creating patient identities. It validates its submitted fields and creates the shared User plus Patient profile. Staff creation remains in the administrator module.

Output and system connection: The accepted patient proceeds through the configured login verification path. A reproducibility test should check duplicate email, invalid name, password rules and rollback if profile creation fails.

Source: patient/views.py:442

```
if request.method == 'POST':
    full_name = request.POST.get('full_name')
    email = request.POST.get('email')
    phone = request.POST.get('phone')
    dob = request.POST.get('date_of_birth')
    gender = request.POST.get('gender')
    password = request.POST.get('password')
    confirm_password = request.POST.get('confirm_password')
```

Password reset token handling

Input: URL uidb64 and token plus a replacement password submission.

The handler decodes the identity and uses the framework token generator. A valid link enables password replacement subject to validation. The configured token timeout is fifteen minutes.

Output and system connection: The account password changes only through an accepted token and validated input. External mail delivery and link construction need separate environment tests.

Source: patient/views.py:554

```
try:
    uid = force_str(urlsafe_base64_decode(uidb64))
    user = User.objects.get(pk=uid)
except (TypeError, ValueError, OverflowError, User.DoesNotExist):
    user = None


token_valid = user is not None and default_token_generator.check_token(user, token)
if not token_valid:
```

Patient profile editing

Input: The authenticated patient, POST profile values and optional uploaded file.

The view obtains the current profile, validates supported values and calls the shared image validator before saving an uploaded profile image. It combines personal, contact, emergency and background information.

Output and system connection: The resulting HTML reflects the current patients record. Storing a file also requires storage and database consistency checks if a later save fails.

Source: patient/views.py:632

```
if request.method == 'POST':
    first_name = request.POST.get('first_name')
    last_name = request.POST.get('last_name')
    email = request.POST.get('email')
```

```
    if not all([first_name, last_name, email]):
        messages.error(request, 'Please fill in all required fields.')
        return redirect('patient_profile')
```

Date specific slot JSON

Input: GET doctor identifier and ISO formatted date.

The view retrieves the doctor, parses the date and returns a JsonResponse from the common slot method. Date parse failures produce a response with available false and status 400.

Output and system connection: The browser can disable unavailable choices before submission. This response is an enquiry and does not reserve a slot or replace final server validation.

Source: patient/views.py:747

```
doctor_id = request.GET.get('doctor')
date_str = request.GET.get('date')


doctor = get_object_or_404(Doctor, id=doctor_id)
try:
    appointment_date = datetime.strptime(date_str, '%Y-%m-%d').date()
except (TypeError, ValueError):
    return JsonResponse({'available': False, 'reason': 'Invalid date.', 'slots': []}, status=400)
```

Session based booking draft

Input: POST doctor, date, time, department, visit type and reason.

The first booking step checks the doctor and date and compares the selected label with current slots. It stores accepted values under pending_appointment in the session and redirects to fee review.

Output and system connection: The output is a draft selection, not an Appointment row. A stale draft may fail final checkout and must not be counted as a completed booking.

Source: patient/views.py:799

```
availability = doctor.get_available_slots(parsed_date)
if not availability['available']:
    messages.error(request, availability['reason'] or 'Doctor is unavailable on this date.')
    return redirect('book_appointment')


matching_slot = next((s for s in availability['slots'] if s['time'] == time_slot), None)
if matching_slot is None or matching_slot['booked']:
    messages.error(request, 'This slot is already booked.')
    return redirect('book_appointment')
```

Review without persistence

Input: pending_appointment session data and the current doctor fee.

The view redirects to booking if the draft is absent. Otherwise it resolves the doctor and formats the stored selection for the confirmation template. The proceed action opens the checkout page.

Output and system connection: No Appointment or Billing row is created in this view. Its purpose is user review before the final POST operation.

Source: patient/views.py:837

```
pending = request.session.get('pending_appointment')
if not pending:
    messages.error(request, 'Please choose a doctor, date, and time slot first.')
    return redirect('book_appointment')


doctor = get_object_or_404(Doctor, id=pending['doctor_id'])
parsed_date = datetime.strptime(pending['appointment_date'], '%Y-%m-%d').date()
```

Final booking and transaction boundary

Input: A patient session draft and checkout POST. Card styled values are not used for gateway processing.

The atomic block locks the doctor, recalculates slots and creates the Appointment only if still valid. After that block, the view creates a bill marked paid, creates a notification, attempts mail and clears the pending draft.

Output and system connection: The result is a pending appointment and simulated paid bill. The narrower transaction boundary means a later billing failure can leave the appointment saved; full business atomicity is an enhancement.

Source: patient/views.py:896

```
with transaction.atomic():
    locked_doctor = Doctor.objects.select_for_update().get(id=doctor.id)
    availability = locked_doctor.get_available_slots(parsed_date)
    matching_slot = next((s for s in availability['slots'] if s['time'] == pending['time_slot']), None)
    if not availability['available'] or matching_slot is None or matching_slot['booked']:
        messages.error(request, 'Sorry, that slot was just taken. Please choose another.')
        del request.session['pending_appointment']
        return redirect('book_appointment')
```

Shared scheduling algorithm

Input: An appointment_date plus the doctors duration, buffer, cap, weekday rules, blocked dates and appointments.

get_available_slots rejects past dates and blocked dates before checking the weekday rule. It uses a 09:00 to 17:00 fallback when no weekday row exists. Existing noncancelled labels are flagged booked; started slots on the current day are skipped.

Output and system connection: The returned mapping contains available, reason and slots. max_per_day caps generated entries, including entries flagged booked. Mixed naive local time and configured UTC remains a boundary concern.

Source: doctor/models.py:111

```
booked_slots = set(
    self.appointments.filter(appointment_date=appointment_date)
    .exclude(status='cancelled')
    .values_list('time_slot', flat=True)
)


# Each slot is slot_duration minutes long, followed by a buffer_time
# gap before the next slot starts.
```

Patient lookup for assisted booking

Input: A q search value from the authenticated receptionist.

The endpoint builds a patient search over its supported identity fields and returns lightweight JSON result entries for selection in the booking interface. This is a role protected helper, not a public patient directory API.

Output and system connection: The result supplies identifiers for a later booking POST. The booking handler must still validate selected records independently of the JSON response.

Source: receptionist/views.py:352

```
query = request.GET.get('q', '').strip()
patients = Patient.objects.select_related('user')
if query:
    patients = patients.filter(
        Q(user__first_name__icontains=query)
        | Q(user__last_name__icontains=query)
        | Q(patient_id__icontains=query)
    )
```

Confirmed front desk booking

Input: Selected patient, doctor, ISO date and a generated free slot label.

The view validates required fields and date, locks the doctors row within transaction.atomic and checks the common availability result. It creates a confirmed Appointment, then a pending consultation bill and patient notification and attempts the booking email.

Output and system connection: This path differs from patient checkout in both appointment and billing status. A repeated sequential slot submission is rejected by the existing automated test; simultaneous engine specific locking is not established by that test.

Source: receptionist/views.py:449

```
with transaction.atomic():
    locked_doctor = Doctor.objects.select_for_update().get(id=doctor.id)
    availability = locked_doctor.get_available_slots(parsed_date)
    if not availability['available']:
        messages.error(request, availability['reason'] or 'Doctor is unavailable on this date.')
        return redirect('receptionist_book_appointment')


    # Find the requested slot among the doctor's available slots for this date,
```

Confirmation and safe return

Input: An appointment identifier and POST next target.

The role protected handler changes supported POST requests to confirmed and adds a notification for the appointment patient. It delegates return navigation to safe_next_redirect instead of allowing an arbitrary relative browser target.

Output and system connection: The view supports confirmation from both today and all appointment lists. GET and wrong role cases in the current test suite leave status unchanged.

Source: receptionist/views.py:517

```
if request.method == 'POST':
    appointment.status = 'confirmed'
    appointment.save()
    Notification.objects.create(
        user=appointment.patient.user,
        notification_type='confirmed',
        message=f'Your appointment with Dr. {appointment.doctor.user.get_full_name()} on
{appointment.appointment_date} has been confirmed.',
    )
```

Attendance fee handling

Input: An appointment identifier and a protected POST action.

The handler rejects already terminal or no show states before applying the no show transition. It creates a separate no_show_fee Billing record using the doctor consultation fee and records the related notification.

Output and system connection: The consultation bill is not replaced by the fee. The existing repeated action test confirms that another fee is not added in the checked sequence.

Source: receptionist/views.py:570

```
if request.method == 'POST':
```

```
    if appointment.status in ('cancelled', 'completed', 'no_show'):
        messages.error(request, 'This appointment cannot be marked as a no-show.')
    else:
        appointment.status = 'no_show'
        appointment.save()

        # No-show fee = the doctor's standard consultation fee (not a distinct,
```

Payment status update

Input: A bill identifier, current role and optional next destination.

The action retrieves the bill and, on POST, changes status to paid and sets paid_at with timezone.now. Safe return navigation leads back to the operational list.

Output and system connection: This is a recording operation. It does not authorise, capture or reconcile an external payment. The administrator module has a corresponding handler.

Source: receptionist/views.py:646

```
if request.method == 'POST':
    bill.status = 'paid'
    bill.paid_at = timezone.now()
    bill.save()
    messages.success(request, 'Bill marked as paid.')
return safe_next_redirect(request, 'receptionist_billing_list')
```

Object level clinical access

Input: The logged in doctor and a patient identifier from the route.

The view retrieves a distinct patient only when an appointment relation matches the current doctor. It prepares the doctors appointment history and own prescriptions with medicine prefetching. Age is calculated from date of birth when supplied.

Output and system connection: An unrelated guessed patient identifier cannot retrieve that detail through this view. The output is a template context suitable for the authorised clinical page.

Source: doctor/views.py:261

```
patient = get_object_or_404(Patient.objects.distinct(), id=patient_id, appointments__doctor=doctor)


history = Appointment.objects.filter(doctor=doctor, patient=patient).order_by('-appointment_date')
prescriptions = Prescription.objects.filter(doctor=doctor, patient=patient).prefetch_related('medicines')


# Compute age in whole years from date_of_birth: start with the year
# difference, then subtract 1 if this year's birthday hasn't happened
```

Prescription and medicine assembly

Input: Patient identifier, required diagnosis, optional appointment and repeating medicine arrays.

The view checks the patient to doctor relationship and any supplied appointment pair. It creates the prescription and then one medicine row for each nonblank name, using empty optional values where a parallel array is shorter.

Output and system connection: The output is a stored clinical header and details plus a patient notification. A surrounding transaction and stronger array and choice validation would improve failure handling.

Source: doctor/views.py:325

```
if appointment_id:
    # Confirm the linked appointment actually belongs to this
    # doctor/patient pair -- otherwise a tampered POST could
    # attach the new prescription to someone else's appointment.
    if not Appointment.objects.filter(id=appointment_id, doctor=doctor, patient=patient).exists():
        messages.error(request, 'That appointment does not belong to this patient.')
        return redirect('add_prescription')
```

Clinical history retrieval

Input: The current doctor and supported query filters.

The queryset is restricted to prescriptions authored by the doctor and uses related object loading for patient identity and medicine details. The template presents the history and print oriented output.

Output and system connection: This is not a patient prescription API. Search and date edge cases should be checked against exactly the queryset conditions in the view.

Source: doctor/views.py:403

```
doctor = get_doctor(request)
query = request.GET.get('q', '').strip()
sort = request.GET.get('sort', 'newest')


prescriptions =
Prescription.objects.filter(doctor=doctor).select_related('patient__user').prefetch_related('medicines')


if query:
    prescriptions = prescriptions.filter(
```

Weekly schedule maintenance

Input: Weekday checkbox and time values, scheduling parameters and parallel blocked date/reason arrays.

The view validates new blocked dates and active weekday time ranges before writing weekday rows. It updates duration, buffer and cap, then deletes and recreates the doctors blocked date rows from the submitted list.

Output and system connection: The display defaults use shorter Saturday hours and Sunday closure when unsaved, while the slot method defaults to open 09:00 to 17:00 for any missing weekday. Unifying those rules is a specific proposed correction.

Source: doctor/views.py:486

```
for day in days:
    if request.POST.get(f'available_{day}') != 'on':
        continue
    start_time = request.POST.get(f'start_{day}') or '09:00'
    end_time = request.POST.get(f'end_{day}') or '17:00'
    try:
        parsed_start = datetime.strptime(start_time, '%H:%M').time()
        parsed_end = datetime.strptime(end_time, '%H:%M').time()
```

Administrative doctor onboarding

Input: Doctor account credentials plus department and professional profile values.

The administrator view validates supplied identity and numeric fields and creates the doctor role account and linked Doctor data. The resulting profile is shared by directories, scheduling and report queries.

Output and system connection: The creation result must preserve agreement between User.role and the matching profile. Account onboarding should be tested for duplicate identity and partial failure.

Source: adminpanel/views.py:215

```
if request.method == 'POST':
    full_name = request.POST.get('full_name')
    email = request.POST.get('email')
    password = request.POST.get('password')
    confirm_password = request.POST.get('confirm_password')
    department = request.POST.get('department')


    if not all([full_name, email, password, confirm_password, department]):
```

Administrative state correction

Input: An appointment identifier and status value.

On POST the handler checks the submitted value against Appointment.STATUS_CHOICES and saves an accepted status. The queryset is hospital wide because this is the administrator oversight route.

Output and system connection: It can correct a completed visit back to another valid status. It does not create the fee or communication side effects of the specialised receptionist workflows.

Source: adminpanel/views.py:571

```
appointment = get_object_or_404(Appointment, id=appointment_id)
if request.method == 'POST':
    status = request.POST.get('status')
    if status in dict(Appointment.STATUS_CHOICES):
        appointment.status = status
        appointment.save()
        messages.success(request, f"Appointment status updated to {appointment.get_status_display()}.")
return safe_next_redirect(request, 'admin_appointment_list')
```

Runtime OTP policy

Input: The singleton settings record and presence of the checkbox in POST.

The handler uses SystemSettings.get_solo and stores whether otp_login_enabled was submitted. It returns to the same screen with a message describing the saved state.

Output and system connection: The next authentication flow reads this database value. Updating the checkbox affects every role and is not limited to the administrator who clicked it.

Source: adminpanel/views.py:672

```
settings_obj = SystemSettings.get_solo()


if request.method == 'POST':
    settings_obj.otp_login_enabled = 'otp_login_enabled' in request.POST
    settings_obj.save()
    state = 'enabled' if settings_obj.otp_login_enabled else 'disabled'
    messages.success(request, f'Two-step (OTP) login verification has been {state}.')
    return redirect('admin_security_settings')
```

Account activation control

Input: Target User identifier and a protected POST action.

The handler refuses all targets whose role is admin, including the acting administrator. A nonadmin target has is_active toggled. Safe return navigation resolves the optional next value.

Output and system connection: The operation preserves the account and profile rather than deleting them. Existing authenticated session behaviour after deactivation deserves an explicit acceptance test.

Source: adminpanel/views.py:710

```
if request.method == 'POST':
```

```
    if target.role == 'admin':
        messages.error(request, 'Administrator accounts cannot be deactivated from here.')
    else:
        target.is_active = not target.is_active
        target.save()
        state = 'activated' if target.is_active else 'deactivated'
        messages.success(request, f'{target.get_full_name()} has been {state}.')
```

Filtered output preparation

Input: GET report_type, range, status, department, doctor, bill_type, q and format.

The active generator validates the report type and reversed range, builds one category queryset and prepares headers and rows. csv writes a UTF-8 attachment; pdf calls _build_pdf; other requests render the result template.

Output and system connection: The same row preparation drives all three outputs. Applicable filters differ by branch, and exports currently materialise rows in memory.

Source: adminpanel/report_views.py:187

```
report_type = request.GET.get('report_type', '').strip()
start_date = request.GET.get('start_date', '').strip()
end_date = request.GET.get('end_date', '').strip()
status = request.GET.get('status', '').strip()
department = request.GET.get('department', '').strip()
doctor_id = request.GET.get('doctor', '').strip()
bill_type = request.GET.get('bill_type', '').strip()
search = request.GET.get('q', '').strip()
export_format = request.GET.get('format', '').strip().lower()
```

ReportLab document construction

Input: Report category, headers, prepared rows, filter summary and optional total.

The helper creates a landscape A4 document, paragraph styles, metadata, a summary table and a repeating header data table. A callback adds page numbers. A no records row handles empty data.

Output and system connection: The helper returns PDF bytes rather than a saved report model. Column widths and long cell wrapping require visual export tests with representative boundary data.

Source: adminpanel/report_views.py:41

```
doc = SimpleDocTemplate(
    buffer,
    pagesize=page_size,
    leftMargin=12 * mm,
    rightMargin=12 * mm,
    topMargin=16 * mm,
```

```
    bottomMargin=16 * mm,
    title=f'HMS {report_type.title()} Report',
    author='Hospital Management System',
)
```

Selected JSON endpoint contracts

Table 74: Internal JSON helpers rather than a public REST API

Endpoint	Inputs	Success output	Failure or access condition
/book-appointment/available-slots/	doctor and date query values	available, reason and slots with time/booked values	Patient role; malformed date returns status 400.
/receptionist/appointments/book/available-slots/	doctor and date query values	Same scheduling result shape for assisted booking	Receptionist role and valid doctor/date.
/receptionist/appointments/book/search-patients/	q query value	results array used by the patient selector	Receptionist role; empty or unmatched search follows handler logic.

The browser uses these JSON helpers to populate form choices. Authentication still uses Django sessions; the repository does not implement token based external API authentication or a documented third party API product. A JSON endpoint is simply one response form within the monolith.

The slot response distinguishes the doctors overall availability from the booked state of individual labels. A date can be generally available while every generated label is already booked. The client should therefore inspect both the top level flag and each row rather than assuming available=true means that a free choice exists.

Shared validation and return navigation

Name validation is implemented by a small regular expression helper that accepts the supported letters, spaces, apostrophes and hyphens. It improves data format checks but does not prove legal identity or support every writing system. Broader multilingual support would require an explicit product decision and suitable tests.

Image validation checks the filename extension, a five megabyte size threshold and Pillow decoded image format. It verifies image content and resets the stream position before storage reads it. The accepted extensions are jpg, jpeg, png and webp and accepted decoded formats are JPEG, PNG and WEBP. An upload rejected by these checks should never be assigned to the profile field.

The return navigation helper first tries to resolve next as a Django route name. It then accepts only an absolute path or HTTP(S) target allowed for the current host and protocol requirement. Unknown route names, unsafe external targets and unrecognised relative paths fall back to a named destination. This avoids malformed concatenated appointment confirmation URLs and external redirection.

Source: safe_redirect.py:safe_next_redirect

```
next_url = request.POST.get('next', '').strip()
    if next_url:
        # Forms may submit a named route rather than a literal URL.
        try:
            next_url = reverse(next_url)
        except NoReverseMatch:
            pass
    if next_url and next_url.startswith(('/', 'http://', 'https://')) and url_has_allowed_host_and_scheme(
        url=next_url,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        return HttpResponseRedirect(next_url)
    return redirect(fallback)
```

Input: the POST next field and a fallback route name. Output: a validated HttpResponseRedirect or a fallback redirect. This helper connects status and billing forms to predictable navigation and is directly covered by three automated test methods.

Pagination and query efficiency

The common paginate helper wraps a queryset in Django Paginator with a default page size of twenty. get_page handles invalid and out of range values, and an elided page range produces compact page controls. Views that call this helper gain bounded row display; other views should not be assumed paginated merely because the helper exists.

select_related is used for foreign key identity and appointment paths, while prefetch_related is used for medicine details where appropriate. These optimisations reduce repeated database access when a template traverses relationships. Query counts and latency were not benchmarked during this task, so no measured performance claim is attached to their presence.

Report exports differ from list pages: they build complete prepared row lists. The PDF helper also constructs table objects in memory. This is acceptable for a small demonstration but should be bounded, streamed or moved to a background job when output volume grows.

Source: pagination.py:paginate

```
    paginator = Paginator(queryset, per_page)
    page_obj = paginator.get_page(request.GET.get('page'))
    elided_page_range = list(paginator.get_elided_page_range(page_obj.number, on_each_side=1, on_ends=1))
    return page_obj, elided_page_range
```

Input: request page query value and a queryset. Output: page_obj and elided_page_range. The caller passes both values into its list template.

Email and reminder implementation

Appointment email helpers construct plain text and HTML alternatives using a common context containing patient first name, doctor name, department, appointment date, slot and visit type. Booking, cancellation and reminder templates share this structure. The appointment send calls use fail_silently=True, so a saved business operation can succeed even when SMTP delivery fails.

The reminder command selects tomorrows appointments with pending or confirmed status and reminder_sent=False, loads related identities, calls the reminder helper and sets reminder_sent=True. Because the helper can fail silently, the current flag indicates that the command processed the row rather than independently verified message delivery. A delivery log and retry policy would make this distinction observable.

The batch reminder launcher is an operational aid. It does not prove that Windows Task Scheduler has an active task or that Linux has a corresponding scheduled job. The documentation task did not invoke the real reminder command because it would send messages and write flags against whichever database was configured.

Source: patient/management/commands/send_appointment_reminders.py

```
appointments = Appointment.objects.filter(
    appointment_date=tomorrow,
    status__in=['pending', 'confirmed'],
    reminder_sent=False,
).select_related('patient__user', 'doctor__user')
```

Input: current framework time and saved appointments. Output: eligible queryset for processing. The command then attempts email and records a processed reminder flag.

Running the application safely

An existing local setup can be checked with the project interpreter and manage.py check. A development server can then be started after the configured database is available. The database guide and environment example should be read privately before selecting a target; credentials, secret keys and patient backups must not be copied into an academic report.

For a fresh installation, create a suitable virtual environment, install requirements, supply environment settings, review the target database, apply migrations and collect static assets as appropriate. Account creation should use a controlled administrative process. Seed scripts are not a necessary part of reproducing the application and should not be executed blindly.

The following commands describe setup activities rather than actions performed on the live database during report creation. In particular, migrations and static collection can change an environment and must be directed to the intended installation.

Source: Local development commands from the repository root

```
.\venv\Scripts\python.exe manage.py check
.\venv\Scripts\python.exe manage.py runserver
```

Table 75: Configuration responsibilities

Environment value	Purpose	Documentation treatment
SECRET_KEY	Framework cryptographic key	Supply a private environment value; no actual key is printed.
DEBUG	Development versus production behaviour	Disable for the intended production configuration.
ALLOWED_HOSTS	Permitted host names	Match the intended deployment domain or local host.
DATABASE_URL	Backend and connection target	Review before schema or data operations; no credential string is included.
EMAIL_BACKEND	SMTP or test delivery selection	Use memory backend for isolated workflow tests.
EMAIL_HOST and related values	Mail transport configuration	Keep authentication details private.
DEFAULT_FROM_EMAIL	Sender identity	Use an appropriate verified sender in deployment.

Deployment architecture and readiness

The deploy directory describes Ubuntu hosting with Nginx forwarding application requests to Gunicorn and serving appropriate assets, while PostgreSQL stores data. A systemd service template manages the application process. This is a deployment plan, not evidence that a public instance was validated during this work.

Before operating a shared service, review allowed hosts, debug mode, secret management, HTTPS termination, secure cookie settings, file permissions and database access. Test migration and rollback procedures with a copy and confirm that static manifest generation matches the templates. Back up media together with the database references that point to it.

Operational validation should include an application health check, a controlled email test, concurrency testing on the real database engine, log review and a restore drill. These checks complement the existing automated SQLite suite. They should not be collapsed into a single claim that deployment is ready because the development server starts.

4.2 Coding Standards

Table 76: Observed practices and proposed standards

Area	Observed convention	Recommended improvement
Naming	Python snake_case functions and fields; PascalCase models	Keep names stable and align route names with their portal.
Organisation	Separate Django apps and role template folders	Move shared business writes into reviewed services as complexity grows.
Modularity	Slot method and common helpers reduce duplication	Use shared forms for repeated validation rather than copied POST parsing.
Comments	Many views have detailed docstrings and workflow comments	Review comments with code; remove stale claims about receipts and booking stages.

Area	Observed convention	Recommended improvement
Formatting	Conventional Python indentation and template blocks	Adopt a formatter and explicit project lint configuration.
Validation	Manual field checks, configured password validators and upload helper	Define consistent choice, numeric and multirow validation.
Errors	Messages, redirects, 404 helpers and selected exception handling	Record operational failures and avoid silently losing required side effects.
API conventions	Small session protected JSON helpers	Document response fields and error codes with contract tests.
Security	Role decorators, CSRF middleware, safe redirects and scoped queries	Expand permission matrix tests, OTP rate limits and deployment hardening.
Version control	Git repository and ignore rules for environment/generated content	Keep secrets and clinical data outside shared commits and review migration changes.

Observed conventions and recommended standards serve different purposes. The existing source already follows ordinary Python naming and Django app organisation, but there is no basis to claim that a particular formatter, linter, coverage threshold or continuous integration gate is enforced. Those can be introduced as explicit project policy in later work.

Comments should explain why a rule exists and remain consistent with executable statements. The source contains examples of comments that lag implementation: the initial booking description mentions an older creation point, and one receipt docstring says the project has no PDF library despite the active ReportLab management exporter. The report therefore explains current statements and routes rather than copying those comments as facts.

Business status changes should be treated as named operations with known side effects. Administrator status correction currently saves an accepted status string, while specialised reception actions can create bills or notifications. A future service layer should make that distinction explicit and prevent accidental bypass of required effects.

Version control practice should preserve source, migrations and meaningful tests while excluding private environment values, uploaded patient material and local database backups. Existing changes in safe_redirect.py and receptionist/tests.py were preserved during this documentation task. Only the report directory and its generation resources were added.

5 Testing

Testing evaluates observable behaviour against requirements. This chapter separates executed checks from proposed acceptance cases. The existing repository suite was executed on 20 October 2026 with DATABASE_URL overridden to sqlite:///:memory: in a dedicated command process. Django created an isolated test database, applied migrations, executed thirteen tests and destroyed the test database.

The existing PortalWorkflowTests class overrides the mail backend to local memory, so its workflow tests do not establish external SMTP delivery. Several tests use force_login, which deliberately bypasses the password and OTP user interaction. A rendered response status of 200 confirms server rendering under the test client; it does not validate a real browsers JavaScript, layout or accessibility.

5.1 Test Cases

Objectives and methodology

The immediate objectives are to verify the existing redirect regression, appointment state changes, sequential duplicate booking rejection, patient registration and search, notification actions and portal rendering. Each automated result below is tied to a named method in receptionist/tests.py. Where a method contains subtests, the report does not inflate them into an unsupported independent test count.

Unit style checks focus on safe_next_redirect with a RequestFactory and SimpleTestCase. Integration style checks use TestCase and the Django client to create users, profiles, appointments and bills and then submit requests through real routes. The fixture builds four role accounts and uses a controlled patient and doctor. Database state assertions supplement response assertions for the checked write operations.

System acceptance requires additional work across complete browser sessions, real authentication, external email and the deployment database. The proposed cases later in this section define that work but are labelled NOT EXECUTED. Source inspection is useful to choose expected results, but it is not a substitute for executing a test.

Table 77: Testing levels and limits

Test layer	Applied method	Evidence or limitation
Unit style	RedirectTests with RequestFactory	Three methods passed for route resolution, unsafe fallback and HTTPS downgrade prevention.
Integration	PortalWorkflowTests with TestCase and test client	Ten methods passed with isolated SQLite and local memory mail.
Validation	Selected malformed date, wrong role and duplicate slot assertions	Limited to the branches asserted by the existing tests.
Server rendering	Role based pages and receipts/details checked for HTTP 200	Does not execute browser scripts or inspect responsive layout.

Test layer	Applied method	Evidence or limitation
Live database concurrency	Proposed simultaneous booking checks	NOT EXECUTED against MySQL or PostgreSQL.
External email and deployment	Proposed acceptance checks	NOT EXECUTED; no real reminder command was run.

Executed automated test record

Table 78: Executed test cases from receptionist/tests.py

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
AT-01	Redirect	test_named_routes_and_safe_urls	POST next containing known route names and a safe local URL	Resolve route names and retain a permitted local path.	All asserted destinations matched.	PASS
AT-02	Redirect	test_invalid_and_external_targets_fall_back	Unknown names, relative path, external and unsafe URL forms	Return the fallback home URL for each rejected value.	All asserted unsafe values used fallback.	PASS
AT-03	Redirect	test_https_does_not_redirect_to_http	Secure request with http://testserver/ next value	Reject an HTTPS to HTTP downgrade and use fallback.	Fallback assertion passed.	PASS
AT-04	Reception appointment	test_receptionist_accept_from_both_lists	Pending visit and next set to each supported receptionist list	Confirm the visit, create notifications and return to the intended list.	Status, redirect chains and notification count assertions passed.	PASS
AT-05	Reception workflow	test_receptionist_cancel_no_show_and_payment	Confirmation state followed by cancellation/no show paths and mark paid	Expected state transitions, a single no show fee and paid timestamp.	All state, fee count and paid_at assertions passed.	PASS
AT-06	Access and method	test_get_and_wrong_role_cannot_accept	GET as receptionist and POST as patient to confirmation	Keep pending status and create no notification.	Unchanged state and notification assertions passed.	PASS
AT-07	Doctor appointment	test_doctor_completion_and_cancellation	Own confirmed visit and supported completion/cancellation actions	Set completed or cancelled and return to intended doctor list.	Status and redirect assertions passed for both action subtests.	PASS
AT-08	Portal rendering	test_portal_pages	force_login for four roles followed by named page requests	Each enumerated page responds with HTTP 200.	All enumerated page response assertions passed.	PASS
AT-09	Patient password navigation	test_patient_password_form_redirect	GET patient password route while logged in as patient	Redirect to patient profile password anchor.	Expected profile anchor URL assertion passed.	PASS

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
AT-10	Booking	test_booking_and_duplicate_slot_rejection	Future free slot, assisted booking payload, repeated payload and invalid date	Create one confirmed booking and fee; reject duplicate slot; invalid date returns 400.	Appointment, bill, notification, duplicate count and 400 assertions passed.	PASS
AT-11	Patient registration	test_patient_registration_and_search	Controlled name, email and matching passwords then name search	Create the patient with registering receptionist and return it in search.	Profile relationship and JSON result assertions passed.	PASS
AT-12	Notifications	test_notification_actions	Current receptionists notification then mark read and dismiss	Persist read flag and delete the selected own notification.	Read and removal assertions passed.	PASS
AT-13	Receipts and details	test_receipts_and_patient_details	Role specific detail and receipt URLs for controlled fixture records	Each enumerated permitted page renders with HTTP 200.	All enumerated response assertions passed.	PASS

Verification commands and observed outcomes

Table 79: Checks actually executed during report preparation

Check	Observed outcome	Meaning
manage.py test --verbosity 2	13 tests; Ran 13 tests in 3.064s; OK	Existing test suite passed in isolated SQLite; runtime is one observation, not a performance benchmark.
manage.py check	System check identified no issues (0 silenced)	Django system checks accepted the loaded configuration.
manage.py makemigrations --check --dry-run	No changes detected	Current model metadata did not request a new migration.
python -m pip check	No broken requirements found	Installed dependency relationships were accepted.

Source: Reproduction commands in a dedicated PowerShell process

```
$env:DATABASE_URL='sqlite:///:memory:'
.\venv\Scripts\python.exe manage.py test --verbosity 2
.\venv\Scripts\python.exe manage.py check
.\venv\Scripts\python.exe manage.py makemigrations --check --dry-run
.\venv\Scripts\python.exe -m pip check
```

The DATABASE_URL override must remain confined to the test process. These commands did not prove that the live MySQL data set or a deployed site was correct, and did not require clearing patient records.

Proposed acceptance cases not executed

Table 80: Proposed acceptance cases with requirement traceability

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
PT-01	Authentication	Login with OTP disabled FR-01, FR-06	Valid credentials, matching role and singleton toggle false	Reach the correct dashboard without an OTP prompt.	Not observed; controlled execution required.	NOT EXECUTED
PT-02	Authentication	Reject a role mismatch FR-01	Patient credentials with doctor selected	Show an error and do not create an authenticated doctor session.	Not observed; controlled execution required.	NOT EXECUTED
PT-03	Authentication	Complete enabled OTP FR-02	Valid credentials and the current controlled mail code	Clear OTP state and establish the correct role session.	Not observed; controlled execution required.	NOT EXECUTED
PT-04	Authentication	Expired code FR-02	A generated code older than configured validity	Reject the code and require a new valid code.	Not observed; controlled execution required.	NOT EXECUTED
PT-05	Authentication	Wrong code lockout FR-03	Three wrong codes for the pending account	Clear code and pending marker and apply a ten minute default lockout.	Not observed; controlled execution required.	NOT EXECUTED
PT-06	Authentication	Resend abuse boundary NFR-01	Repeated resend calls independent of the displayed countdown	Characterise current server behaviour and decide a required rate limit.	Not observed; controlled execution required.	NOT EXECUTED
PT-07	Registration	Duplicate email FR-04	Already used email with otherwise acceptable fields	Reject a duplicate identity without leaving a partial profile.	Not observed; controlled execution required.	NOT EXECUTED
PT-08	Registration	Invalid name and passwords FR-04	Digits or markup in name; mismatched or weak passwords	Reject the relevant invalid value and preserve existing records.	Not observed; controlled execution required.	NOT EXECUTED
PT-09	Password recovery	Invalid and expired reset token FR-05	Altered token and token beyond fifteen minute timeout	Refuse password replacement through the invalid link.	Not observed; controlled execution required.	NOT EXECUTED
PT-10	Profile	Invalid image content FR-07	HTML renamed to .png and a genuine oversized image	Reject invalid decoded content and files exceeding five megabytes.	Not observed; controlled execution required.	NOT EXECUTED

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
PT-11	Profile	Valid image formats FR-07	Small genuine JPEG, PNG and WEBP images	Accept each allowed format and display the stored image.	Not observed; controlled execution required.	NOT EXECUTED
PT-12	Scheduling	Past and blocked dates FR-09, FR-10	Yesterday and a future saved BlockedDate	Return unavailable with no bookable slots.	Not observed; controlled execution required.	NOT EXECUTED
PT-13	Scheduling	Closed weekday FR-10	Future day with is_available false	Return unavailable regardless of a normal time range.	Not observed; controlled execution required.	NOT EXECUTED
PT-14	Scheduling	Unsaved weekday defaults FR-10, NFR-05	Saturday and Sunday without DoctorAvailability rows	Characterise current 09:00 to 17:00 slot fallback and compare the form defaults.	Not observed; controlled execution required.	NOT EXECUTED
PT-15	Scheduling	Duration buffer and cap FR-10	Known duration, buffer and max_per_day on a future open day	Generate the expected spaced labels and stop at the current cap.	Not observed; controlled execution required.	NOT EXECUTED
PT-16	Patient booking	Draft does not reserve FR-11	Initial patient selection POST only	Session contains pending details; no new Appointment yet.	Not observed; controlled execution required.	NOT EXECUTED
PT-17	Patient booking	Abandon a draft FR-11, FR-12	Pending appointment followed by cancel pending action	Remove pending data without a new appointment or bill.	Not observed; controlled execution required.	NOT EXECUTED
PT-18	Patient booking	Stale checkout slot FR-13	Reserve the selected slot through another controlled session before checkout	Reject final checkout and clear the stale draft.	Not observed; controlled execution required.	NOT EXECUTED
PT-19	Patient booking	Demo checkout output FR-13	Free slot followed by checkout POST using demo values	Create pending appointment and paid simulation bill; do not contact a gateway.	Not observed; controlled execution required.	NOT EXECUTED
PT-20	Booking concurrency	Simultaneous final booking FR-13, FR-18, NFR-02	Two concurrent sessions on MySQL or PostgreSQL for one doctor/date/slot	At most one active reservation through supported locked paths.	Not observed; controlled execution required.	NOT EXECUTED

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
PT-21	Booking rollback	Failure after appointment save NFR-02	Inject bill creation failure in an isolated environment	Characterise persisted partial state and use it to define an atomicity improvement.	Not observed; controlled execution required.	NOT EXECUTED
PT-22	Ownership	Foreign patient receipt FR-27, NFR-01	Patient requests another patients bill identifier	Refuse access without disclosing the foreign bill.	Not observed; controlled execution required.	NOT EXECUTED
PT-23	Ownership	Unrelated doctor patient FR-22	Doctor requests a patient without an appointment relationship	Refuse patient detail access.	Not observed; controlled execution required.	NOT EXECUTED
PT-24	Prescription	Required diagnosis FR-23	Valid related patient with empty diagnosis	Reject save and create no clinical header.	Not observed; controlled execution required.	NOT EXECUTED
PT-25	Prescription	Forged appointment pair FR-23	Related patient plus appointment for another doctor or patient	Reject the linked appointment.	Not observed; controlled execution required.	NOT EXECUTED
PT-26	Prescription	Multiple medicine rows FR-24	Three named rows and one empty row with uneven optional arrays	Save only named medicine rows and the supported optional values.	Not observed; controlled execution required.	NOT EXECUTED
PT-27	Prescription rollback	Detail save failure NFR-02	Inject a medicine write failure after header save	Characterise partial state; verify later transaction improvement.	Not observed; controlled execution required.	NOT EXECUTED
PT-28	Availability	Invalid active time range FR-26	End before start on an active weekday	Reject before writing any weekday changes.	Not observed; controlled execution required.	NOT EXECUTED
PT-29	Availability	New blocked date boundary FR-26	Today as a new blocked date and tomorrow as a new blocked date	Reject today; accept tomorrow when otherwise valid.	Not observed; controlled execution required.	NOT EXECUTED
PT-30	Availability	Existing old blocked date FR-26	Resubmit a previously saved date that has become today	Preserve the allowed existing date while saving unrelated changes.	Not observed; controlled execution required.	NOT EXECUTED

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
PT-31	Billing	Repeated paid action FR-28	Mark a paid bill paid again	Characterise timestamp update and define intended idempotency.	Not observed; controlled execution required.	NOT EXECUTED
PT-32	Administration	Reject admin deactivation FR-30	Any admin target including the acting account	Leave is_active unchanged and show the protected target error.	Not observed; controlled execution required.	NOT EXECUTED
PT-33	Administration	Invalid status correction FR-30	POST a value outside Appointment.STATUS_CHOICES	Do not save the invalid state.	Not observed; controlled execution required.	NOT EXECUTED
PT-34	Notifications	Foreign dismissal FR-31	Current user submits another users notification id	Refuse or leave the foreign record unchanged.	Not observed; controlled execution required.	NOT EXECUTED
PT-35	Reports	Cross format row parity FR-32, FR-33	Same filters exported as HTML, CSV and PDF	Same selected row identities, count and relevant totals.	Not observed; controlled execution required.	NOT EXECUTED
PT-36	Reports	Empty result and long fields FR-33	No matching rows; then long names email and medicine-like text	Readable output with no clipping and an explicit empty state.	Not observed; controlled execution required.	NOT EXECUTED
PT-37	Reports	Date interpretation FR-32	Bills with appointment creation and payment dates in different ranges	Billing range follows linked appointment date as currently implemented.	Not observed; controlled execution required.	NOT EXECUTED
PT-38	Reports	Malformed filter values FR-32, NFR-04	Invalid date text, doctor id or incompatible status	Record actual response and improve validation where unhandled.	Not observed; controlled execution required.	NOT EXECUTED
PT-39	Reports	CSV formula safety NFR-01	Export text starting with spreadsheet formula characters	Characterise current exporter and define safe spreadsheet handling.	Not observed; controlled execution required.	NOT EXECUTED
PT-40	Email	Real transport delivery FR-02, FR-35	Controlled test mailbox with correct SMTP configuration	Verify receipt, sender identity and error observability.	Not observed; controlled execution required.	NOT EXECUTED

Test ID	Module	Test Scenario	Input	Expected Result	Actual Result	Status
PT-41	Reminder	Eligibility and flag FR-35	Tomorrow pending and confirmed visits plus excluded states	Only eligible unprocessed rows are attempted; evaluate flags after delivery failure.	Not observed; controlled execution required.	NOT EXECUTED
PT-42	UI	Mobile and keyboard workflow NFR-05	Representative browser widths and keyboard only interaction	Common tasks remain readable and reachable with visible focus.	Not observed; controlled execution required.	NOT EXECUTED
PT-43	Operations	Restore rehearsal NFR-11	Separate database/media backup copy	Restore and verify linked records and profile files without touching live data.	Not observed; controlled execution required.	NOT EXECUTED

Results and observations

The executed evidence supports the redirect correction and selected core portal workflows. In particular, confirmation can return correctly from both receptionist appointment lists, sequential duplicate slots are rejected, repeated no show handling does not add another fee in the checked path, and permitted details and receipts render in the isolated fixture.

The suite is not exhaustive. It does not execute real OTP interaction, browser JavaScript, production email delivery, MySQL row lock contention or large report output. It also does not establish complete branch coverage or prove that all malformed inputs are handled without server errors. The proposed acceptance cases identify concrete gaps rather than representing them as passed.

The appropriate release judgement is that the current automated baseline is passing in its documented test environment. Broader operational approval requires the additional checks selected for the deployment and its data sensitivity. Test evidence should be refreshed after source, dependency or environment changes rather than copied indefinitely from this report.

6 Proposed Enhancements

The proposed enhancements address gaps visible in the implementation and opportunities that follow from the present scope. They are future work. None of the items in this chapter should be interpreted as a feature already delivered by the current source.

Complete transaction units

Group appointment, bill and notification creation within one reviewed transaction, and trigger external messages after commit. Apply the same principle to prescription header and medicine detail creation.

Reason: A failure between separate writes can currently leave incomplete related state. The improvement should include injected failure tests and an explicit retry strategy rather than simply adding a broad try/except.

Completion criterion: Verify that every injected write failure leaves either the complete operation or no operation, and that mail is not sent for a rolled back booking.

Database backed booking integrity

Define a database appropriate invariant for active slot occupancy and retain row locking for the supported workflows. Consider a dedicated slot or reservation model if future scheduling requires multiple resources.

Reason: The current Appointment model lacks a doctor/date/slot unique constraint, and a naive unconditional constraint would conflict with cancelled history. Design must preserve the ability to rebook a cancelled slot.

Completion criterion: Run concurrent requests on the deployment database and confirm at most one active reservation, including cancellation and rebooking sequences.

Unified schedule and timezone rules

Use a shared default schedule for the availability form and slot algorithm, and centralise timezone aware date/time calculations. Validate duration, buffer and cap consistently before saving.

Reason: The unsaved availability form defaults to a shorter Saturday and closed Sunday, while the model fallback opens missing days 09:00 to 17:00. Naive local calculations can disagree with UTC configuration around date boundaries.

Completion criterion: Compare form display and JSON slots for every unconfigured weekday and test hospital midnight boundary cases.

Authentication hardening

Introduce server enforced resend and login limits, consider hashed OTP storage and add a complete password/OTP test suite. Record delivery errors without exposing codes in logs.

Reason: The displayed resend cooldown is not enough to prove a backend rate limit, and existing force_login tests bypass the actual login journey. Optional OTP policy should remain an explicit administrative decision.

Completion criterion: Verify lockout, expiry, repeated resend, replay, inactive accounts and reset link handling through controlled sessions.

Reusable validation and services

Introduce Django forms or equivalent validated input objects for repeated account, schedule, prescription and billing data. Centralise named business operations and their side effects.

Reason: Many views read raw POST dictionaries. Model choices do not automatically validate values during save, and parallel medicine arrays can be inconsistent. Shared services would reduce divergence between portals.

Completion criterion: Use boundary tests for required values, numeric ranges, choices and related identifiers, then compare equivalent workflows across roles.

Patient prescription access

Add an explicitly authorised patient prescription history and download flow with doctor attribution and medicine details. Define what clinical information may be shown and when.

Reason: Current doctor routes provide prescription entry and history, but a dedicated patient prescription route was not found. The enhancement should therefore be implemented and tested before being advertised.

Completion criterion: A patient can retrieve only personal published clinical documents; another patients identifier is rejected.

Real payment integration

Replace the demonstration checkout with a selected gateway integration using server side verification, signed webhooks, idempotency and reconciliation. Define cancellation and refund policy.

Reason: The current card form does not validate, store or charge card data. A paid database status from that flow is an educational simulation. Financial state should be separated from appointment workflow.

Completion criterion: Test sandbox success, declined payment, timeout, duplicate webhook, delayed settlement and refund without storing sensitive card credentials.

Auditing and record retention

Add immutable application audit events for clinical, account, appointment and billing changes, and define retention and archival policies. Prefer controlled deactivation to unreviewed deletion.

Reason: created_at and updated_at record times but not complete actor and prior state history. Cascading deletion can remove substantial related records.

Completion criterion: Reconstruct an authorised state change from the audit trail and confirm that archival and restore preserve required relationships.

Reporting correctness and scale

Validate all filter inputs, show only filters actually applied, distinguish paid revenue from total charges and add bounded or background exports for large data sets. Handle spreadsheet formula prefixes safely.

Reason: The doctor report ignores date inputs, billing ranges use visit dates and matched billing totals are not automatically collected revenue. Exports materialise complete row lists in memory.

Completion criterion: Compare cross format outputs, test long fields and malformed filters, and measure memory and latency at the intended volume.

Observable messaging and background work

Add a delivery record, retry policy and background queue for appointment communication and reminders. Install and monitor an explicit scheduler with documented ownership.

Reason: Appointment email sends can fail silently, and reminder_sent can be set after an unsuccessful attempt. A launcher file alone does not prove a running schedule.

Completion criterion: Simulate transport outage, verify retries and failure visibility, and confirm that only genuinely processed or delivered states receive their intended flag.

Operational deployment and recovery

Automate configuration checks, release tests, health monitoring and database/media backup restoration in a separate environment. Review secure hosting settings and access management.

Reason: The repository contains deployment templates, not a measured uptime result or a completed restore programme. A documented clean dependency installation should resolve installed versus pinned differences.

Completion criterion: Run a reproducible staging release and restore rehearsal and record the actual recovery time and remaining manual steps.

Usability and inclusive access

Perform keyboard, screen reader and small screen testing; improve focus, field errors and terminology based on observed task completion. Consider local asset hosting and broader name support.

Reason: Bootstrap responsiveness and supplied screenshots do not establish accessibility conformance. CDN dependencies and a restricted name expression can affect particular users or environments.

Completion criterion: Complete role based tasks with assistive interaction and record usability defects and verified corrections.

Table 81: Suggested enhancement sequence

Stage	Enhancements to prioritise	Dependency
First	Transactions, schedule consistency, input validation and permission tests	Stable core workflows before adding integrations.
Second	OTP hardening, auditing and reproducible deployment	Clear identity and operational ownership.
Third	Reporting scale, observable messaging and patient prescriptions	Validated data and background processing design.
Later	Real payments and broader service expansion	Approved business rules, sandbox verification and operational support.

7 Conclusion

The Hospital Management System demonstrates how a common relational application can connect outpatient administration and doctor consultation work. The original problem is the effort and inconsistency that can arise when patient identity, doctor schedules, visits and charges are maintained in disconnected places. The implemented solution gives these records explicit relationships and presents them through four role focused portals.

Patients can register, maintain their profiles, discover doctors, select consultation slots and inspect personal appointments and bills. Receptionists can register walk in patients, search existing records, create confirmed visits, manage attendance and record bill payment. Doctors can maintain availability, review related patients and write diagnoses and medicine details. Administrators can manage staff access, inspect hospital wide operational lists, control the OTP policy and generate reports.

The project uses Python and Django with server rendered templates, shared authentication, framework sessions and an environment selected relational database. The model structure separates User from role profiles, appointments from bills and prescription headers from medicine lines. Shared helpers support slot generation, pagination, upload checks and safe return navigation. Management reports support browser display, CSV and ReportLab PDF, while individual receipts use browser print layouts.

Several objectives are demonstrated by specific code and passing test assertions. The existing thirteen tests passed in an isolated SQLite environment on 20 October 2026. They checked safe redirects, selected appointment transitions, sequential duplicate booking rejection, assisted registration and search, notifications and permitted page rendering. Framework checks, the migration dry run and the installed dependency check also completed successfully.

These results have practical limits. They do not establish live MySQL concurrency, browser accessibility, external email reliability or production recovery. The card checkout is simulated and must not be treated as a real financial gateway. Dedicated patient prescription access is a future capability, not an existing route. The current transaction boundaries and direct POST handling also leave clear opportunities for stronger consistency and validation.

The application is useful as an academic demonstration of integrated role based web development and outpatient workflow modelling. Its value lies in the connection between requirements, data design, views, templates and testable outcomes. Further work should first strengthen core integrity, scheduling consistency, authentication and operational observability before expanding the functional scope. A wider deployment should be accepted against controlled end to end and environment specific tests rather than inferred from the existence of complete looking screens.

8 Bibliography

The following references use a consistent numbered format. Repository sources identify implementation evidence; external references identify official technical documentation consulted during report preparation. Web references were accessed on 20 October 2026. They are explanatory resources rather than evidence that every recommended framework practice is already enabled in the project.

[1] Project team. Hospital Management System source repository. Local project snapshot inspected on 20 October 2026.

Primary implementation evidence: accounts, patient, doctor, receptionist, adminpanel, config, templates and shared helper modules.

[2] Project team. requirements.txt and Django migrations. Local repository files, inspected on 20 October 2026.

Evidence for declared dependency versions and schema evolution. The migration check reported no new changes in the isolated environment.

[3] Project team. receptionist/tests.py. Local automated test suite, executed on 20 October 2026.

Evidence for the thirteen named automated results in Chapter 5. Test client and SQLite limits are documented with the results.

[4] Project team. DATABASE.md and deploy/DEPLOY.md. Local setup and hosting guides, inspected on 20 October 2026.

Evidence for local MySQL background and proposed Ubuntu, Nginx, Gunicorn and PostgreSQL hosting.

[5] Project team. HOSPITAL MANAGEMENT SYSTEM UPDATED.docx. Supplied prior report.

Source for college and team identity, the supplied logo and saved application screenshots. Current source takes precedence for feature descriptions.

[6] Django Software Foundation. Django 6.1 documentation. https://docs.djangoproject.com/en/6.1/ . Accessed 20 October 2026.

Official overview consulted for framework concepts and documentation organisation, corresponding to the Django version declared by the project.

[7] Python Software Foundation. Python documentation. https://docs.python.org/3/ . Accessed 20 October 2026.

Official language documentation consulted as a programming reference. The rolling documentation version is distinct from the project interpreter.

[8] Oracle. MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/ . Accessed 20 October 2026.

Official database reference consulted for the database technology identified in the local setup guide.

[9] ReportLab. ReportLab PDF Generation User Guide. https://www.reportlab.com/docs/reportlab-userguide.pdf . Accessed 20 October 2026.

Official PDF library guide consulted for document and table output concepts used in the report implementation.

[10] Pillow contributors. Pillow documentation. https://pillow.readthedocs.io/en/stable/ . Accessed 20 October 2026.

Official image library documentation consulted in connection with uploaded image decoding and validation.–